مستند جامع فنی و آموزشی پروژه

FinOps + OpenCost + Odoo Hub

از صفر مطلق تا یکپارچه‌سازی واقعی **Kubernetes Cost** با **Odoo**



راهنمای گام‌به‌گام + مستند فنی + نقشه تکمیل نهایی	نوع سند
Gateway محلی، Docker/Odoo و OpenCost واقعی راه‌اندازی شده؛ اتصال نهایی Real Data در حال تکمیل	وضعیت پروژه
2026-09-16	تاریخ تدوین

این سند هم مراحل انجام‌شده را مستند می‌کند و هم مسیر باقیمانده را تا تحویل نهایی پروژه مشخص می‌کند.

راهنمای استفاده از این سند

این سند به‌گونه‌ای نوشته شده است که اگر چند هفته بعد هیچ‌کدام از فرمان‌ها، ابزارها یا معماری پروژه را به خاطر نداشتی، بتوانی فقط با همین فایل پروژه را از صفر بالا بیاوری. بخش‌های «انجام‌شده» بازتاب اجرای واقعی روی سیستم هستند و با اسکرین‌شات‌های ثبت‌شده مستند شده‌اند. بخش‌های «ادامه مسیر» برنامه اجرایی پیشنهادی برای تکمیل پروژه‌اند و در متن با برچسب «برنامه» مشخص شده‌اند.

> نکته امنیتی
>
> تصویر صفحه ساخت دیتابیس Odoo شامل Master Password تولیدشده بود. در این مستند رمز از روی تصویر حذف شده است. هیچ رمز واقعی را در گزارش دانشگاهی، Git یا اسکرین‌شات‌های ارائه منتشر نکن.

وضعیت فعلی پروژه در یک نگاه

ردیف	مرحله	وضعیت
1	اجرای Gateway محلی با Python/FastAPI	انجام شده
2	Swagger، API، Dashboard و تست‌های اولیه	انجام شده
3	Docker Desktop + WSL2 + Compose	انجام شده
4	PostgreSQL + Odoo Community 19 + FinOps Connector	انجام شده
5	Sync ماکت Gateway → Odoo و تست عدم Duplicate	انجام شده
6	Kubernetes محلی kind + Prometheus + OpenCost	انجام شده
7	چهار Workload پروژه‌ای در Kubernetes	انجام شده
8	OpenCost UI و Allocation API واقعی	انجام شده
9	provider_mode گیت‌وی = opencost	انجام شده
10	رفع 500 Dashboard ناشی از تقسیم بر صفر	در آستانه انجام

باقیمانده	اولین Sync واقعی ← OpenCost Gateway	11
باقیمانده	حذف/تفکیک Mock در Odoo و Sync داده واقعی	12
باقیمانده	Budget مستقل، Recommendation واقعی، پایداری و اسکریپت Demo	13

فهرست بخش‌های اصلی

1. مفاهیم پایه و تصویر کلان پروژه

2. ساختار فایل ZIP و اجزای کد

3. فاز اول: اجرای محلی Gateway و Mock Data

4. فاز دوم: Docker، Odoo و اتصال ERP

5. فاز سوم: Kubernetes، Prometheus و OpenCost واقعی

6. فاز چهارم: تغییر Gateway از Mock به OpenCost

7. ادامه مسیر تا پروژه نهایی و قابل ارائه

8. عیب‌یابی و بازیابی

9. چک‌لیست ارائه دانشگاهی

10. پیوست دستورات و کدهای کلیدی

11. منابع رسمی

1-1. FinOps چیست؟

FinOps را می‌توان همکاری مهندسی، مالی و کسب‌وکار برای فهم، تخصیص، کنترل و بهینه‌سازی هزینه Cloud دانست. در پروژه حاضر تمرکز روی «Cost Allocation» است: یعنی هزینه زیرساخت و Workloadهای Kubernetes را به پروژه‌های سازمانی نسبت بدهیم و نتیجه را وارد Odoo کنیم تا از دید ERP نیز قابل تحلیل باشد.

در فاز دانشگاهی هدف ما «صورتحساب واقعی یک Cloud Provider» نیست؛ هدف این است که یک زنجیره متن‌باز واقعی بسازیم که مصرف Kubernetes را اندازه‌گیری، به هزینه تبدیل، نرمال‌سازی و به Odoo منتقل کند. بعداً همین معماری می‌تواند به داده‌های AWS/Azure/GCP یا Custom Pricing متصل شود.

1-2. OpenCost

OpenCost یک پروژه متن‌باز و vendor-neutral برای اندازه‌گیری و تخصیص هزینه زیرساخت و Containerهای Kubernetes است. API استاندارد Allocation می‌تواند خروجی را بر اساس namespace، pod، container، label و ابعاد دیگر تجمیع کند. در این پروژه هر Namespace نقش مرز تخصیص هزینه را دارد.

1-3. Odoo Community

Odoo در این پروژه نقش ERP و لایه مصرف‌کننده داده FinOps را دارد. ماژول اختصاصی FinOps Connector رکوردهای هزینه را از Gateway می‌گیرد، آن‌ها را به مدل finops.cost.record تبدیل می‌کند، به Project واقعی Odoo لینک می‌دهد و امکان List، Pivot و Graph را فراهم می‌کند.

1-4. چرا Gateway وسط OpenCost و Odoo قرار گرفته؟

OpenCost و Odoo مدل داده متفاوتی دارند. Gateway با FastAPI نقش Anti-Corruption Layer / Integration Layer را بازی می‌کند: OpenCost را Query می‌کند، نام پروژه‌ها و مؤلفه‌های Cost را Normalization می‌کند، داده را در SQLite نگه می‌دارد و API ساده و پایدار برای Odoo ارائه می‌دهد. این جداسازی باعث می‌شود اگر منبع Cost عوض شد، Odoo کمترین تغییر را نیاز داشته باشد.

معماری منطقی نهایی

```
Kubernetes Workloads
        ↓
Prometheus Metrics
        ↓
OpenCost Allocation API
        ↓
FinOps Gateway (FastAPI)
        ↓
Normalized Cost Records
        ↓
Odoo FinOps Connector
        ↓
Project / Pivot / Graph / Reporting
```

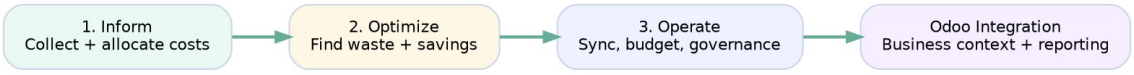
1-5. ابزارها و نقش هرکدام

نقش در پروژه	ابزار
زبان Gateway و تست‌ها	Python 3.13
API و Dashboard Gateway	FastAPI
ORM و ارتباط با SQLite	SQLAlchemy
دیتابیس سبک Gateway	SQLite
اجرای Containerها و Kubernetes محلی	Docker Desktop
بالا آوردن PostgreSQL، Odoo و Gateway	Docker Compose
دیتابیس Odoo	PostgreSQL 16
ERP و داشبورد تحلیلی FinOps	Odoo Community 19
محیط واقعی Workload برای OpenCost	Kubernetes kind
جمع‌آوری و نگه‌داری Metrics	Prometheus
محاسبه و تخصیص Cost	OpenCost
نصب Prometheus و OpenCost روی Kubernetes	Helm
مدیریت Kubernetes	kubectl

تست خودکار Backend	pytest

1-6. Mock Data در برابر Real Data

Mock Data برای توسعه اولیه ضروری بود: بدون Kubernetes و OpenCost توانستیم API، Dashboard، Odoo Connector و Sync را تست کنیم. اما Mock صرفاً داده ساختگی بود. در فاز واقعی، Provider به opencost تغییر می‌کند و Cost از Allocation API خوانده می‌شود. مزیت روش فازی این است که خطاها را جداگانه پیدا کردیم: اول خود برنامه، بعد Docker/Odoo، بعد Kubernetes/OpenCost، و در نهایت Integration واقعی.



شکل 1 - فازبندی پروژه از **Mock** تا **Integration** واقعی

فصل 2

ساختار فایل ZIP و اجزای کد

2-1. ساختار پروژه

درخت فایل‌ها

```
finops-odoo-hub/
├── gateway/
│   ├── app/
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── db.py
│   │   ├── models.py
│   │   ├── schemas.py
│   │   ├── seed.py
│   │   ├── services.py
│   │   ├── providers/
│   │   │   ├── base.py
│   │   │   └── opencost.py
│   │   ├── templates/dashboard.html
│   │   └── static/style.css
│   ├── tests/
│   ├── requirements.txt
│   └── Dockerfile
├── odoo_addons/
│   └── finops_connector/
│       ├── models/
│       ├── views/
│       ├── data/ir_cron.xml
│       └── __manifest__.py
├── docker-compose.yml
├── .env.example
├── docs/
└── scripts/
```

2-2. فایل‌های Gateway

فایل	وظیفه
main.py	ساخت FastAPI، endpointها، lifespan و Dashboard route
config.py	خواندن تنظیمات FINOPS_* از Environment/.env
db.py	Engine و Session دیتابیس
models.py	جدول‌های Budget، CostRecord و Recommendation
seed.py	ساخت داده ماکت و بودجه‌های نمونه

services.py	Summary، Queryها، Dashboard aggregate و Sync Provider
providers/base.py	قرارداد NormalizedCost و CostProvider
providers/opencost.py	Adapter اتصال به OpenCost
dashboard.html	قالب HTML داشبورد
style.css	ظاهر داشبورد

2-3. فایل‌های Odoo Add-on

فایل	وظیفه
manifest__.py__	شناسنامه افزونه و dependencyها
models/finops_cost.py	مدل Cost و Sync از Gateway
models/res_config_settings.py	Gateway URL در تنظیمات Odoo
views/finops_cost_views.xml	List/Form/Pivot/Graph و منوها
data/ir_cron.xml	Sync خودکار هر 6 ساعت
security/ir.model.access.csv	سطح دسترسی مدل

2-4. APIهای Gateway

Endpoint	کاربرد
GET /health	سلامت برنامه و provider_mode
GET /api/v1/summary	KPIهای مدیریتی
GET /api/v1/costs	رکوردهای هزینه با Filter
GET /api/v1/budgets	بودجه‌ها
GET /api/v1/recommendations	پیشنهادهای بهینه‌سازی
POST /api/v1/sync/provider	همگام‌سازی Provider خارجی
GET /api/v1/odoo/export	Payload نرمال‌شده برای Odoo
GET /	Dashboard HTML

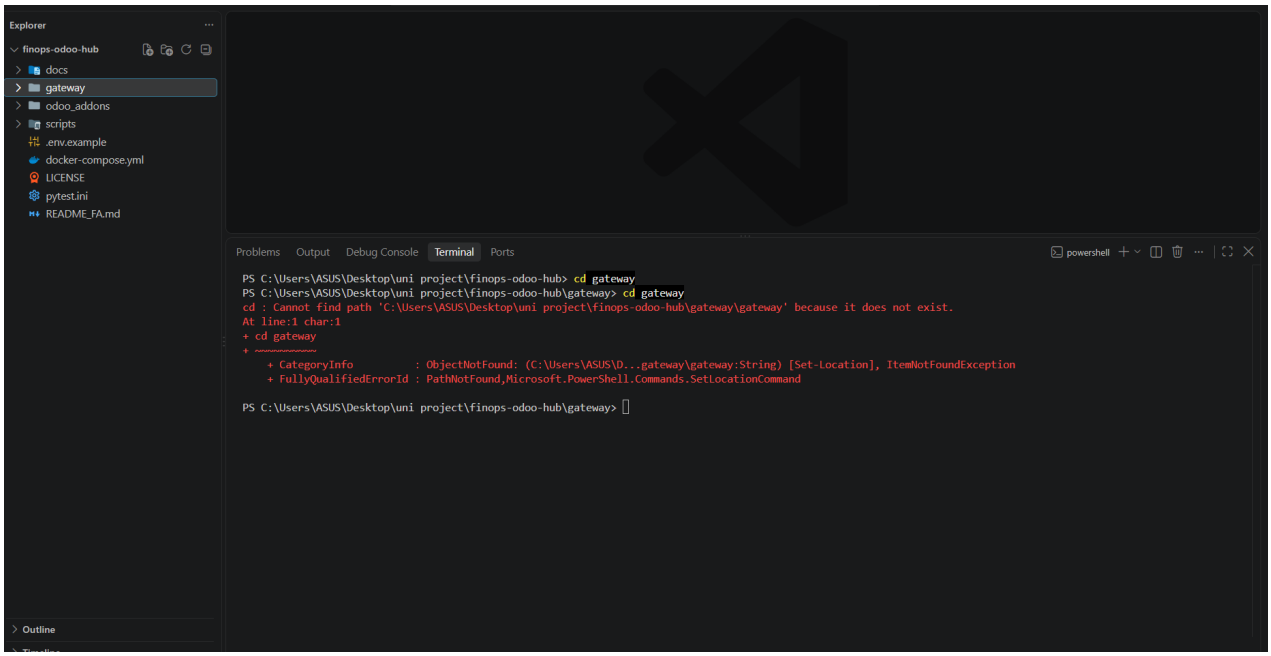
Mock Data و Gateway، API فاز اول - اجرای محلی

وضعیت فاز

این فاز کامل انجام شده و با اسکرین‌شات‌های واقعی مستند است.

3-1. باز کردن پروژه و ورود به gateway

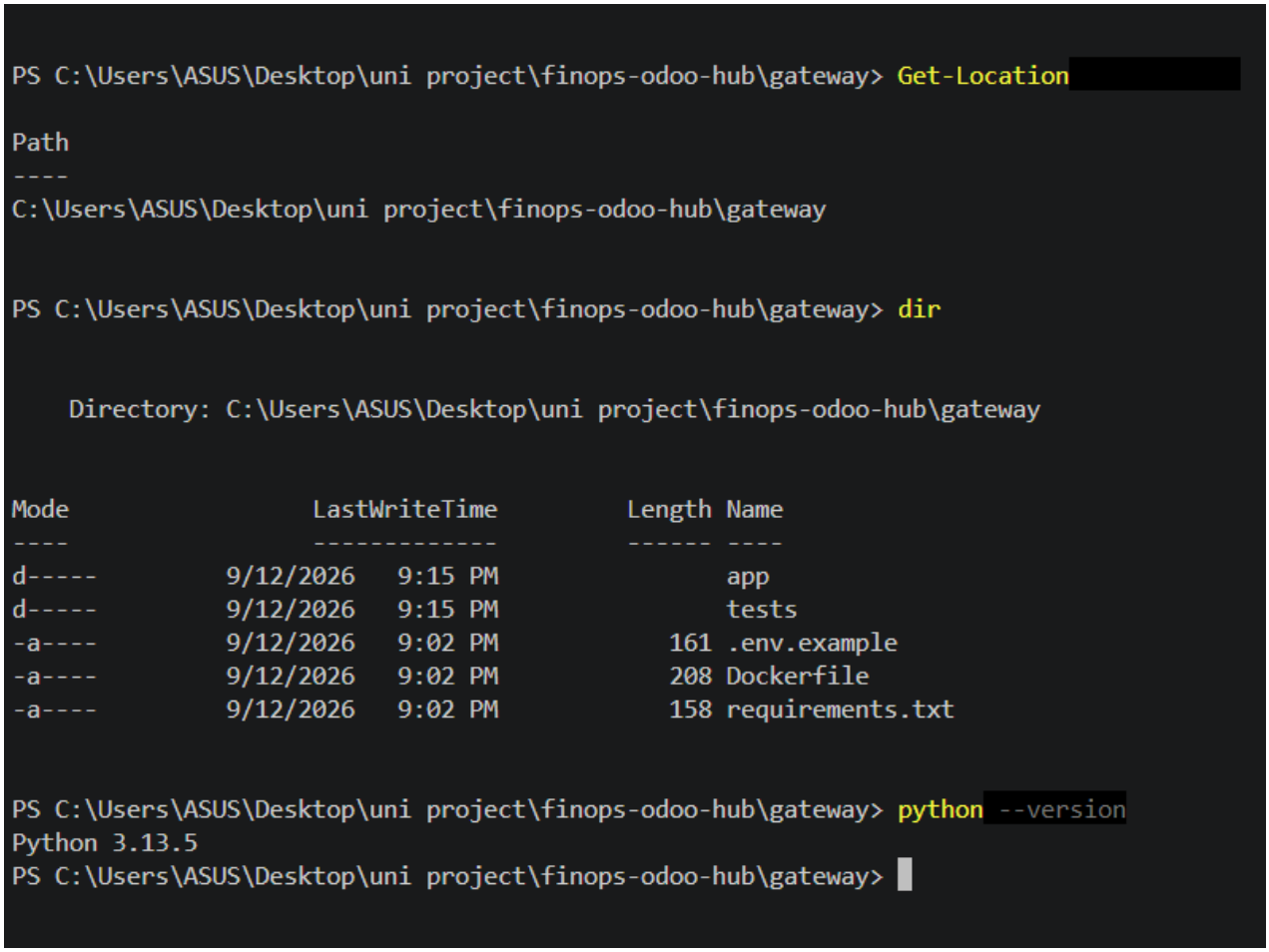
پوشه ZIP استخراج شد و با VS Code باز شد. خطای اولیه به این دلیل بود که کاربر پس از ورود به gateway دوباره دستور cd gateway را اجرا کرد؛ در نتیجه PowerShell دنبال gateway\gateway گشت.



شکل 2 - خطای مسیر به دلیل اجرای دوباره **cd gateway**

دستورات تشخیص مسیر و Python

```
Get-Location
dir
python --version
```



شکل 3 - تأیید مسیر **gateway**، فایل‌ها و **Python 3.13.5**

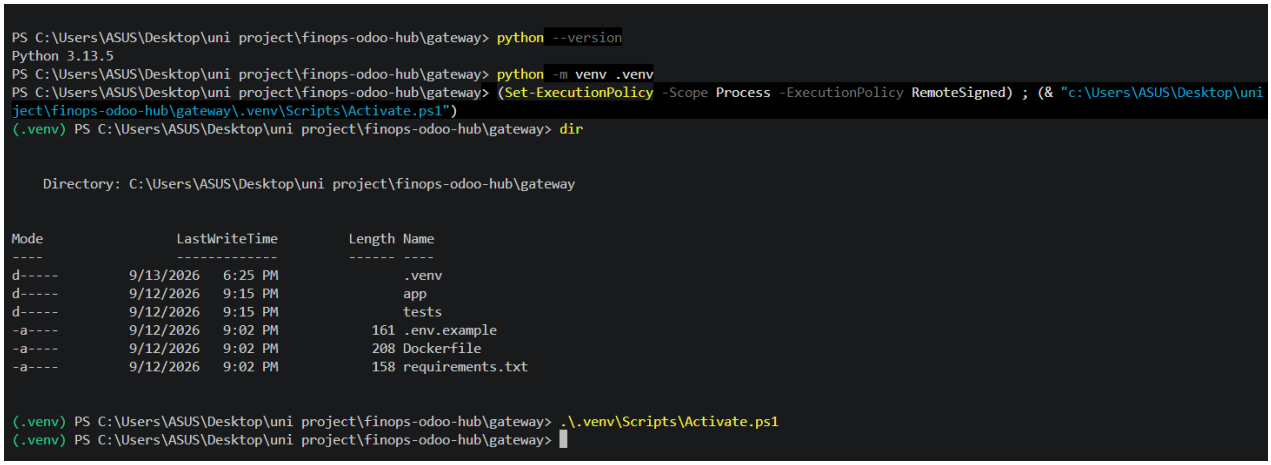
3-2. ساخت Virtual Environment

ساخت و فعال‌سازی محیط مجازی

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

اگر PowerShell اجازه اجرای Script ندهد، فقط برای همان Session می‌توان Execution Policy را موقتاً تغییر داد:

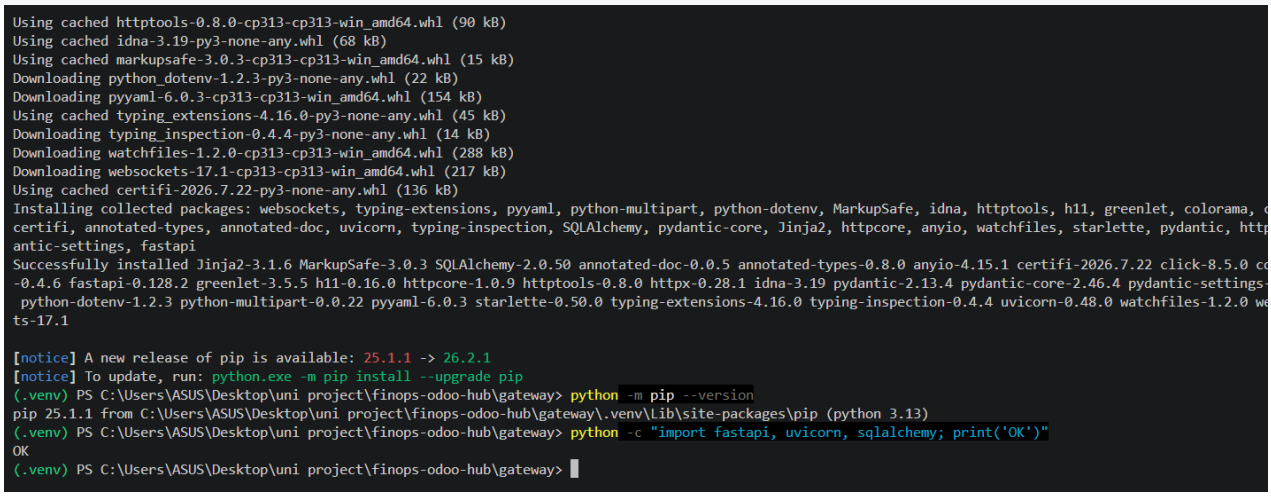
```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```



شکل 4 - ایجاد **venv.** و فعال‌شدن محیط مجازی

3-3. نصب Dependencyها و تست Import

```
pip install -r requirements.txt
python -m pip --version
python -c "import fastapi, uvicorn, sqlalchemy; print('OK')
```



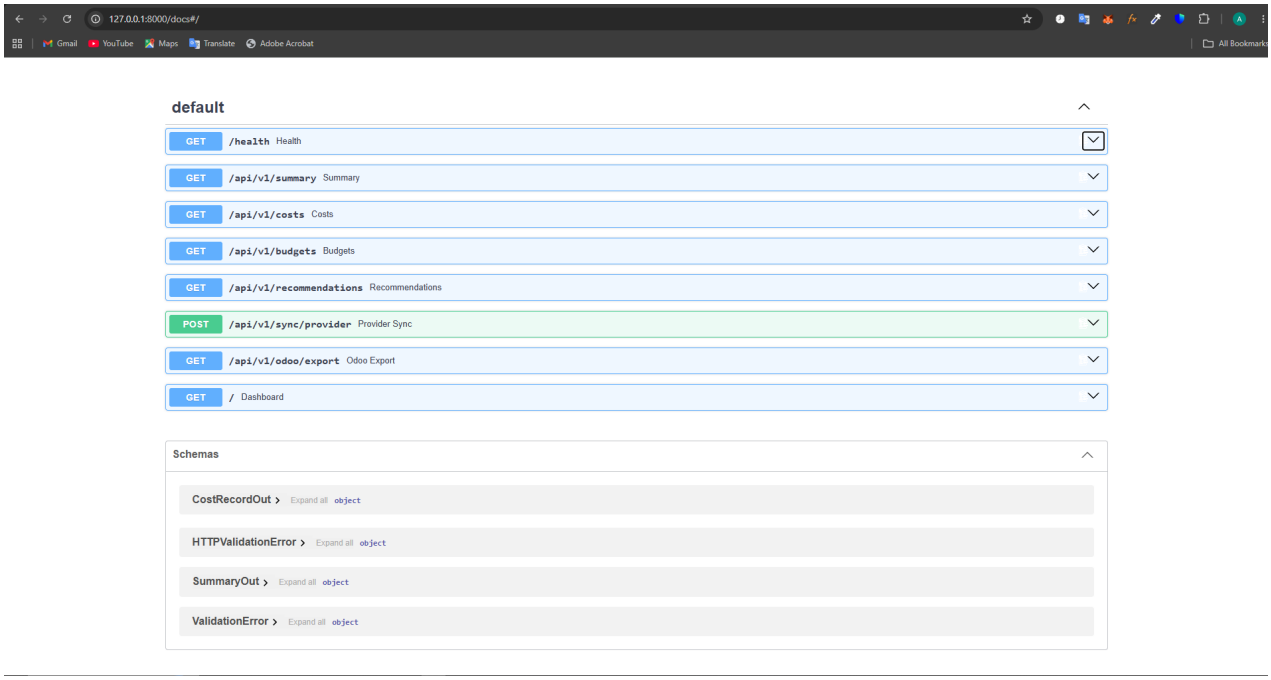
شکل 5 - نصب موفق **Dependency**ها و خروجی **OK**

3-4. اجرای FastAPI

اجرای توسعه محلی

```
uvicorn app.main:app --reload
```

- Dashboard: http://127.0.0.1:8000
- Swagger: http://127.0.0.1:8000/docs
- Health: http://127.0.0.1:8000/health

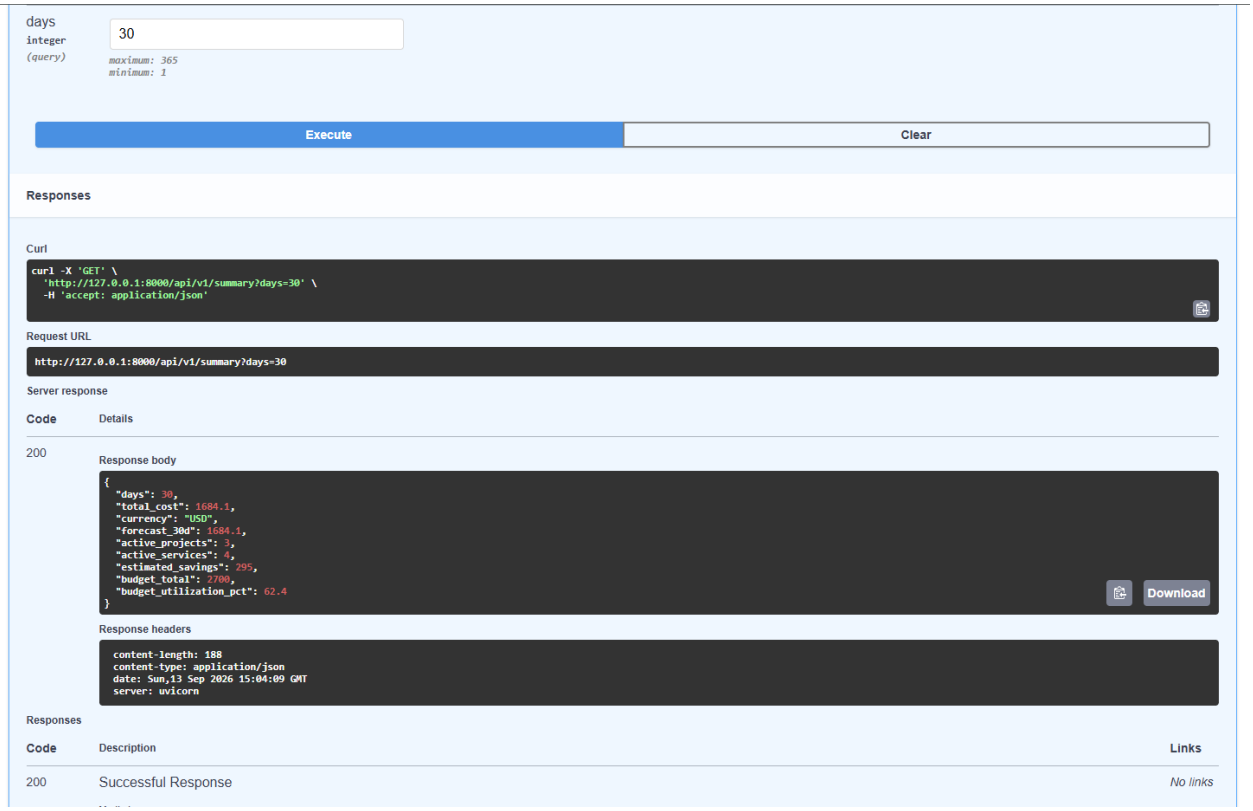


شکل 6 - Swagger و Endpointهای Gateway

3-5. تست Summary API

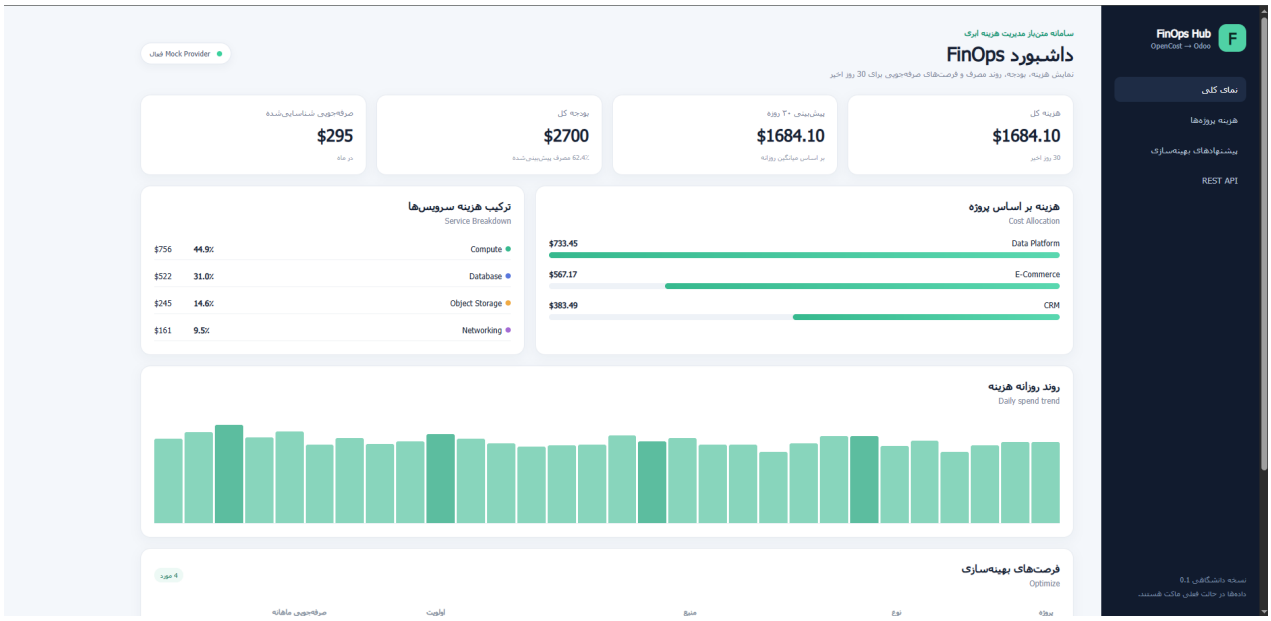
```
GET /api/v1/summary?days=30
```

در اجرای اولیه داده ماکت شامل 3 پروژه و 4 سرویس بود. Summary هزینه 30 روزه، Forecast، بودجه و Savings را برمی‌گرداند.

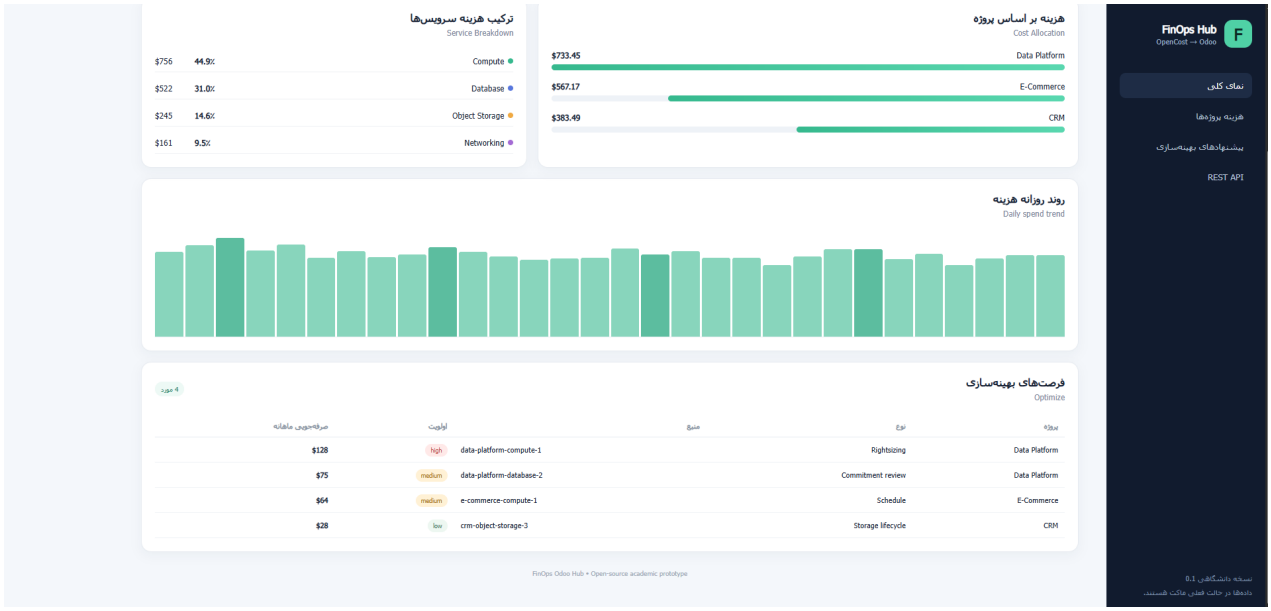


شکل 7 - پاسخ 200 برای **Summary** و **KPI**های اولیه

3-6. Dashboard اولیه



شکل 8 - نمای بالایی **Dashboard** ماکت



شکل 9 - نمای پایین **Dashboard** شامل **Trend** و **Recommendations**

3-7. منطق Mock Data

در seed.py سه پروژه E-Commerce، Data Platform و CRM و چهار Service تعریف شده بود. برای 90 روز، به ازای هر Project و Service یک رکورد ساخته می‌شد؛ یعنی حدود 90×3×4=1080 رکورد. همچنین برای Data Platform / Compute در روزهای آخر افزایش 42 درصدی ایجاد شده بود تا یک Anomaly نمونه داشته باشیم.

تعریف پروژه‌ها و سرویس‌های ماکت

```
PROJECTS = [
    ("E-Commerce", "Digital", 900.0),
    ("Data Platform", "Data", 1150.0),
    ("CRM", "Sales", 650.0),
]

SERVICES = [
    ("Compute", "Compute", 1.00),
    ("Database", "Database", 0.72),
    ("Object Storage", "Storage", 0.34),
    ("Networking", "Network", 0.22),
]
```

3-8. اولین Customization: افزودن AI Platform

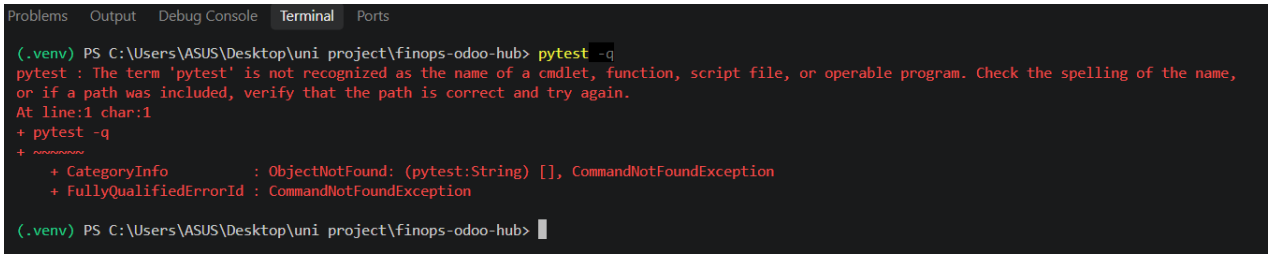
```
PROJECTS = [
    ("E-Commerce", "Digital", 900.0),
    ("Data Platform", "Data", 1150.0),
    ("CRM", "Sales", 650.0),
    ("AI Platform", "AI", 800.0),
]

project_factor = [1.0, 1.24, 0.68, 1.10][p_idx]
```

برای بازسازی داده ماکت، Server متوقف و فایل SQLite حذف شد تا Seed با 4 پروژه اجرا شود. نتیجه باید active_projects=4 و budget_total=3500 باشد.

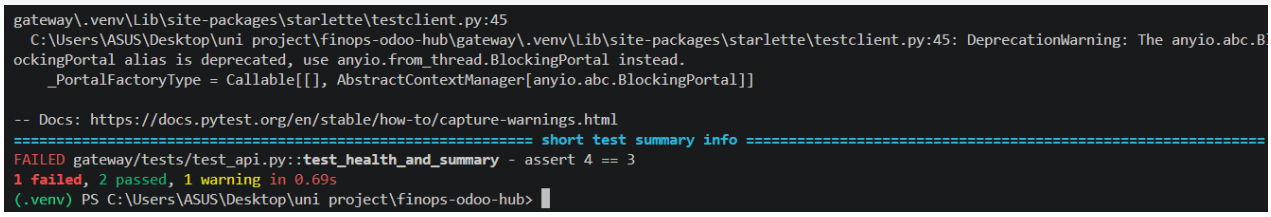
3-9. تست خودکار و مفهوم Regression

pytest در محیط مجازی نصب نبود و ابتدا CommandNotFound رخ داد. سپس پس از نصب pytest، یکی از تست‌ها به‌درستی Fail شد چون هنوز انتظار active_projects=3 را داشت.



شکل 10 - خطای **pytest** نصب‌نشده

```
python -m pip install pytest
python -m pytest --version
python -m pytest -q
```



شکل 11 - **Fail** شدن تست با **assert 4 == 3** پس از افزودن **AI Platform**

اصلاح تست مطابق **Specification** جدید

```
# در gateway/tests/test_api.py
assert data["active_projects"] == 4
```

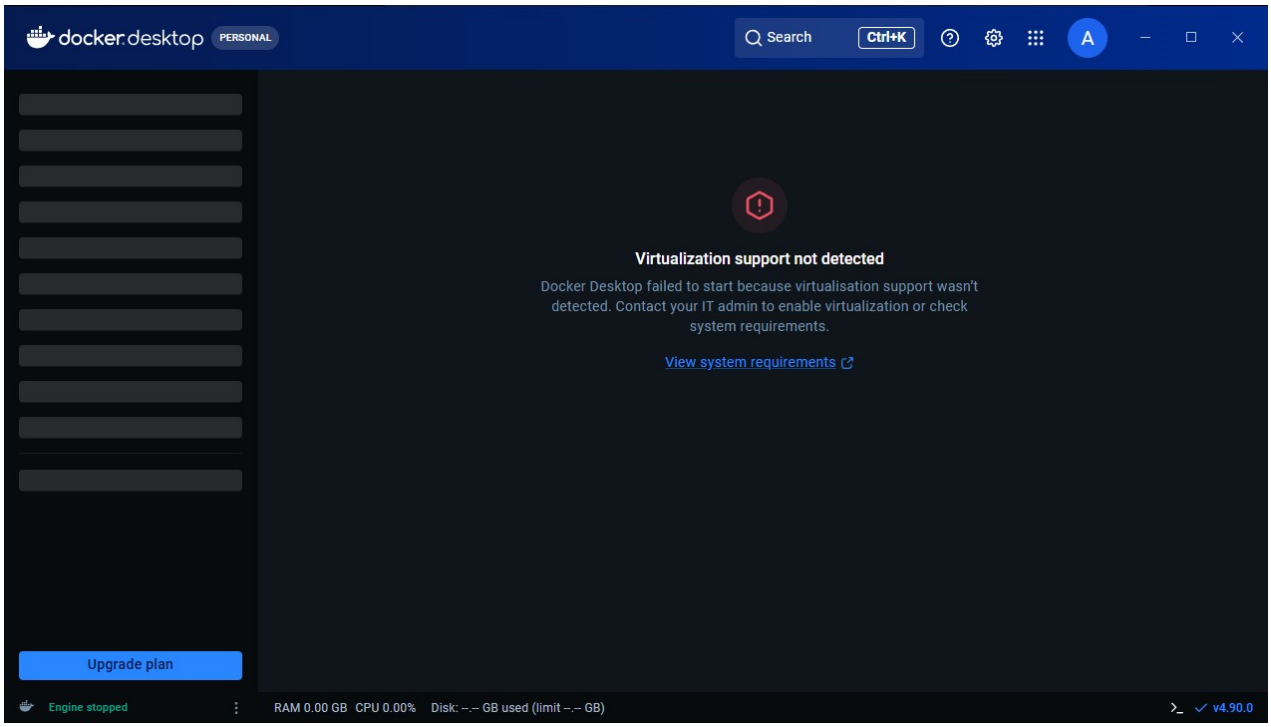
> اصل مهم
>
> وقتی رفتار سیستم عمداً تغییر می‌کند، تستی که رفتار قبلی را Assert می‌کند باید به‌روزرسانی شود. هدف تست جلوگیری از تغییر ناخواسته است، نه جلوگیری از توسعه عمدی.

فاز دوم - Odoo، PostgreSQL، Docker و FinOps Connector

وضعیت فاز

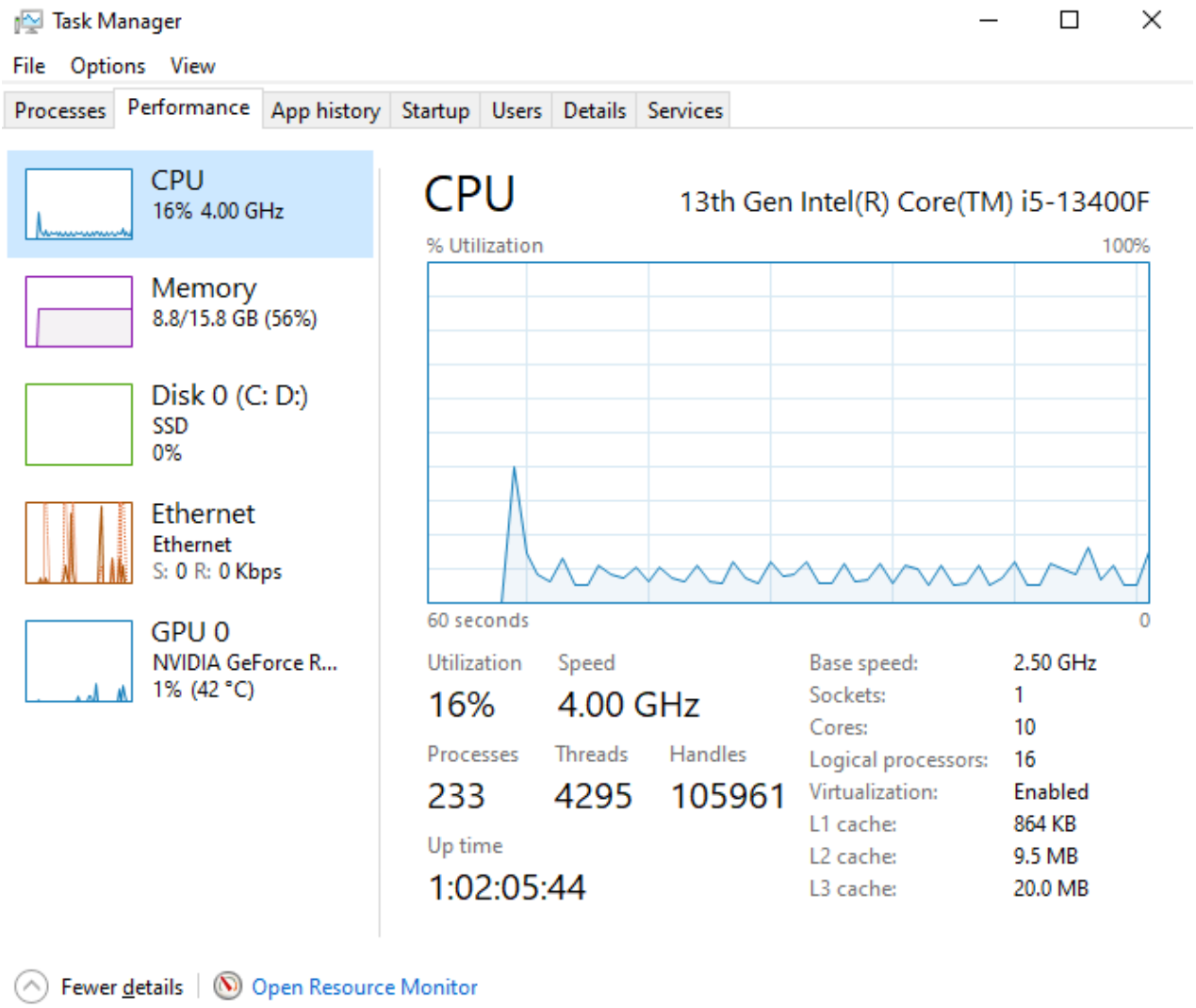
Odoo Community 19، PostgreSQL، Docker و FinOps Connector راه‌اندازی و Sync ماکت با 480 رکورد با موفقیت آزمایش شده است.

4-1. خطای Virtualization در Docker Desktop



شکل 12 - **Docker Desktop: Virtualization support not detected**

Task Manager نشان داد Virtualization در BIOS فعال است؛ بنابراین مشکل از BIOS نبود و عیب‌یابی روی WSL2 و Virtual Machine Platform متمرکز شد.

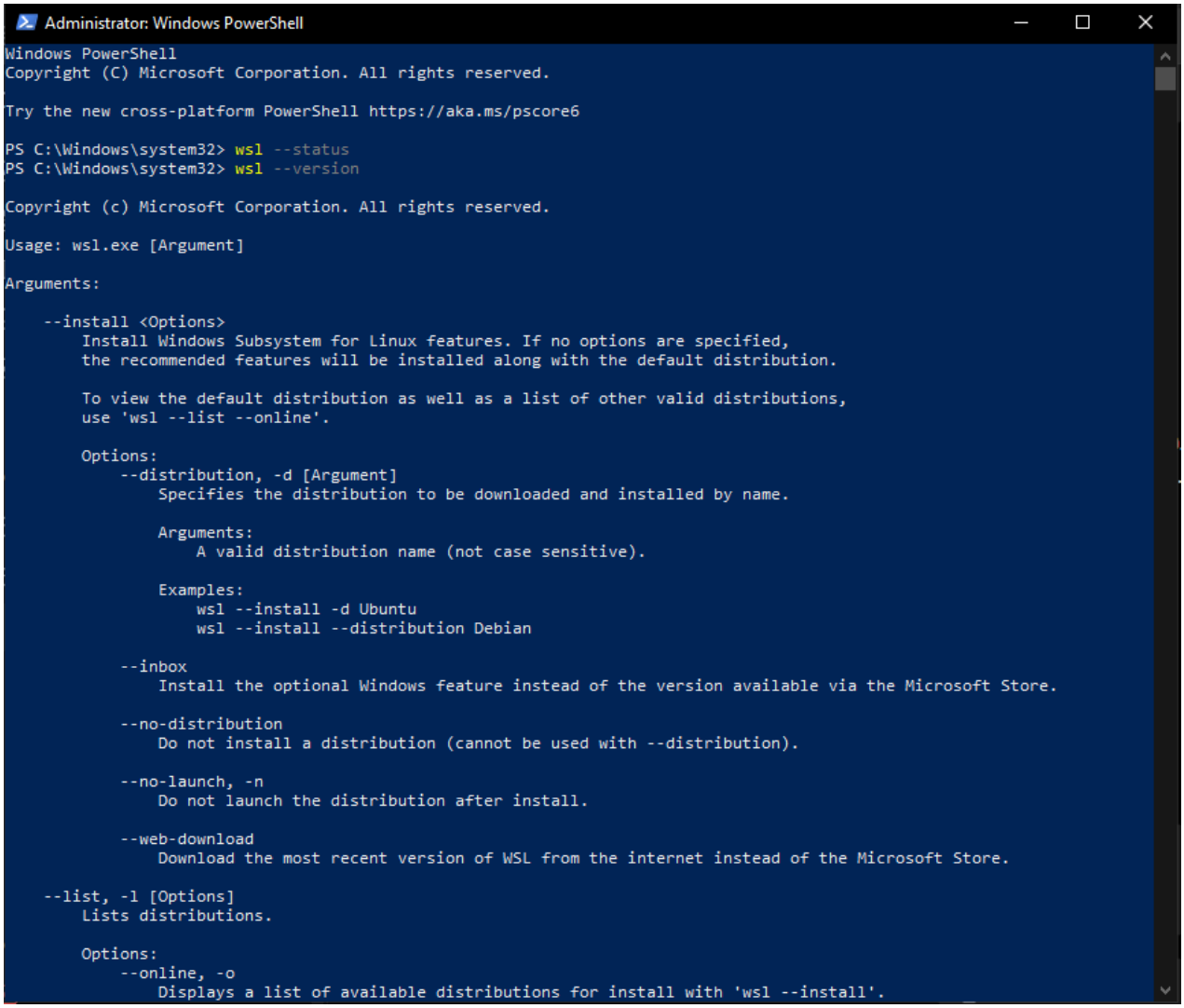


Task Manager: Virtualization = Enabled - شکل 13

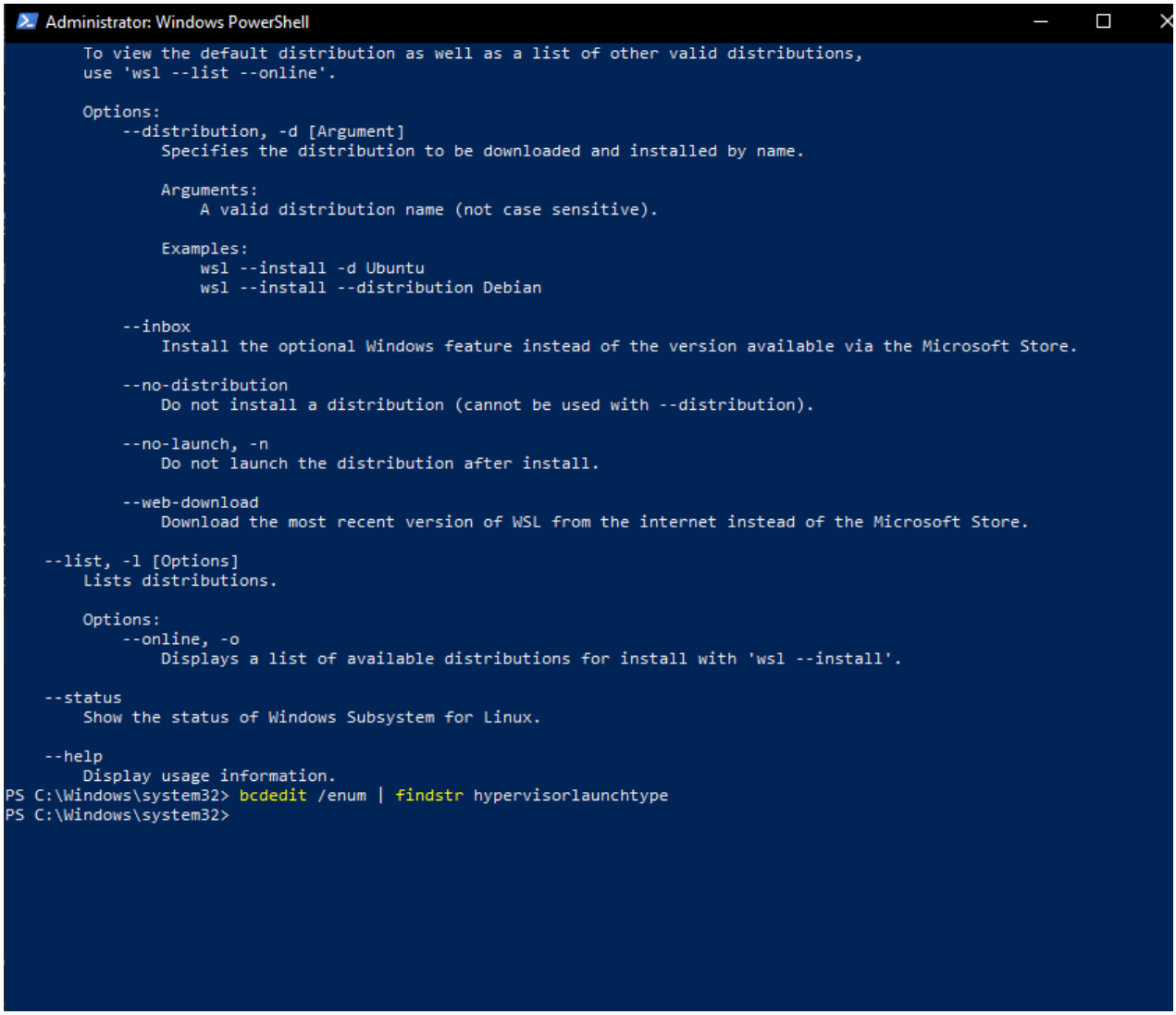
4-2. بررسی و ترمیم WSL2

فرمان‌های تشخیص WSL

```
wsl --status
wsl --version
bcdedit /enum | findstr hypervisorlaunchtype
```



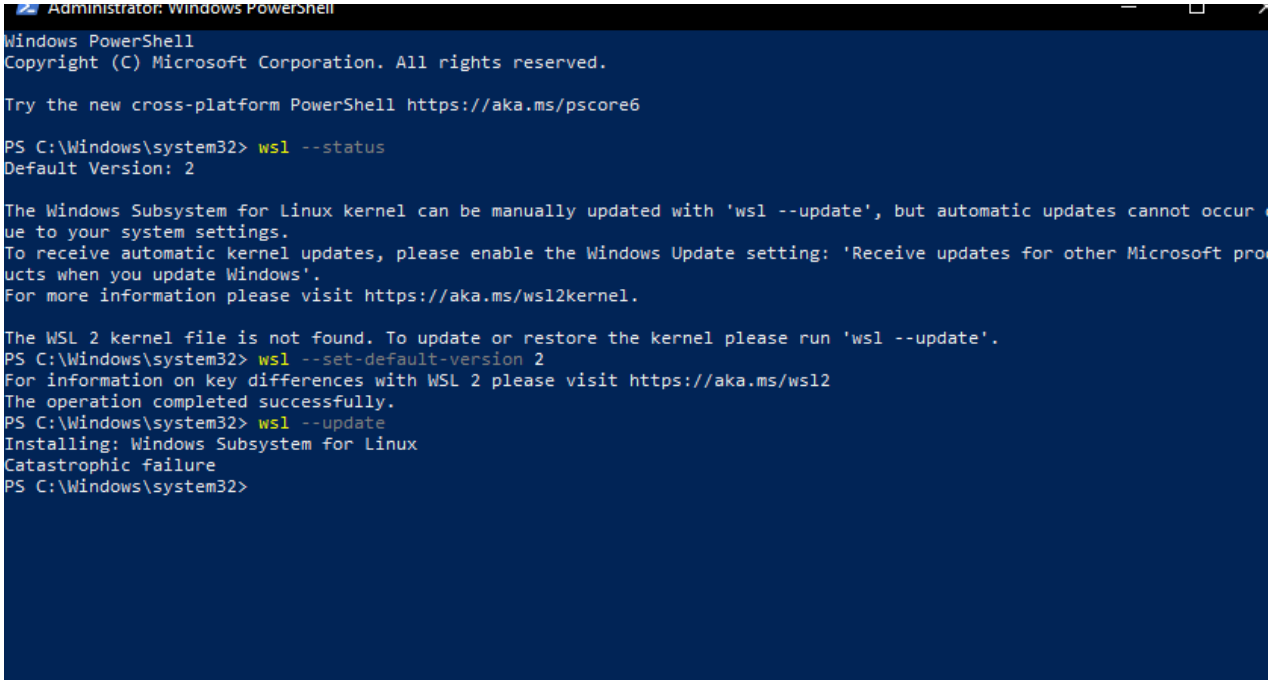
شکل 14 - نسخه قدیمی/**inbox WSL** و نمایش **Help** به جای **wsl --version**



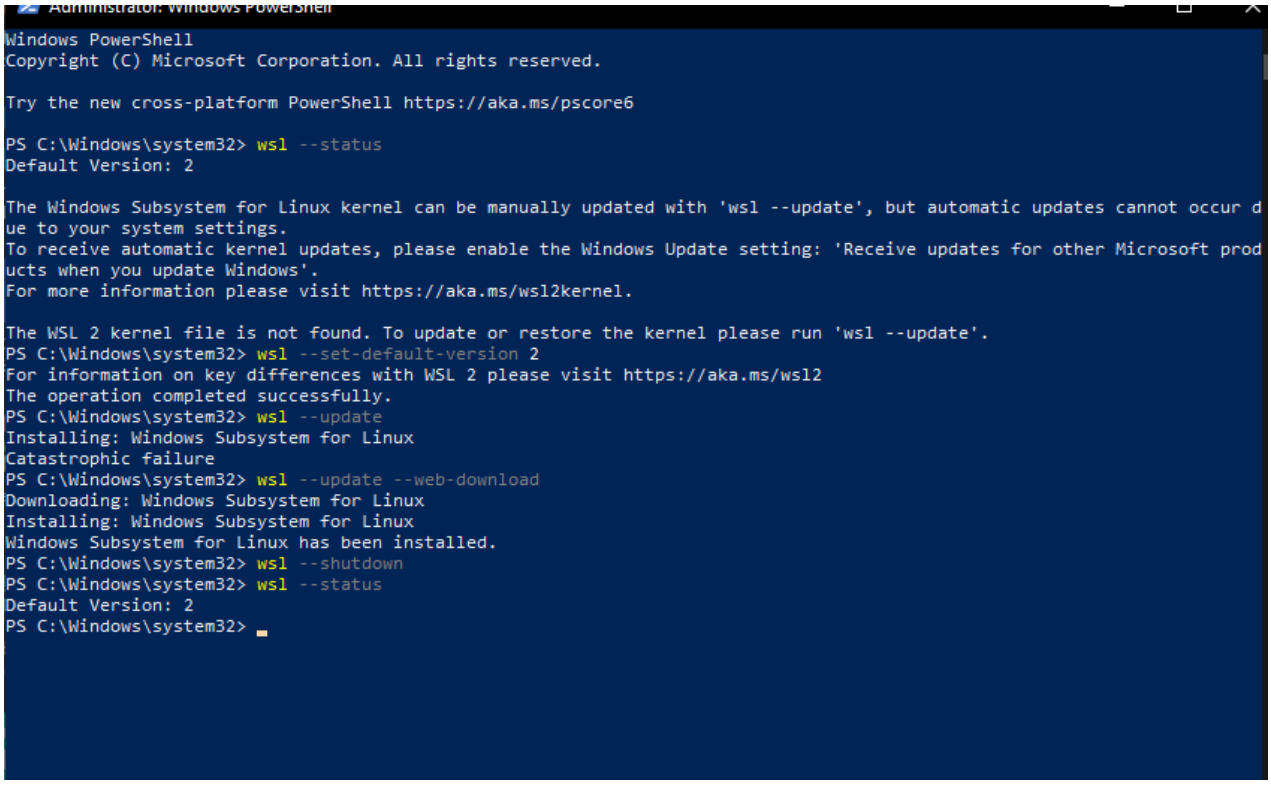
شکل 15 - ادامه تشخیص WSL و **hypervisorlaunchtype**

فعال‌سازی **WSL2** و **Virtual Machine Platform**

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
bcdedit /set hypervisorlaunchtype auto
wsl --install --no-distribution
# Restart
wsl --set-default-version 2
wsl --update --web-download
```



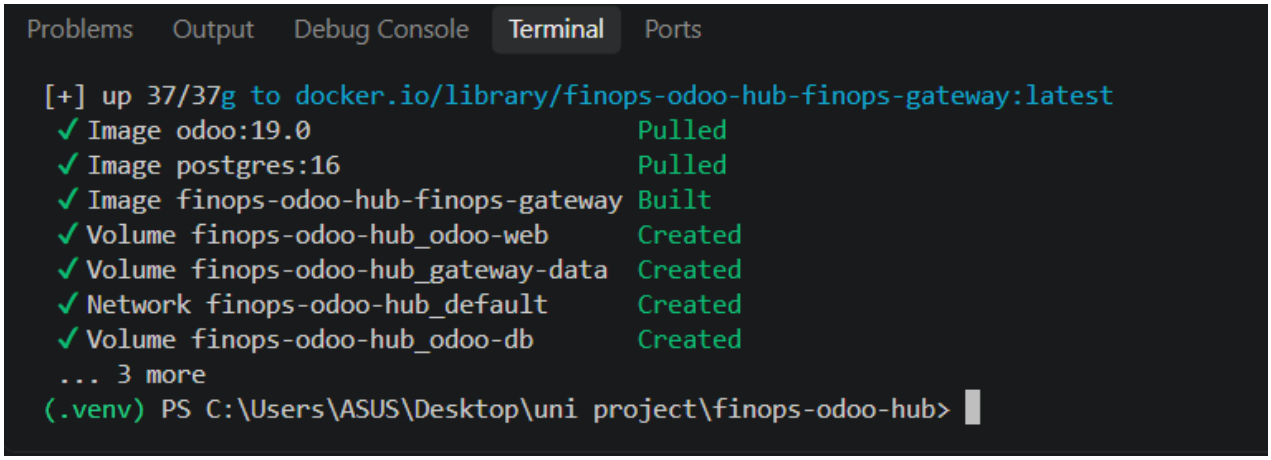
شکل 16 - خطای **WSL2 kernel file not found** و شکست **update** اولیه



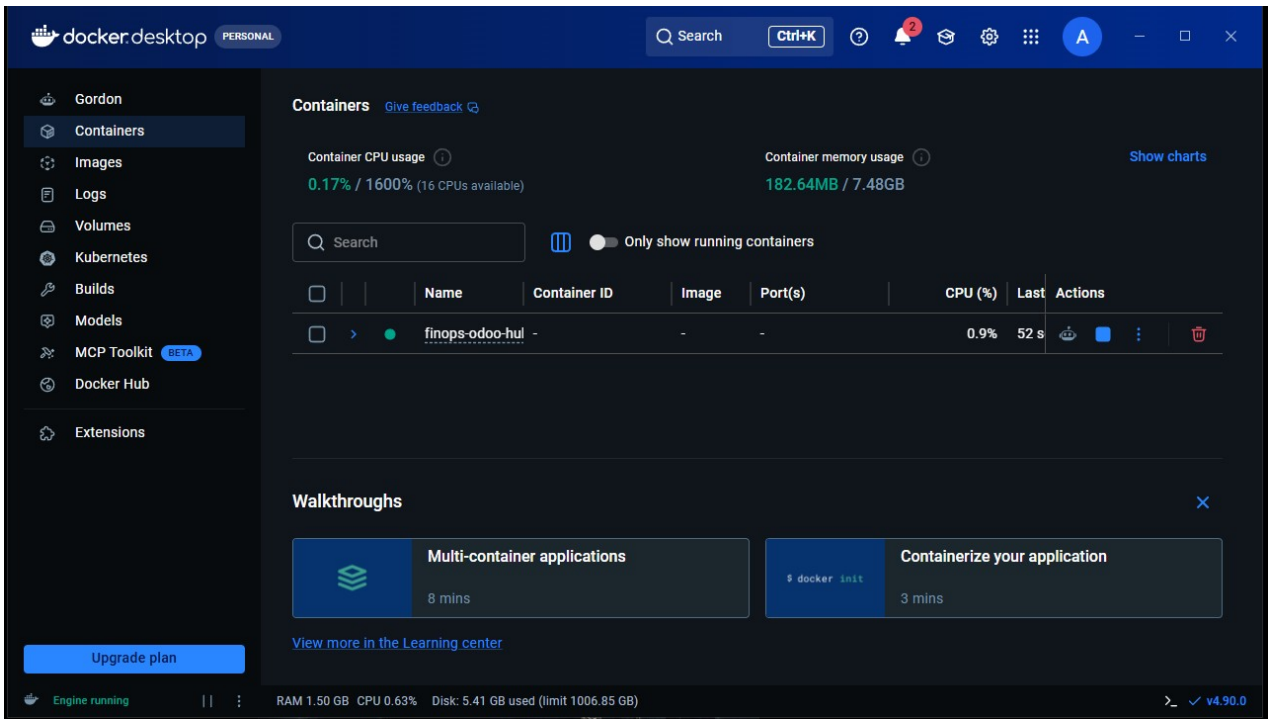
شکل 17 - نصب موفق **WSL** با **web-download** و **Default Version 2**

4-3. بالا آوردن Compose Stack

```
Copy-Item .env.example .env
docker compose up -d --build
docker compose ps
```



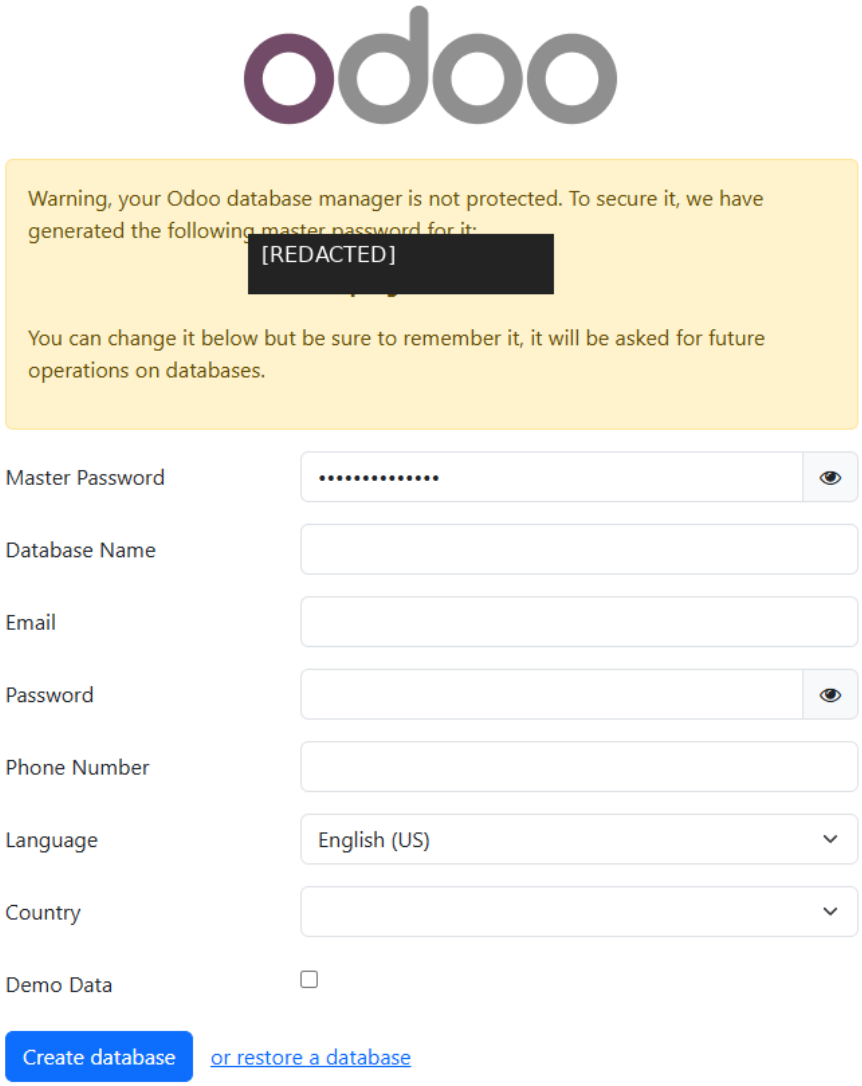
شکل 18 - **Pull/Build Image**ها و ساخت **Volume/Network** توسط **Compose**



شکل 19 - **Docker Engine Running** و **Compose Project** فعال

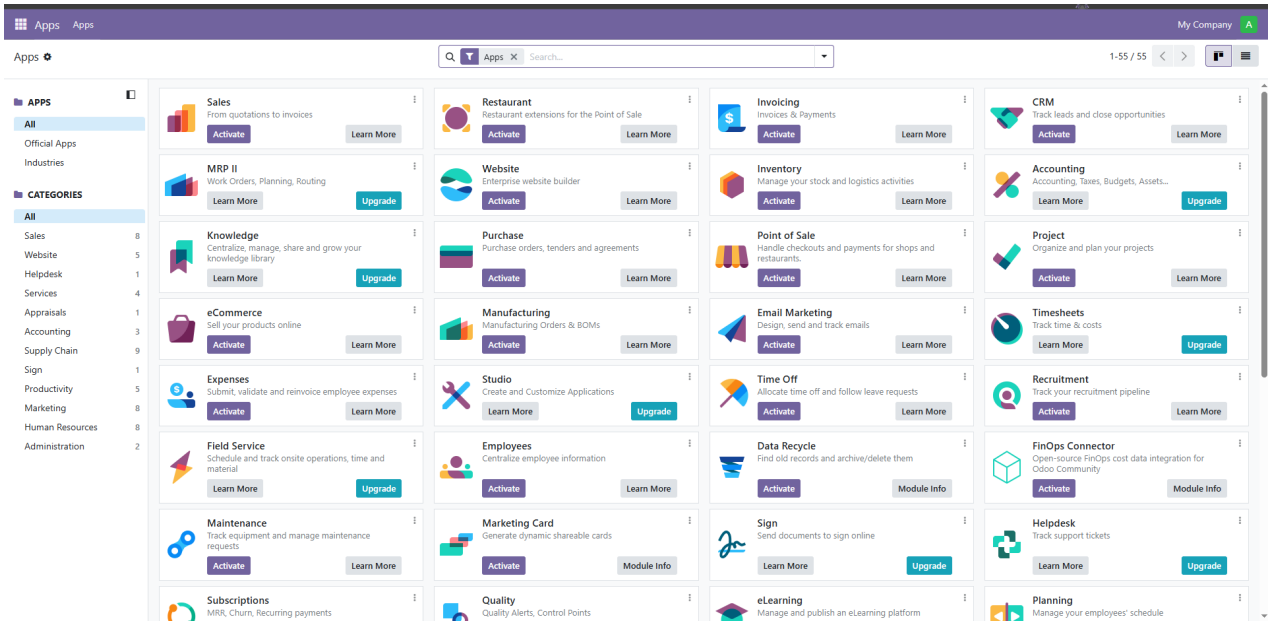
Odoo Database .4-4

Odoo روی http://localhost:8069 در دسترس قرار گرفت. در اولین اجرا Database ساخته شد. Demo Data غیرفعال نگه داشته شد تا محیط ارائه فقط شامل داده‌های پروژه باشد.

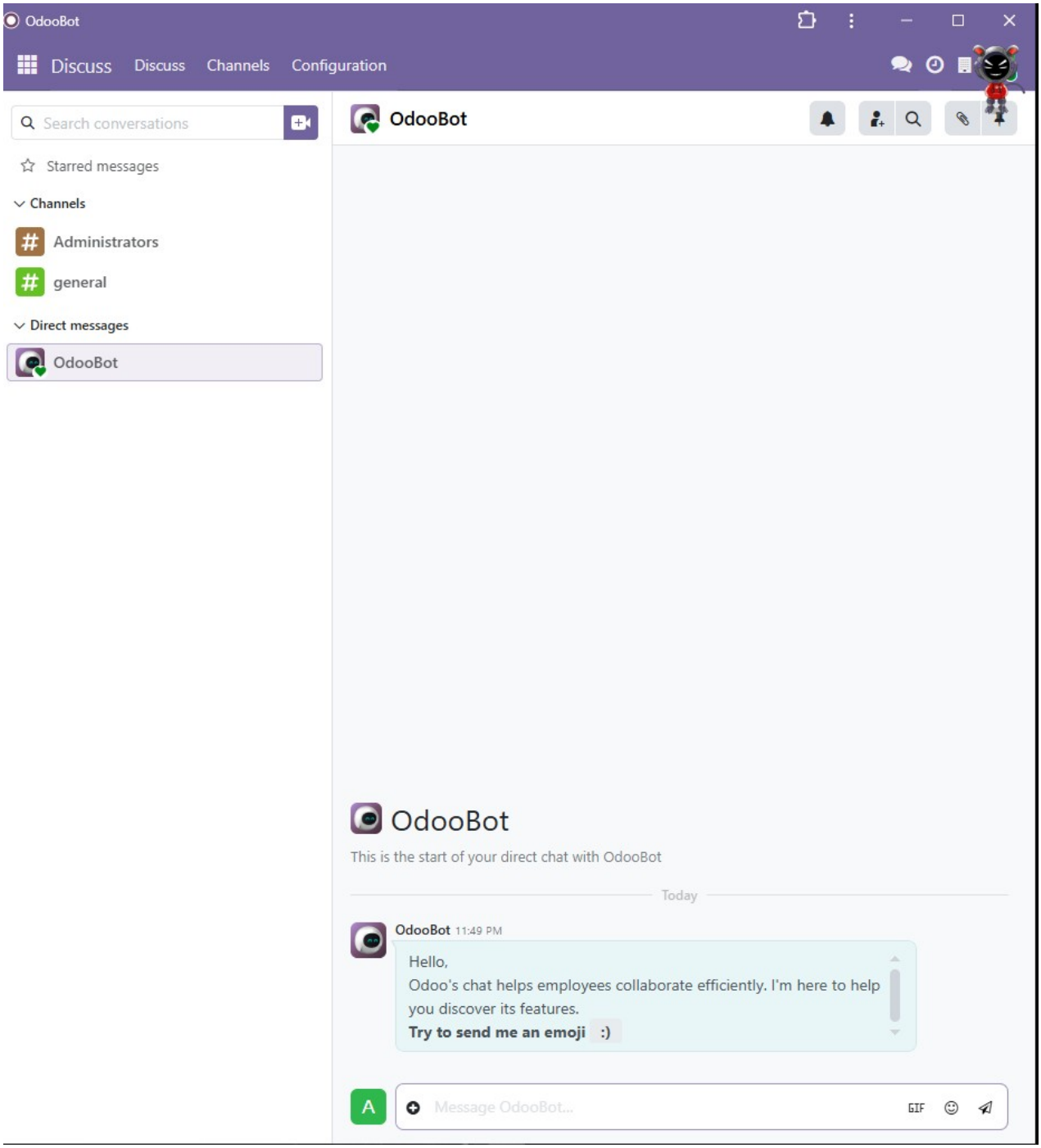


شکل 20 - صفحه ساخت دیتابیس **Odoo** (**Master Password** در مستند حذف شده است)

4-5. نصب FinOps Connector



شکل 21 - شناخته‌شدن **FinOps Connector** در **Apps**

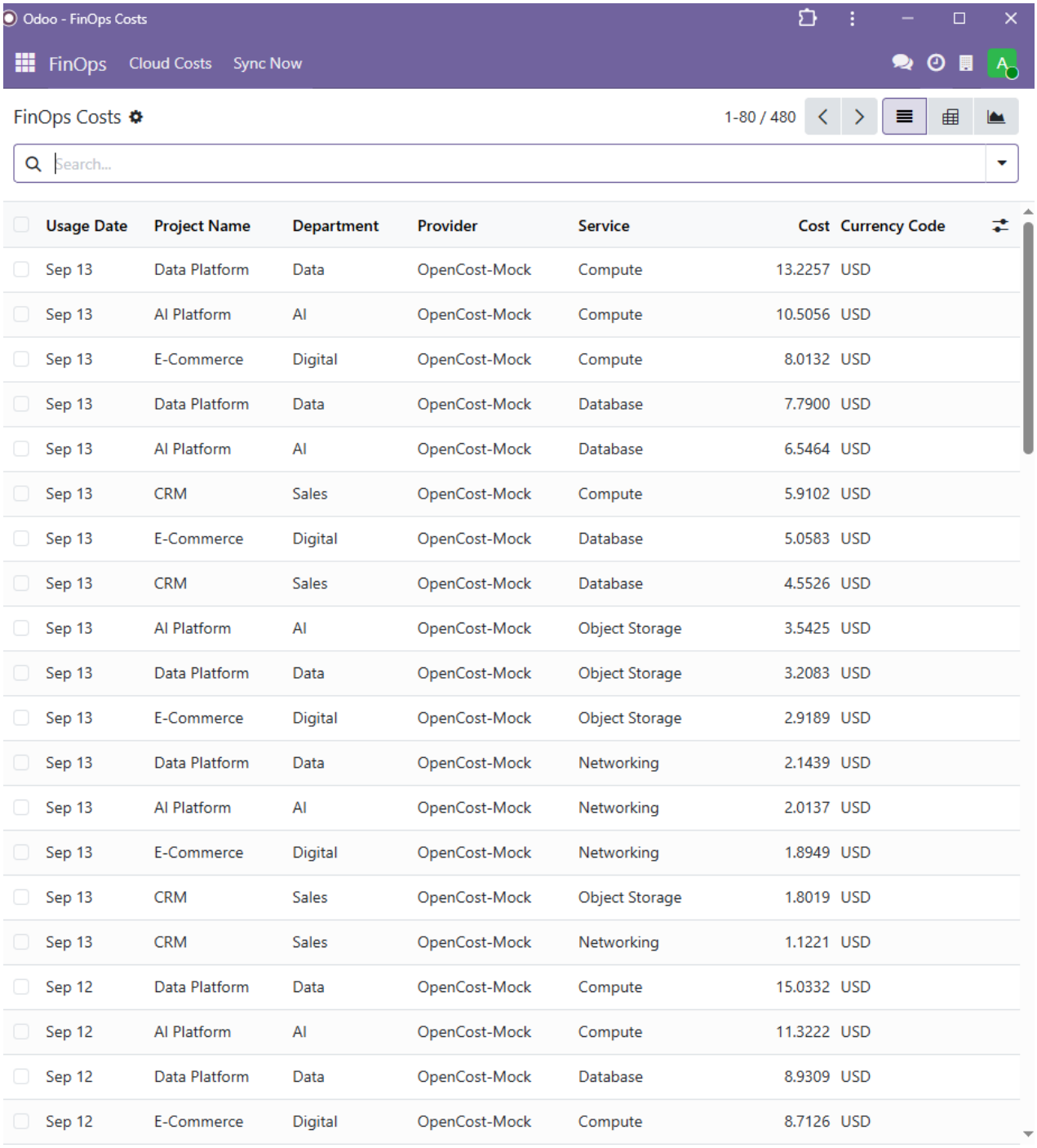


شکل 22 - بازگشت **Odoo** به **Discuss** بعد از **Activate**؛ رفتار عادی پس از نصب

4-6. اولین Sync Gateway → Odoo

Connector endpoint /api/v1/odoo/export را می‌خواند و هر record را بر اساس external_id Upsert می‌کند.

با 30 روز × 4 پروژه × 4 سرویس، دقیقاً 480 رکورد در Cloud Costs مشاهده شد.



Usage Date	Project Name	Department	Provider	Service	Cost	Currency Code
Sep 13	Data Platform	Data	OpenCost-Mock	Compute	13.2257	USD
Sep 13	AI Platform	AI	OpenCost-Mock	Compute	10.5056	USD
Sep 13	E-Commerce	Digital	OpenCost-Mock	Compute	8.0132	USD
Sep 13	Data Platform	Data	OpenCost-Mock	Database	7.7900	USD
Sep 13	AI Platform	AI	OpenCost-Mock	Database	6.5464	USD
Sep 13	CRM	Sales	OpenCost-Mock	Compute	5.9102	USD
Sep 13	E-Commerce	Digital	OpenCost-Mock	Database	5.0583	USD
Sep 13	CRM	Sales	OpenCost-Mock	Database	4.5526	USD
Sep 13	AI Platform	AI	OpenCost-Mock	Object Storage	3.5425	USD
Sep 13	Data Platform	Data	OpenCost-Mock	Object Storage	3.2083	USD
Sep 13	E-Commerce	Digital	OpenCost-Mock	Object Storage	2.9189	USD
Sep 13	Data Platform	Data	OpenCost-Mock	Networking	2.1439	USD
Sep 13	AI Platform	AI	OpenCost-Mock	Networking	2.0137	USD
Sep 13	E-Commerce	Digital	OpenCost-Mock	Networking	1.8949	USD
Sep 13	CRM	Sales	OpenCost-Mock	Object Storage	1.8019	USD
Sep 13	CRM	Sales	OpenCost-Mock	Networking	1.1221	USD
Sep 12	Data Platform	Data	OpenCost-Mock	Compute	15.0332	USD
Sep 12	AI Platform	AI	OpenCost-Mock	Compute	11.3222	USD
Sep 12	Data Platform	Data	OpenCost-Mock	Database	8.9309	USD
Sep 12	E-Commerce	Digital	OpenCost-Mock	Compute	8.7126	USD

شکل 23 - نمای **Cloud Costs** در **Odoo** با 480 رکورد **Mock**

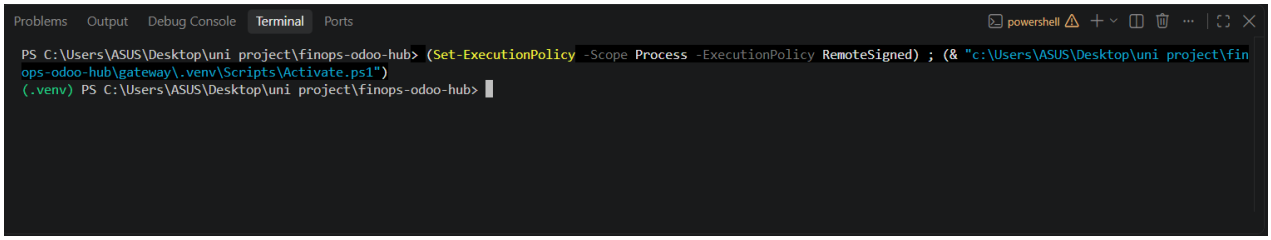
Idempotency / Upsert .4-7

Sync Now دوباره اجرا شد و تعداد روی 480 باقی ماند. این نشان می‌دهد Odoo به‌جای Duplicate کردن، رکورد موجود را با external_id پیدا و Update می‌کند. این ویژگی برای Cron شش‌ساعته حیاتی است.

منطق **Upsert** در **Odoo Connector**

```
record = self.search([("external_id", "=", vals["external_id"])], limit=1)
if record:
    record.write(vals)
    updated += 1
else:
    self.create(vals)
    created += 1
```

4-8. اجرای روزمره پروژه با Docker



شکل 24 - ترمینال **VS Code** در ریشه پروژه و فعال بودن **venv**.

از این فاز به بعد Gateway با Docker اجرا می‌شود؛ فعال بودن .venv ضرری ندارد اما برای Docker لازم نیست.

نباید Uvicorn محلی را همزمان روی پورت 8000 اجرا کرد.

```
# روشن کردن
Docker Desktop

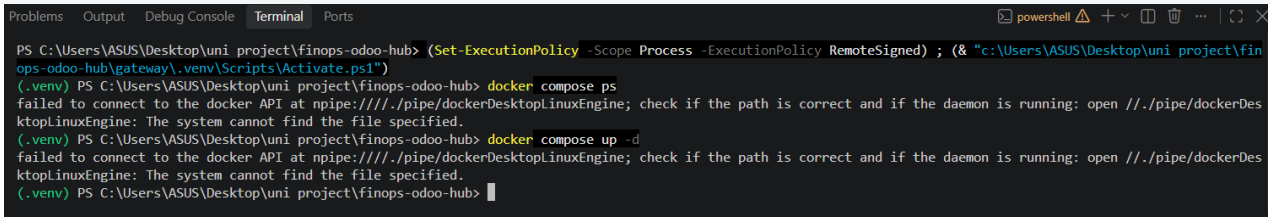
docker compose up -d
docker compose ps

# آدرس‌ها
http://localhost:8000
http://localhost:8000/docs
http://localhost:8069

# توقف
docker compose stop

# شروع مجدد
docker compose start

# حذف Containerها (داده Volumeها باقی می‌ماند)
docker compose down
```



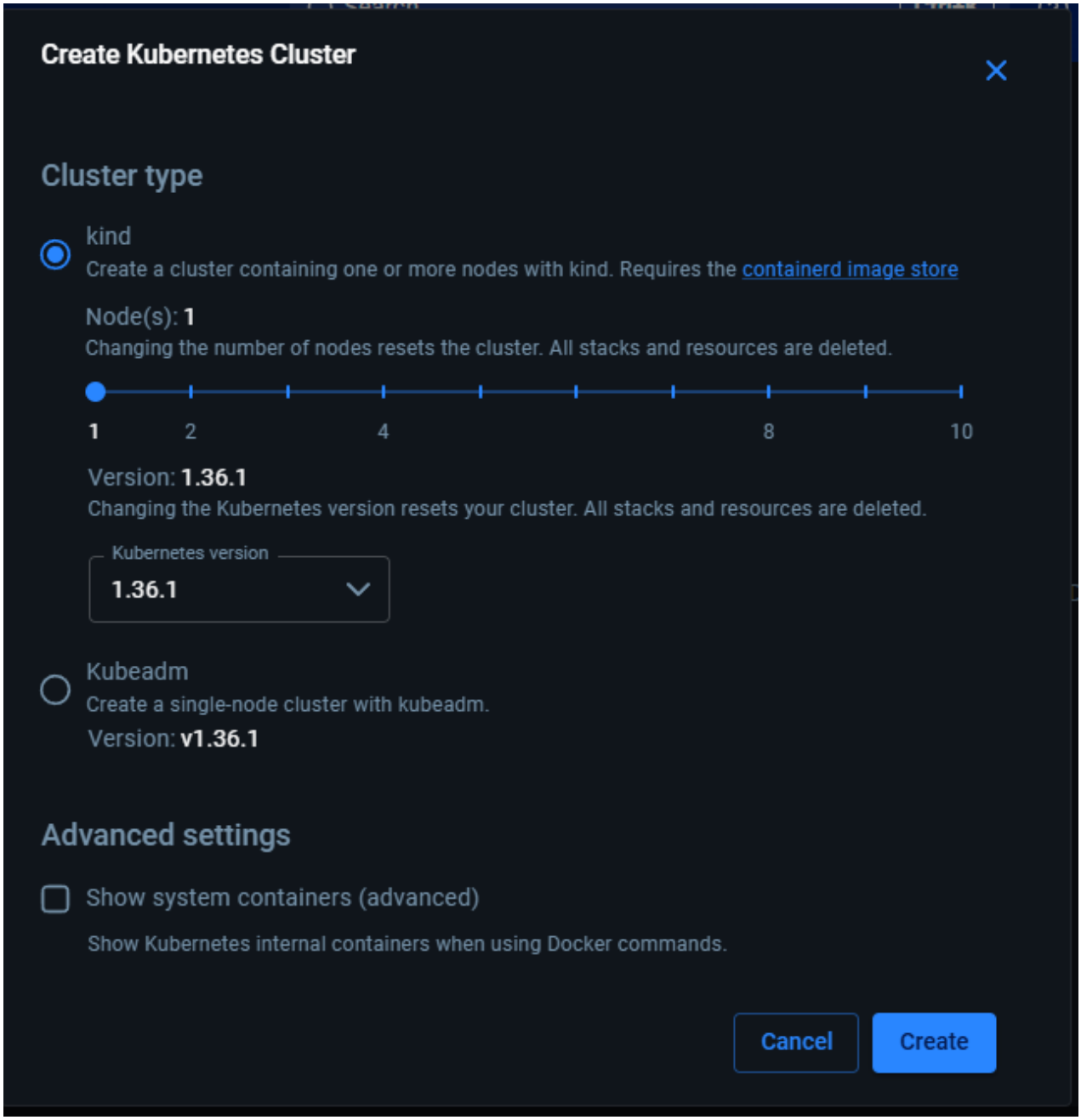
شکل 25 - خطای **dockerDesktopLinuxEngine** وقتی **Docker Desktop** خاموش بود

فاز سوم - Prometheus، Kubernetes و OpenCost واقعی

وضعیت فاز

پروژه‌ای Workload و چهار API و UI قابل دسترسی، OpenCost و Prometheus نصب، فعال Cluster kind هستند Running.

5-1. ساخت Kubernetes Cluster در Docker Desktop



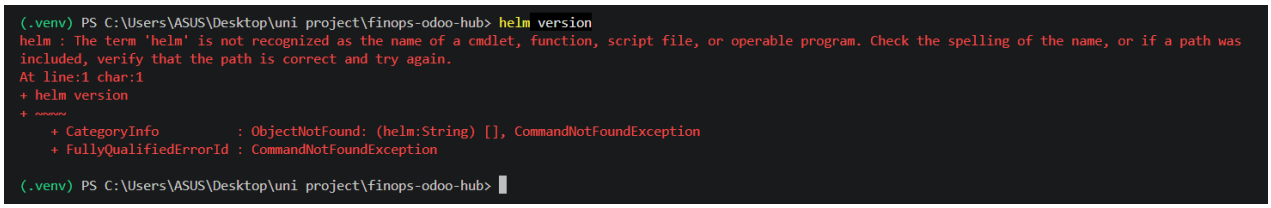
شکل 26 - ساخت **Cluster** نوع **kind** با یک **Node** و **Kubernetes 1.36.1**

برای محیط دانشگاهی یک Node کافی است و مصرف RAM/CPU را پایین نگه می‌دارد.

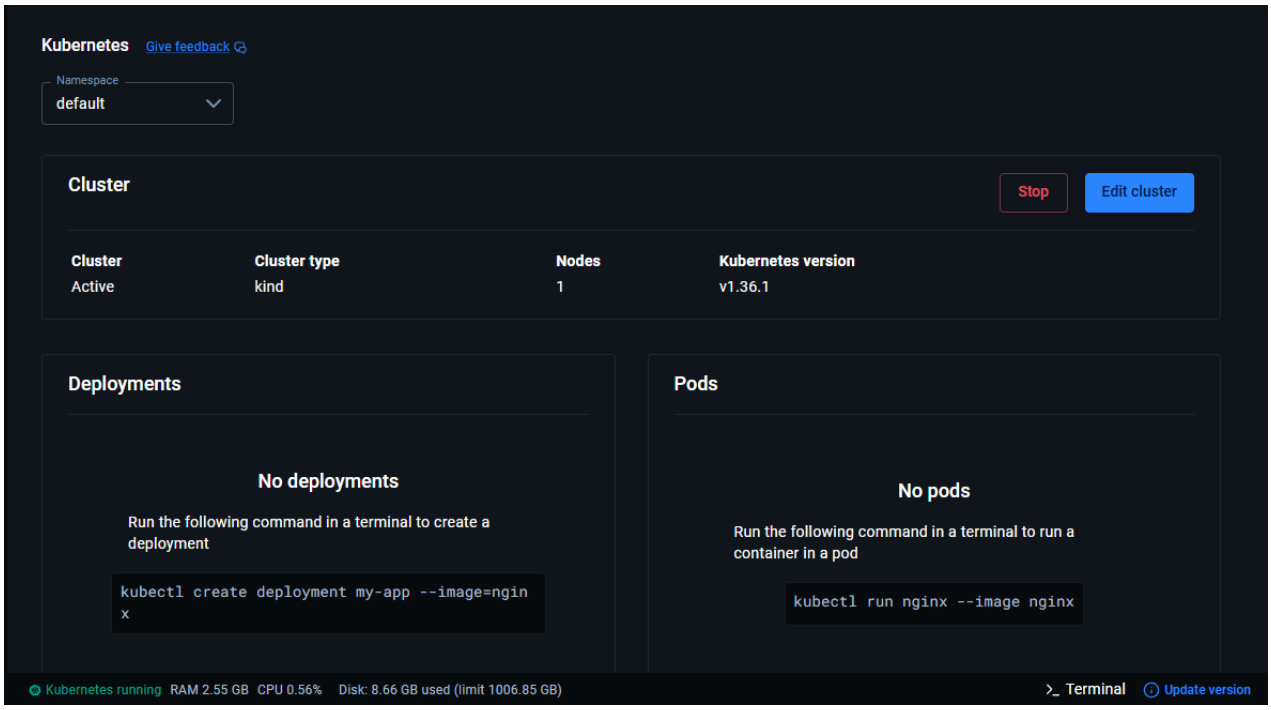
```
kubectl get nodes
kubectl version --client
```

```
NAME                 STATUS   ROLES           AGE   VERSION
desktop-control-plane   Ready    control-plane   58s   v1.36.1
(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub> kubectl version --client
Client Version: v1.36.1
Kustomize Version: v5.8.1
(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub> helm version
helm : The term 'helm' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was
included, verify that the path is correct and try again.
At line:1 char:1
+ helm version
```

شکل 27 - Node با STATUS=Ready؛ Helm هنوز نصب نبود



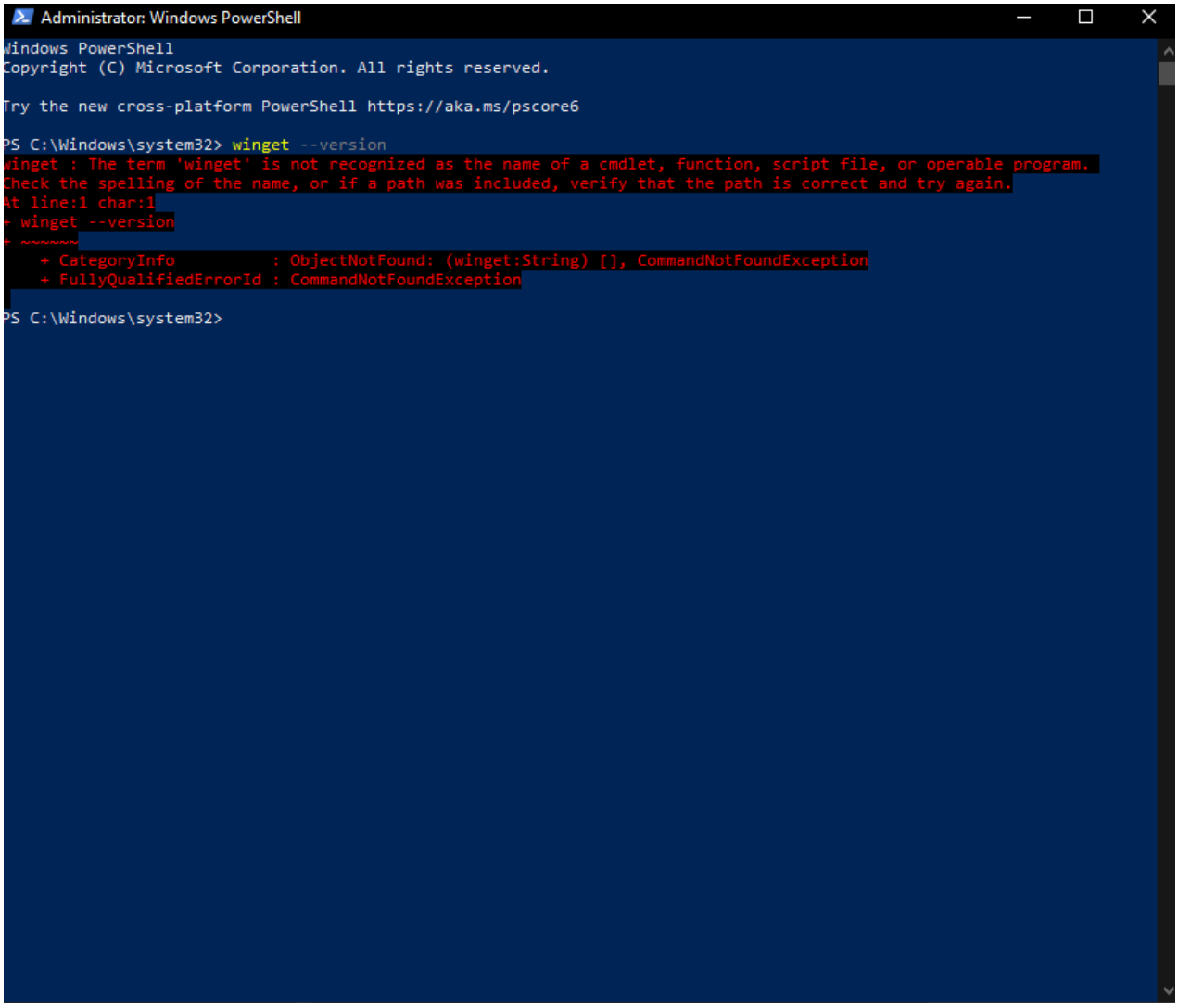
شکل 28 - خطای helm is not recognized



شکل 29 - Docker Desktop: Kubernetes Active، kind، یک Node

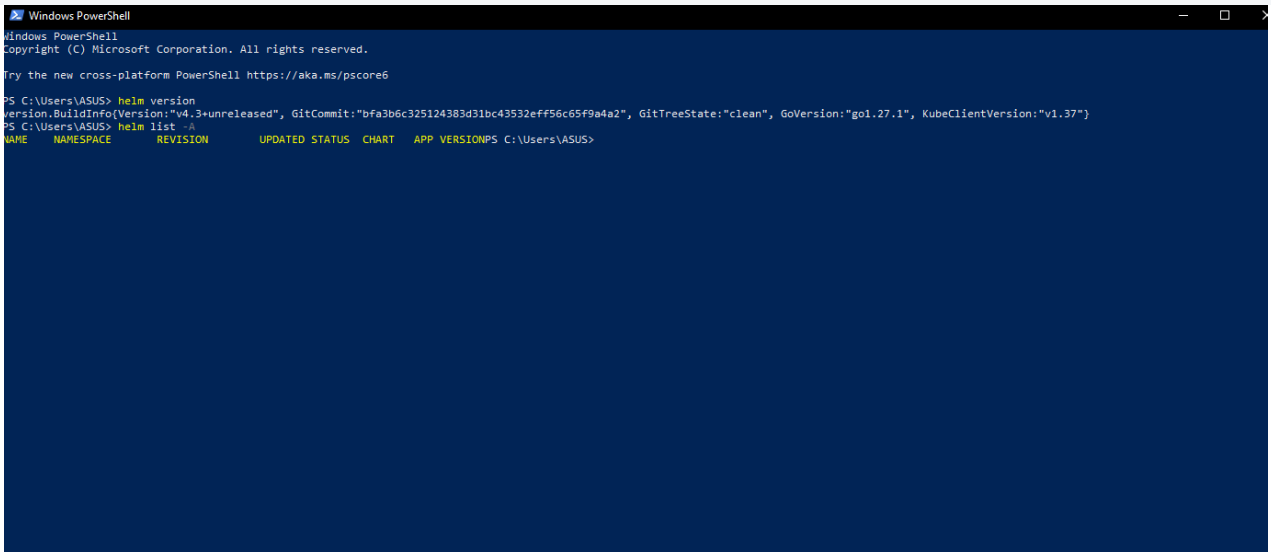
5-2. نصب Helm در Windows

winget روی این Windows نصب نبود؛ بنابراین Helm به‌صورت باینری رسمی نصب و مسیر C:\helm به PATH اضافه شد.



شکل 30 - **winget is not recognized**؛ انتخاب نصب دستی **Helm**

```
# پس از قرار دادن helm.exe در C:\helm و افزودن C:\helm به PATH
helm version
helm list -A
```



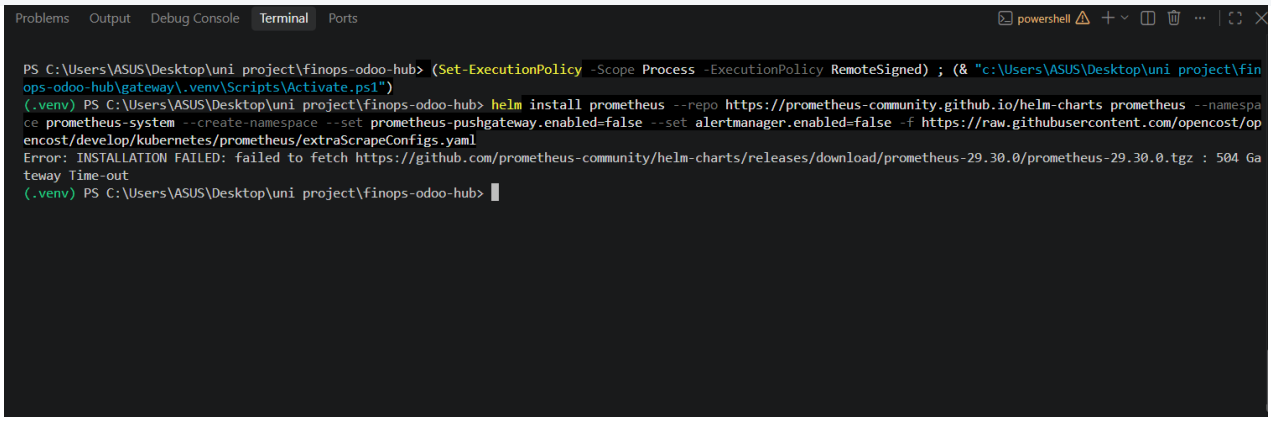
شکل 31 - نصب موفق **Helm** و اتصال به **Kubernetes**

5-3. نصب Prometheus

Prometheus prerequisite رسمی OpenCost است و Metrics مربوط به CPU، Namespace، Pod و Node را نگه می‌دارد.

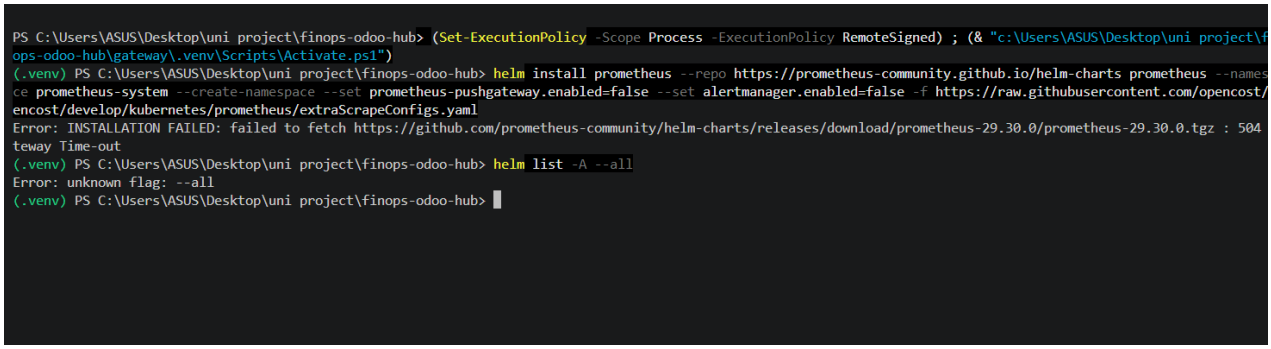
فرمان استاندارد

```
helm install prometheus --repo https://prometheus-community.github.io/helm-charts prometheus --
namespace prometheus-system --create-namespace --set prometheus-pushgateway.enabled=false --set
alertmanager.enabled=false -f
https://raw.githubusercontent.com/opencost/opencost/develop/kubernetes/prometheus/
extraScrapeConfigs.yaml
```



شکل 32 - خطای 504 **Gateway Time-out** هنگام دریافت **Chart** از **GitHub Releases**

در این سیستم Helm 4 فلگ --all را برای helm list نپذیرفت؛ helm list -A کافی بود. سپس برای دور زدن 504 از OCI registry استفاده شد.



شکل 33 - تفاوت **Helm 4**: فلگ **--all** ناشناخته

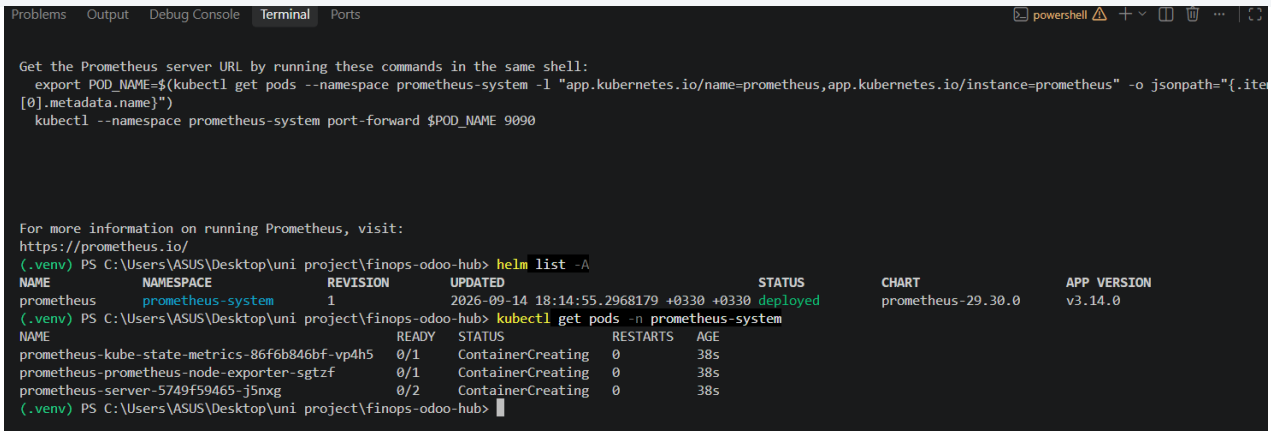
Fallback موفق با OCI

```
Invoke-WebRequest -Uri "https://raw.githubusercontent.com/opencost/opencost/develop/kubernetes/
prometheus/extraScrapeConfigs.yaml" -OutFile ".\extraScrapeConfigs.yaml"

helm install prometheus oci://ghcr.io/prometheus-community/charts/prometheus `
  --namespace prometheus-system `
```

```
  --create-namespace `
  --set prometheus-pushgateway.enabled=false `
  --set alertmanager.enabled=false `
  -f .\extraScrapeConfigs.yaml

kubectl get pods -n prometheus-system
```



شکل 34 - **Prometheus deployed**؛ Podها در حال **ContainerCreating**

5-4. نصب OpenCost

```
helm repo add opencost-charts https://opencost.github.io/opencost-helm-chart
helm repo update
helm install opencost opencost-charts/opencost --namespace opencost --create-namespace
kubectl get pods -n opencost
```

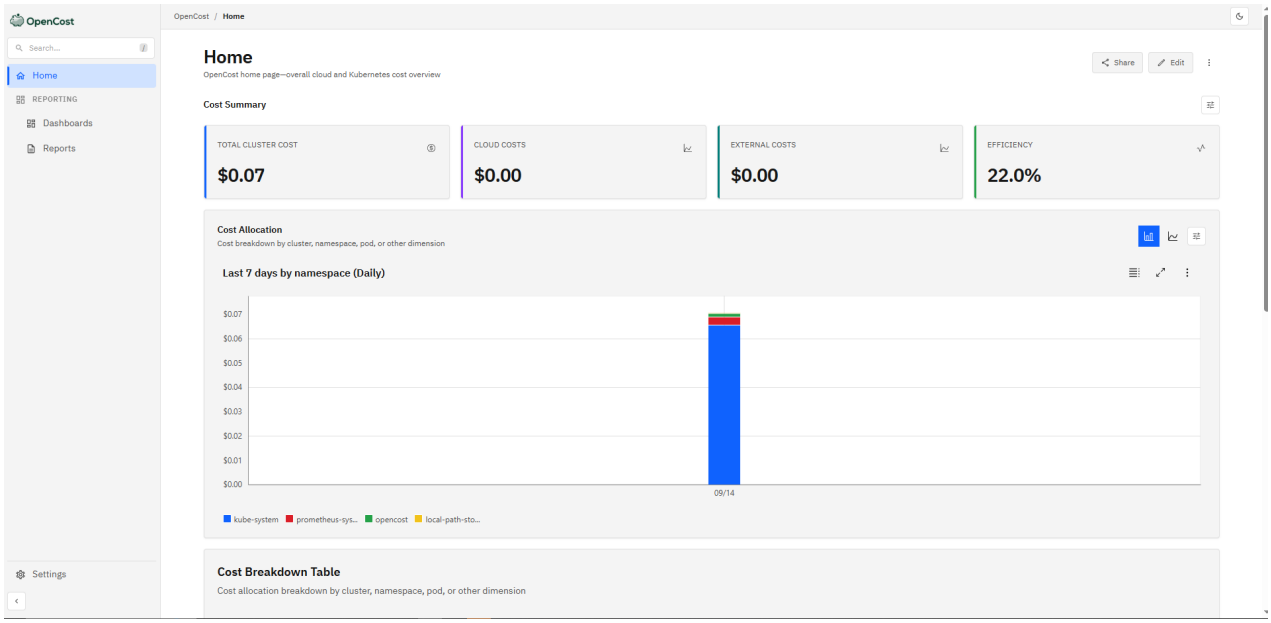
5-5. Port Forward و UI/API

Port Forward

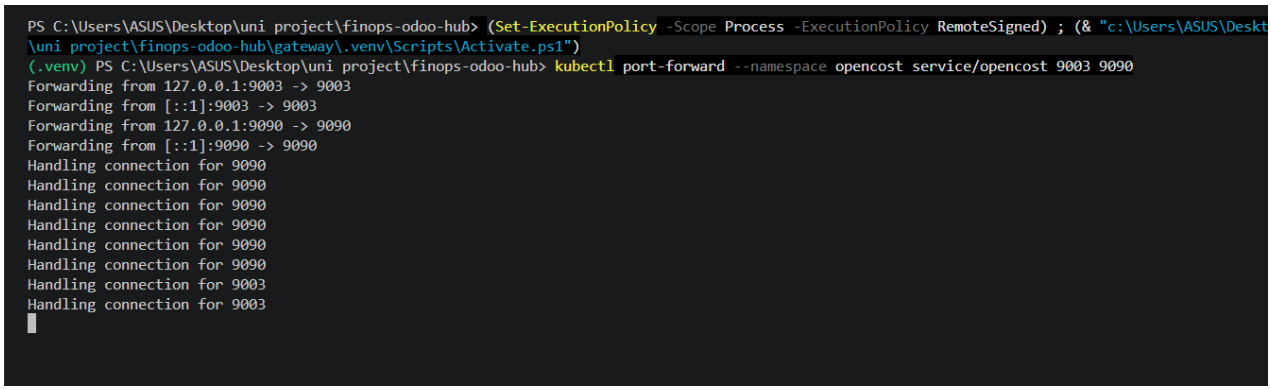
```
kubectl port-forward --namespace opencost service/opencost 9003 9090
```

- OpenCost API: http://localhost:9003

- OpenCost UI: http://localhost:9090



شکل 35 - OpenCost UI و Cost Summary اولیه



شکل 36 - Port Forward فعال روی 9003 و 9090

5-6. ساخت Namespace و Workloadهای پروژه

```
kubectl create namespace e-commerce
kubectl create namespace data-platform
kubectl create namespace crm
kubectl create namespace ai-platform

kubectl create deployment web --image=nginx:alpine -n e-commerce
kubectl create deployment crm-app --image=nginx:alpine -n crm
kubectl create deployment data-app --image=nginx:alpine -n data-platform
kubectl create deployment ai-app --image=nginx:alpine -n ai-platform

kubectl get pods -A
```

```
prometheus-system    prometheus-server-5749f59465-j5nxg              2/2     Running            0          148m
(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub> kubectl get pods -A
NAMESPACE            NAME                                            READY   STATUS    RESTARTS   AGE
ai-platform          ai-app-7bd86556bf-4cbkf                         1/1     Running   0          4m30s
crm                  crm-app-5554b48555-wtbp5                        1/1     Running   0          4m42s
data-platform        data-app-7cdcc65d75-6kfm5                       1/1     Running   0          4m36s
e-commerce           web-5d6b5c7df9-gfwjn                            1/1     Running   0          4m48s
kube-system          coredns-589f44dc88-p2m8n                        1/1     Running   0          3h11m
kube-system          coredns-589f44dc88-zlqqp                        1/1     Running   0          3h11m
kube-system          etcd-desktop-control-plane                      1/1     Running   0          3h11m
kube-system          kindnet-rpkfn                                   1/1     Running   0          3h11m
kube-system          kube-apiserver-desktop-control-plane            1/1     Running   0          3h11m
kube-system          kube-controller-manager-desktop-control-plane   1/1     Running   0          3h11m
kube-system          kube-proxy-jvwlj                                1/1     Running   0          3h11m
kube-system          kube-scheduler-desktop-control-plane            1/1     Running   0          3h11m
local-path-storage   local-path-provisioner-855c7b7774-lzgxq         1/1     Running   0          3h11m
opencost             opencost-85dc5987dd-vgrvs                       2/2     Running   0          112m
prometheus-system    prometheus-kube-state-metrics-86f6b846bf-vp4h5  1/1     Running   0          153m
prometheus-system    prometheus-prometheus-node-exporter-sgtzf       1/1     Running   0          153m
prometheus-system    prometheus-server-5749f59465-j5nxg              2/2     Running   0          153m
(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub>
```

شکل 37 - چهار **Workload** پروژه‌ای به همراه **Prometheus** و **OpenCost** همگی **Running**

5-7. Resource Requestهای متفاوت

برای ایجاد سناریوی Cost Allocation معنی‌دار، Request و Limit چهار Workload متفاوت تعیین شد. طبق Specification OpenCost، برای CPU/Memory Cost بر مبنای بزرگ‌ترِ request و usage تخصیص داده می‌شود.

```
kubectl set resources deployment web -n e-commerce --requests=cpu=200m,memory=128Mi --
limits=cpu=400m,memory=256Mi
kubectl set resources deployment crm-app -n crm --requests=cpu=100m,memory=64Mi --
limits=cpu=200m,memory=128Mi
kubectl set resources deployment data-app -n data-platform --requests=cpu=350m,memory=256Mi --
limits=cpu=700m,memory=512Mi
kubectl set resources deployment ai-app -n ai-platform --requests=cpu=600m,memory=512Mi --
limits=cpu=1000m,memory=768Mi
```

پروژه	CPU Request	RAM Request
CRM	100m	64Mi
E-Commerce	200m	128Mi
Data Platform	350m	256Mi
AI Platform	600m	512Mi

5-8. تست Allocation API

```
$r = Invoke-RestMethod "http://localhost:9003/allocation?
window=30m&aggregate=namespace&resolution=1m"
$r.code

$r.data[0].PSObject.Properties |
Where-Object { $_.Name -in @("e-commerce","data-platform","crm","ai-platform") } |
ForEach-Object {
    [PSCustomObject]@{
        Namespace = $_.Name
        TotalCost = [math]::Round($_.Value.totalCost, 6)
        CPUCost   = [math]::Round($_.Value.cpuCost, 6)
```

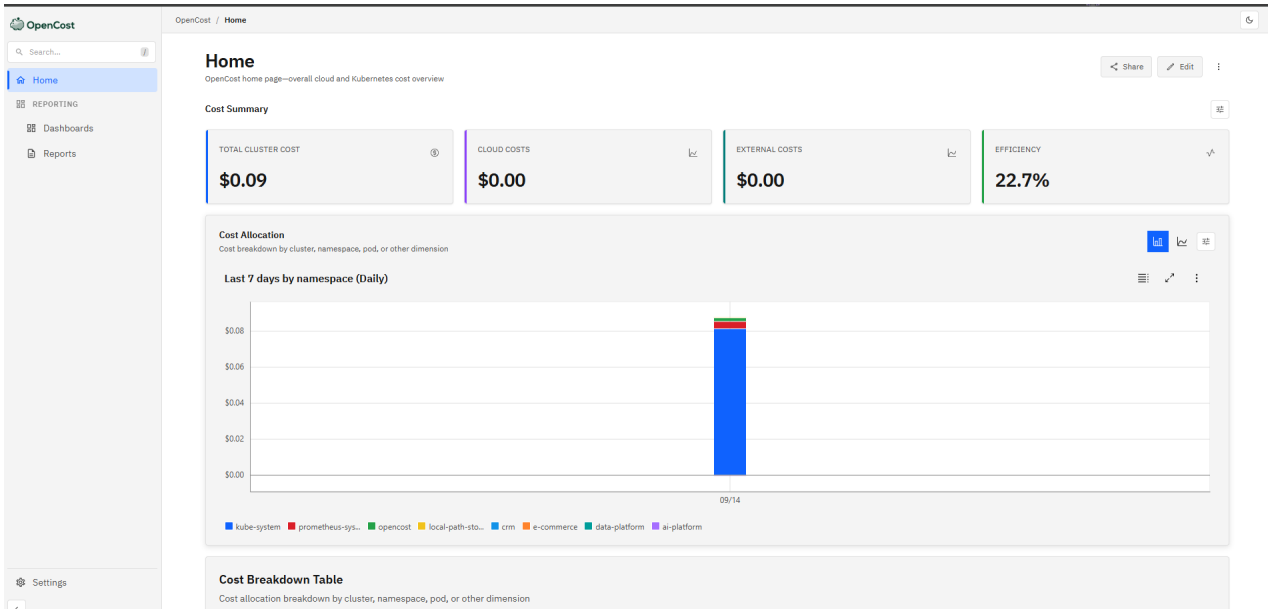
```
        RAMCost   = [math]::Round($_.Value.ramCost, 6)
    }
} | Format-Table
```

```
200
(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub> $r.data[0].PSObject.Properties |
>> Where-Object { $_.Name -in @("e-commerce","data-platform","crm","ai-platform") } |
>> ForEach-Object {
>>     [PSCustomObject]@{
>>         Namespace = $_.Name
>>         TotalCost = [math]::Round($_.Value.totalCost, 6)
>>         CPUCost   = [math]::Round($_.Value.cpuCost, 6)
>>         RAMCost   = [math]::Round($_.Value.ramCost, 6)
>>     }
>> } | Format-Table

Namespace     TotalCost CPUCost RAMCost
---------     --------- ------- -------
ai-platform           0       0       0
crm                   0       0       0
data-platform         0       0       0
e-commerce      0.00001       0       0


(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub>
```

شکل 38 - **Allocation API** کد 200 و **Cost**های بسیار کوچک روی **Cluster** محلی



شکل 39 - **OpenCost UI** با **namespace**های **data-platform**، **e-commerce**، **crm** و **ai-platform**

تفسیر عددهای کوچک

در kind محلی Cloud Bill واقعی وجود ندارد. OpenCost Allocation را با مدل قیمت‌گذاری محاسبه می‌کند و Workloadها هم سبک‌اند؛ بنابراین صفر/عدد بسیار کوچک در Window کوتاه طبیعی است. برای Demo خواناتر می‌توان در فاز تکمیل از Custom Pricing استفاده کرد.

فاز چهارم - تغییر Gateway از Mock به OpenCost واقعی

وضعیت فاز

provider_mode روی opencost تأیید شده است. در آخرین وضعیت ثبت‌شده، Dashboard به علت تقسیم بر صفر 500 می‌دهد و اولین Sync واقعی هنوز باید Verify شود.

6-1. Adapter پیشنهادی OpenCost Allocation

Adapter اولیه Starter برای مدل دیگری از OpenCost بود. برای سناریوی Kubernetes واقعی، endpoint استاندارد /allocation با aggregate=namespace استفاده می‌شود و چهار namespace به چهار Project تجاری Mapping می‌شوند.

پروژه و مؤلفه هزینه **Mapping**

```
PROJECTS = {
    "e-commerce": {"project": "E-Commerce", "department": "Digital"},
    "data-platform": {"project": "Data Platform", "department": "Data"},
    "crm": {"project": "CRM", "department": "Sales"},
    "ai-platform": {"project": "AI Platform", "department": "AI"},
}

COST_COMPONENTS = [
    ("cpuCost", "CPU", "Compute"),
    ("ramCost", "RAM", "Compute"),
    ("gpuCost", "GPU", "Compute"),
    ("pvCost", "Persistent Volume", "Storage"),
    ("networkCost", "Networking", "Network"),
    ("loadBalancerCost", "Load Balancer", "Network"),
    ("sharedCost", "Shared", "Other"),
    ("externalCost", "External", "Other"),
]
```

منطق fetch باید GET {base_url}/allocation را با aggregate=namespace، window و resolution=1m صدا بزند، سپس Cost componentهای مثبت را به NormalizedCost تبدیل کند. external_id پایدار باید مانع Duplicate شود.

6-2. Upsert در Gateway

منطق پیشنهادی Create/Update

```
async def sync_provider(db: Session) -> dict:
    settings = get_settings()
    if settings.provider_mode != "opencost":
        return {"provider": settings.provider_mode, "created": 0, "updated": 0}

    provider = OpenCostProvider(settings.opencost_base_url)
    rows = await provider.fetch_costs(settings.opencost_window)
    created = updated = 0

    for row in rows:
        record = db.scalar(select(CostRecord).where(CostRecord.external_id == row.external_id))
        if record:
            for field_name, value in row.__dict__.items():
                setattr(record, field_name, value)
            updated += 1
        else:
            db.add(CostRecord(**row.__dict__))
            created += 1

    db.commit()
    return {"provider": "opencost", "created": created, "updated": updated, "received": len(rows)}
```

6-3. جلوگیری از Seed ماکت در حالت opencost

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(bind=engine)
    with SessionLocal() as db:
        if settings.provider_mode == "mock":
            seed_demo_data(db)
    yield
```

6-4. ساخت فایل .env واقعی

مشکل مهم این بود که فایل .env ریشه پروژه اصلاً وجود نداشت؛ بنابراین Compose مقدار پیش‌فرض mock را استفاده می‌کرد.

```
(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub> Get-Content .\.env
Get-Content : Cannot find path 'C:\Users\ASUS\Desktop\uni project\finops-odoo-hub\.env' because it does not exist.
At line:1 char:1
+ Get-Content .\.env
+ ~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (C:\Users\ASUS\D...s-odoo-hub\.env:String) [Get-Content], ItemNotFoundException
    + FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.GetContentCommand

(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub>
```

شکل 40 - خطای **Get-Content**: فایل **.env** در ریشه پروژه وجود نداشت

ساخت .env و Recreate Gateway

```
Copy-Item .\.env.example .\.env

@"
FINOPS_PROVIDER_MODE=opencost
```

```
FINOPS_OPENCOST_BASE_URL=http://host.docker.internal:9003
FINOPS_OPENCOST_WINDOW=1h
"@ | Set-Content -Encoding ascii .\.env

Get-Content .\.env
docker compose config | Select-String "FINOPS_"
docker compose up -d --build --force-recreate finops-gateway
docker compose exec finops-gateway printenv FINOPS_PROVIDER_MODE
```

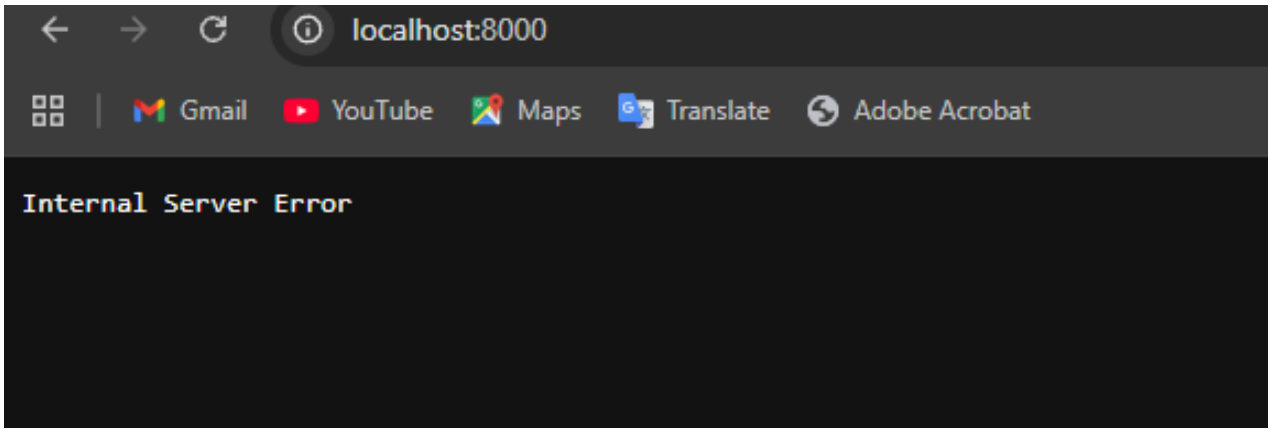
6-5. تأیید Health روی opencost

```
n render
finops-gateway-1  |     self.environment.handle_exce
ption()
finops-gateway-1  |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~^^
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/jinja2/environment.py", line 942, in
 handle_exception
finops-gateway-1  |     raise rewrite_traceback_stac
k(source=source)
finops-gateway-1  |   File "/app/app/templates/dashb
finops-gateway-1  |     <div class="bar-row"><div cl
ass="bar-label"><span>{{ name }}</span><b>${{ '%.2f'
|format(value) }}</b></div><div class="track"><div c
lass="fill" style="width: {{ (value / maxv * 100)|ro
und(1) }}%"></div></div></div>
finops-gateway-1  |     ^^^^^^^^^^^^
n by zero
(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-
odoo-hub> Invoke-RestMethod http://localhost:8000/health

status provider_mode version
------ ------------- -------
ok     opencost      0.1.0
```

شکل 41 - **Health: provider_mode = opencost**

6-6. خطای فعلی Dashboard: تقسیم بر صفر



شکل 42 - **Internal Server Error** روی **Dashboard** پس از خالی‌شدن **Cost**ها

```
n by zero
(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-
odoo-hub> docker compose logs --tail=120 finops-gate
way
finops-gateway-1  |            ~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~^
finops-gateway-1  |         "dashboard.html",
finops-gateway-1  |         ^^^^^^^^^^^^^^^^^
finops-gateway-1  |     ...<5 lines>...
finops-gateway-1  |         },
finops-gateway-1  |         ^^
finops-gateway-1  |     )
finops-gateway-1  |     ^
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/starlette/templating.py", line 210,
in TemplateResponse
finops-gateway-1  |     return _TemplateResponse(
finops-gateway-1  |         template,
finops-gateway-1  |     ...<4 lines>...
finops-gateway-1  |         background=background,
finops-gateway-1  |     )
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/starlette/templating.py", line 41, i
n __init__
finops-gateway-1  |     content = template.render(co
ntext)
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/jinja2/environment.py", line 1295, i
n render
finops-gateway-1  |     self.environment.handle_exce
ption()
finops-gateway-1  |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~^^
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/jinja2/environment.py", line 942, in
 handle_exception
```

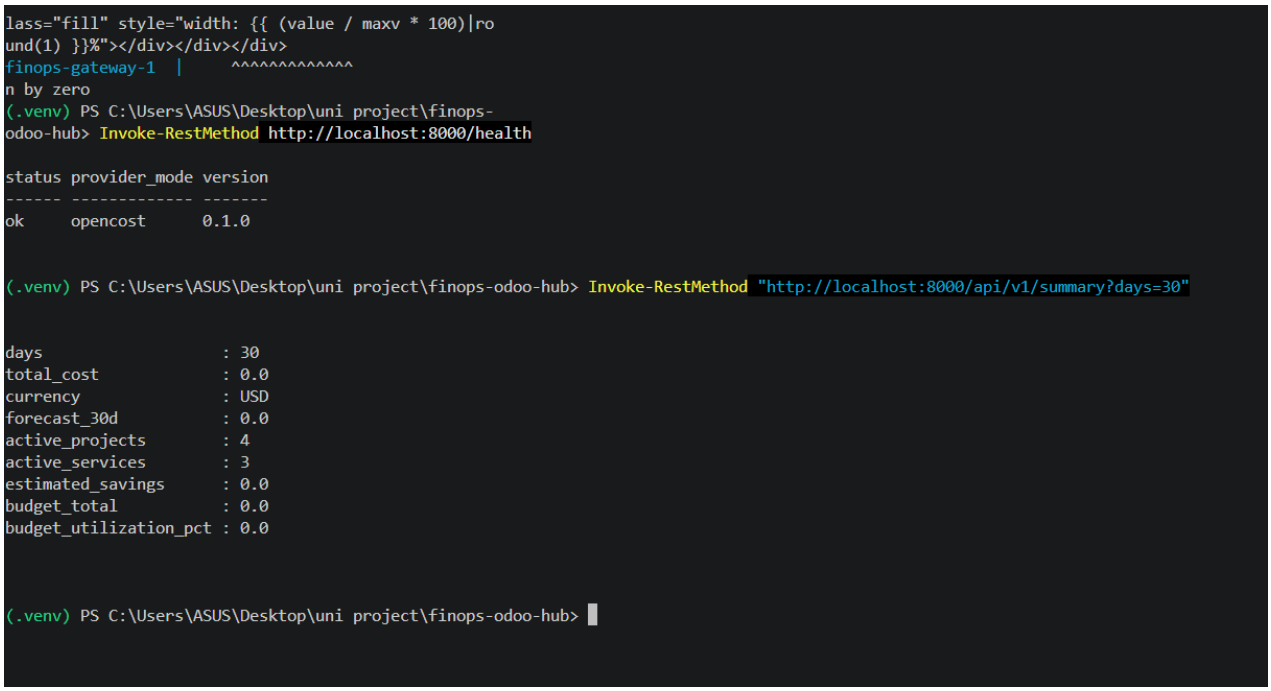
شکل 43 - **Traceback** اولیه از **Jinja Template**

```
finops-gateway-1  |     content = template.render(co
ntext)
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/jinja2/environment.py", line 1295, i
n render
finops-gateway-1  |     self.environment.handle_exce
ption()
finops-gateway-1  |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~^^
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/jinja2/environment.py", line 942, in
 handle_exception
finops-gateway-1  |     raise rewrite_traceback_stac
k(source=source)
finops-gateway-1  |   File "/app/app/templates/dashb
oard.html", line 34, in top-level template code
finops-gateway-1  |     <div class="bar-row"><div cl
ass="bar-label"><span>{{ name }}</span><b>${{ '%.2f'
|format(value) }}</b></div><div class="track"><div c
lass="fill" style="width: {{ (value / maxv * 100)|ro
und(1) }}%"></div></div></div>
finops-gateway-1  |     ^^^^^^^^^^^^^
finops-gateway-1  | ZeroDivisionError: float divisio
n by zero
finops-gateway-1  | INFO:     172.18.0.1:35984 - "GE
T / HTTP/1.1" 500 Internal Server Error
finops-gateway-1  | ERROR:    Exception in ASGI appl
ication
finops-gateway-1  | Traceback (most recent call last
):
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/uvicorn/protocols/http/httptools_imp
l.py", line 421, in run_asgi
finops-gateway-1  |     result = await app(  # type:
 ignore[func-returns-value]
finops-gateway-1  |                  ^^^^^^^^^^^^^^^^^^
```

شکل 44 - محل دقیق خطا: **value / maxv** و **ZeroDivisionError**

Summary API سالم بود و مقدارها صفر بودند؛ بنابراین مشکل از API نبود و فقط Template فرض کرده بود

maxv همیشه بزرگ‌تر از صفر است.



شکل 45 - **Summary** در حالت **opencost** قبل از **Sync** واقعی: هزینه و بودجه صفر

رفع **ZeroDivision**

```
# dashboard.html - من | نسخه
{{ ((value / maxv * 100) if maxv else 0)|round(1) }}

# سپس
docker compose up -d --build --force-recreate finops-gateway
```

Checkpoint فعلی

در لحظه تدوین این سند: health=opencost تأیید شده؛ OpenCost API کد 200 می‌دهد؛ Workloadها Running هستند؛ رفع ZeroDivision و اولین Sync واقعی Gateway باید در ادامه اجرا و Verify شود.

فصل 7

ادامه مسیر - تکمیل پروژه تا نسخه نهایی و قابل ارائه

این فصل برنامه اجرایی است

مراحل این فصل هنوز به‌طور کامل اجرا نشده‌اند. ترتیب آن‌ها طوری طراحی شده که از وضعیت فعلی مستقیم به یک Demo پایدار، قابل دفاع و قابل Deploy برسیم.

7-1. مرحله A - رفع Dashboard و اولین Sync واقعی

1) اصلاح تمام عبارت‌های value / maxv در dashboard.html با شرط if maxv else 0.

2) Rebuild کردن فقط finops-gateway.

3) باز نگه داشتن Port Forward OpenCost روی 9003/9090.

4) تست دسترسی Container به OpenCost با httpx.

5) اجرای POST /api/v1/sync/provider.

6) تأیید received > 0 و provider=opencost.

7) اجرای Sync دوم و تأیید Create=0 و Update>0 (عدم Duplicate).

8) باز کردن api/v1/costs/ و بررسی provider=OpenCost و source=opencost-allocation.

9) Refresh Dashboard و بررسی اینکه بدون 500 باز می‌شود.

```
# 1) اتصال Container به OpenCost
docker compose exec finops-gateway python -c "import httpx;
r=httpx.get('http://host.docker.internal:9003/allocation',
params={'window':'1h','aggregate':'namespace','resolution':'1m'}); print(r.status_code);
print(r.json().get('code'))"

# انتظار: 200 و 200

# 2) Sync واقعی
Invoke-RestMethod -Method Post http://localhost:8000/api/v1/sync/provider

# 3) مشاهده رکوردها
```

```
Invoke-RestMethod "http://localhost:8000/api/v1/costs?days=7
```

7-2. مرحله B - Persistence واقعی SQLite

در Compose اولیه Volume روی /app/data mount شده اما database_url پیش‌فرض sqlite:///./finops.db است؛ بنابراین ممکن است فایل دیتابیس خارج از Volume نوشته شود. برای پایداری پس از Recreate، database URL باید صریحاً به Volume اشاره کند.

اصلاح پیشنهادی Persistence

```
# docker-compose.yml / finops-gateway environment
FINOPS_DATABASE_URL: sqlite:////app/data/finops.db

# سپس
docker compose up -d --build --force-recreate finops-gateway
```

بعد از این اصلاح، Recreate Container نباید Costهای Syncشده را از بین ببرد. تست: Sync → تعداد رکورد → force-recreate → همان تعداد رکورد.

7-3. مرحله C - جدا کردن Budget از Mock Data

Budget یک داده مدیریتی است و نباید وابسته به Provider Cost باشد. در نسخه نهایی، بودجه‌ها همیشه Seed/Config می‌شوند ولی Mock Cost فقط در provider_mode=mock ساخته می‌شود.

طرح پیشنهادی Budget مستقل

```
BUSINESS_BUDGETS = [
    ("E-Commerce", 900.0),
    ("Data Platform", 1150.0),
    ("CRM", 650.0),
    ("AI Platform", 800.0),
]

def seed_budgets(db):
    for project, amount in BUSINESS_BUDGETS:
        row = db.scalar(select(Budget).where(Budget.project == project))
        if row:
            row.monthly_budget = amount
        else:
            db.add(Budget(project=project, monthly_budget=amount, currency="USD"))
    db.commit()

# lifespan
with SessionLocal() as db:
    seed_budgets(db)
    if settings.provider_mode == "mock":
        seed_demo_data(db)
```

7-4. مرحله D - خواناتر کردن Cost در Demo با Custom Pricing

برای kind محلی رقم‌ها بسیار کوچک‌اند. OpenCost اجازه Custom Pricing برای محیط on-prem/default می‌دهد. این مرحله اختیاری است ولی برای ارائه دانشگاهی مفید است؛ باید واضح گفته شود این نرخ‌ها «قیمت قراردادی/فرضی» هستند، نه Bill واقعی Cloud.

اختیاری Custom Pricing

```
# نمونه values-opencost.yaml - اعداد صرفاً مثال آموزشی هستند
opencost:
  customPricing:
    enabled: true
    provider: default
    costModel:
      description: "Academic demo pricing"
      CPU: 12.00
      RAM: 10.50
      storage: 40.25

# اعمال
helm upgrade opencost opencost-charts/opencost -n opencost -f values-opencost.yaml
```

7-5. مرحله E - پاک‌سازی Mock در Odoo و Sync واقعی

در View فعلی delete=false است؛ بنابراین توصیه نمی‌شود 480 رکورد را دستی از UI حذف کنیم. دو راه تمیز وجود دارد: (1) ساخت Action مخصوص Purge Mock در ماژول، یا (2) برای Demo نهایی ایجاد یک دیتابیس Odoo تازه و Sync فقط Real Data. راه اول برای پروژه حرفه‌ای‌تر است.

متد پیشنهادی Purge Mock

```
# پیشنهاد در finops_cost.py
@api.model
def purge_mock_records(self):
    records = self.search([("provider", "=", "OpenCost-Mock")])
    count = len(records)
    records.unlink()
    return count
```

بعد از پاک‌سازی، چهار Project واقعی Odoo باید وجود داشته باشند: CRM، Data Platform، E-Commerce و AI Platform. سپس FinOps → Sync Now اجرا می‌شود. در Cloud Costs باید Provider=OpenCost و Source=opencost-allocation دیده شود و Odoo Project پر باشد.

7-6. مرحله F - Scheduled Sync End-to-End

ماژول از قبل ir.cron شش‌ساعته دارد. در نسخه نهایی باید یک سناریوی تست ثبت شود: Cost در Kubernetes تغییر می‌کند → OpenCost Metric به‌روز می‌شود → Gateway Sync می‌کند → Odoo Cron رکورد را Update می‌کند. برای Demo می‌توان Sync Now را دستی اجرا کرد و Cron را به‌عنوان قابلیت Automation توضیح داد.

7-7. مرحله G - Recommendation واقعی (توسعه پیشنهادی)

در Real Mode جدول Recommendation ماکت خالی می‌شود. برای کامل‌تر شدن FinOps، می‌توان از نسبت Request/Usage و Efficiency برای پیشنهاد Rightsizing استفاده کرد. این بخش برای MVP الزامی نیست اما امتیاز دانشگاهی خوبی دارد.

- اگر CPU efficiency بسیار پایین و CPU request بالا باشد → پیشنهاد کاهش CPU Request.
- اگر RAM efficiency پایین باشد → پیشنهاد کاهش Memory Request.
- اگر workload در ساعات مشخص تقریباً بدون مصرف باشد → پیشنهاد Schedule/Scale-to-zero.
- Estimated Saving = Cost فعلی - Cost تخمینی پس از Rightsizing؛ باید در گزارش با عنوان «تخمینی» ذکر شود.

7-8. مرحله H - Mapping پویا به جای دیکشنری Hard-coded

برای نسخه بهتر، Namespaceها با Labelهای project، department و environment برچسب‌گذاری می‌شوند و Gateway به‌جای PROJECTS ثابت، اطلاعات را از Labelها می‌خواند. OpenCost API می‌تواند بر اساس label نیز Aggregate شود. این تغییر پروژه را برای افزودن Project پنجم بدون تغییر کد آماده می‌کند.

نمونه **Labeling** (در نام **Label** بهتر است از **dash** استفاده شود)

```
kubectl label namespace e-commerce project=E-Commerce department=Digital environment=production
kubectl label namespace data-platform project="Data-Platform" department=Data
environment=production
kubectl label namespace crm project=CRM department=Sales environment=production
kubectl label namespace ai-platform project="AI-Platform" department=AI environment=production
```

7-9. مرحله I - تاریخچه و Windowهای بزرگ‌تر

Window یک‌ساعته برای Debug مناسب است اما برای گزارش نهایی باید حداقل 24h و سپس 7d آزمایش شود. برای History دقیق‌تر می‌توان step را نیز مشخص کرد تا Allocation Sets چند بازه زمانی داشته باشند و external_id شامل timestamp window شود. در غیر این صورت Sync روزانه ممکن است فقط Snapshot روز جاری را نگه دارد.

```
# نمونه API تاریخی
http://localhost:9003/allocation?window=7d&step=1d&resolution=10m&aggregate=namespace
```

7-10. مرحله J - Deployment نهایی

برای پروژه دانشگاهی دو سطح Deploy تعریف می‌شود:

کاربرد	معماری	سطح
ارائه و دفاع؛ کم‌هزینه و قابل نمایش روی لپ‌تاپ	Docker Desktop + kind + Prometheus/OpenCost + Compose Odoo/Gateway/Postgres	Demo محلی قابل تکرار
کار آینده؛ مناسب سازمان واقعی	Linux host یا VM برای Odoo/Gateway/PostgreSQL + Kubernetes مستقل برای OpenCost + Ingress/TLS/Secrets	Production-like

7-11. اسکریپت یک‌کلیکی Demo

برای کم‌کردن ریسک روز ارائه، یک scripts/start_demo.ps1 ساخته شود که وضعیت Docker/Kubernetes را چک کند، Compose را بالا بیاورد، Port Forward را Start کند، Gateway Sync را بزند و URLها را چاپ کند. این کار اجرای دستی چندین Terminal را کاهش می‌دهد.

طرح اسکریپت Demo

```
# start_demo.ps1 طرح ساده
$ErrorActionPreference = 'Stop'
docker compose up -d
kubectl get nodes
Start-Process powershell -ArgumentList '-NoExit','-Command','kubectl port-forward --
```

```
address=0.0.0.0 -n opencost service/opencost 9003:9003 9090:9090'
Start-Sleep -Seconds 3
Invoke-RestMethod -Method Post http://localhost:8000/api/v1/sync/provider
Write-Host 'Gateway: http://localhost:8000'
Write-Host 'Odoo:    http://localhost:8069'
Write-Host 'OpenCost:http://localhost:9090'
```

7-12. تعریف Done نهایی پروژه

- OpenCost چهار پروژه را در UI و /allocation نشان دهد.
- Gateway health روی opencost باشد و Dashboard بدون 500 باز شود.
- POST sync/provider received>0 بدهد و Sync دوم Duplicate نسازد.
- Gateway Cost Records فقط Provider واقعی OpenCost را برای Demo نهایی نشان دهد.
- Odoo Mock Data پاک/فیلتر و Real Data Sync شده باشد.
- هر Cost Record به Odoo Project متناظر لینک شده باشد.
- Pivot و Graph Odoo کار کند.
- Budget مستقل نمایش داده شود.
- حداقل یک تست Restart/Persistence و یک تست خطای Provider ثبت شود.
- اسکرین‌شات‌های نهایی و نتایج وارد گزارش دانشگاهی شوند.
- Runbook روز ارائه و دستورات Recovery آماده باشد.

عیب‌یابی، خطاهای واقعی و راه بازیابی

8-1. ماتریس خطاها

نشانه	علت محتمل	راه‌حل
خطا cd gateway	از قبل داخل gateway هستی	Get-Location؛ دوباره cd نزن
pytest not recognized	pytest در venv نصب نیست	python -m pip install pytest
assert 4 == 3	تست هنوز Spec قدیمی دارد	assert را به 4 تغییر بده
Docker virtualization support not detected	WSL2/VirtualMachinePlatform ناقص	Virtualization را در Task Manager چک، WSL2 را repair کن
dockerDesktopLinuxEngine not found	Docker Desktop/Engine خاموش	Docker Desktop را باز و Engine را صبر کن running
helm not recognized	Helm نصب/PATH نیست	helm.exe در C:\helm و افزودن PATH
winget not recognized	App Installer/winget در دسترس نیست	نصب دستی Helm
Prometheus 504	GitHub release timeout	OCI chart از ghcr.io
helm list --all unknown	تفاوت Helm 4	helm list -A
Cost خیلی کم/صفر	Window کوتاه + محلی kind	صبر/Window بزرگ‌تر/Custom Pricing
env does not exist.	فایل واقعی ساخته نشده	Copy-Item .env.example .env
mock هنوز health	Container قدیمی یا env اشتباه	compose config + force-recreate
Dashboard 500	Template در maxv=0	if maxv else 0
Gateway به OpenCost نمی‌رسد	Port-forward فقط localhost یا بسته	address=0.0.0.0-- و host.docker.internal
Duplicate رکورد Odoo	external_id ناپایدار	external_id deterministic + Upsert
خالی Odoo Project	Project mismatch نام	چهار Project با نام دقیق یا Mapping مبتنی بر label

8-2. فرمان‌های Diagnostic ضروری

```
# Docker
docker info
docker compose ps
docker compose logs --tail=120 finops-gateway

# Gateway
Invoke-RestMethod http://localhost:8000/health
Invoke-RestMethod "http://localhost:8000/api/v1/summary?days=30"

# Kubernetes
kubectl get nodes
kubectl get pods -A
kubectl get svc -A

# Prometheus/OpenCost
kubectl get pods -n prometheus-system
kubectl get pods -n opencost
helm list -A

# OpenCost API
Invoke-RestMethod "http://localhost:9003/allocation?window=1h&aggregate=namespace&resolution=1m"

# Env داخل Container
docker compose exec finops-gateway printenv FINOPS_PROVIDER_MODE
```

8-3. Resetهای کنترل‌شده

```
# فقط Gateway را rebuild/recreate کن
docker compose up -d --build --force-recreate finops-gateway

# Compose را متوقف کن، بدون حذف volume
docker compose stop

# Kubernetes resources پروژه‌ای را ببین
kubectl get all -A

# حذف OpenCost در صورت نیاز
helm uninstall opencost -n opencost
kubectl delete namespace opencost

# حذف Prometheus (فقط اگر واقعاً لازم است)
helm uninstall prometheus -n prometheus-system
kubectl delete namespace prometheus-system
```

سناریوی ارائه دانشگاهی و دفاع فنی

9-1. ترتیب پیشنهادی Demo

1) در یک اسلاید معماری را نشان بده: Odoo Gateway OpenCost Prometheus Kubernetes.

2) OpenCost UI را باز کن و چهار namespace پروژه را نشان بده.

3) /allocation را Query کن و توضیح بده Cost واقعی از Metrics Kubernetes محاسبه می‌شود.

4) Gateway /health را نشان بده که provider_mode=opencost است.

5) Gateway Dashboard را باز کن و Cost Allocation نرمال‌شده را نشان بده.

6) Odoo FinOps Cloud Costs را باز کن و Provider=OpenCost را نشان بده.

7) Pivot View را بر اساس Project × Service نمایش بده.

8) Sync Now را اجرا کن و نشان بده Duplicate ایجاد نمی‌شود.

9) بودجه و Recommendation را به‌عنوان لایه تجاری/تحلیلی توضیح بده.

10) در پایان محدودیت Local kind را شفاف بگو و مسیر اتصال Cloud Billing واقعی را به‌عنوان Future Work ارائه کن.

9-2. جواب کوتاه به سؤال‌های محتمل استاد

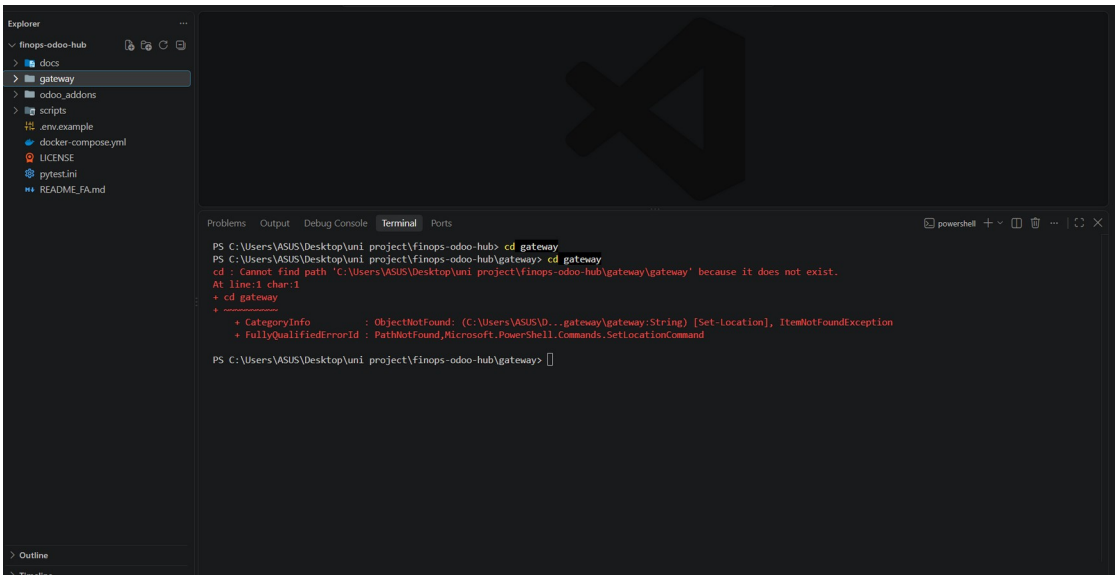
سؤال	پاسخ پیشنهادی
چرا OpenCost؟	متن‌باز، vendor-neutral، استاندارد برای Kubernetes Cost Allocation و API مناسب Integration.
چرا Odoo؟	ERP متن‌باز است و Cost را کنار Project و ساختار سازمانی قرار می‌دهد.

چرا Gateway؟	برای جداکردن مدل OpenCost از مدل Odoo، Normalization، ذخیره‌سازی، کنترل API و توسعه‌پذیری.
این Cost واقعی است؟	مصرف و Allocation Kubernetes واقعی است؛ روی kind محلی Bill واقعی CSP نداریم. نرخ Cost از مدل OpenCost/Custom Pricing می‌آید.
Mock چه شد؟	برای توسعه مرحله‌ای استفاده شد؛ در نسخه نهایی provider=opencost و Mock از مسیر Demo واقعی جدا می‌شود.
چطور Duplicate نمی‌سازی؟	external_id پایدار + Upsert در Gateway/Odoo.
چطور Scale می‌شود؟	Mapping را می‌توان label-based کرد، DB Gateway را ارتقا داد و OpenCost را روی Kubernetes واقعی Deploy کرد.
Automation چیست؟	Odoo Cron هر 6 ساعت و قابلیت اسکریپت Sync/Startup؛ در Production می‌توان Scheduler مستقل نیز اضافه کرد.

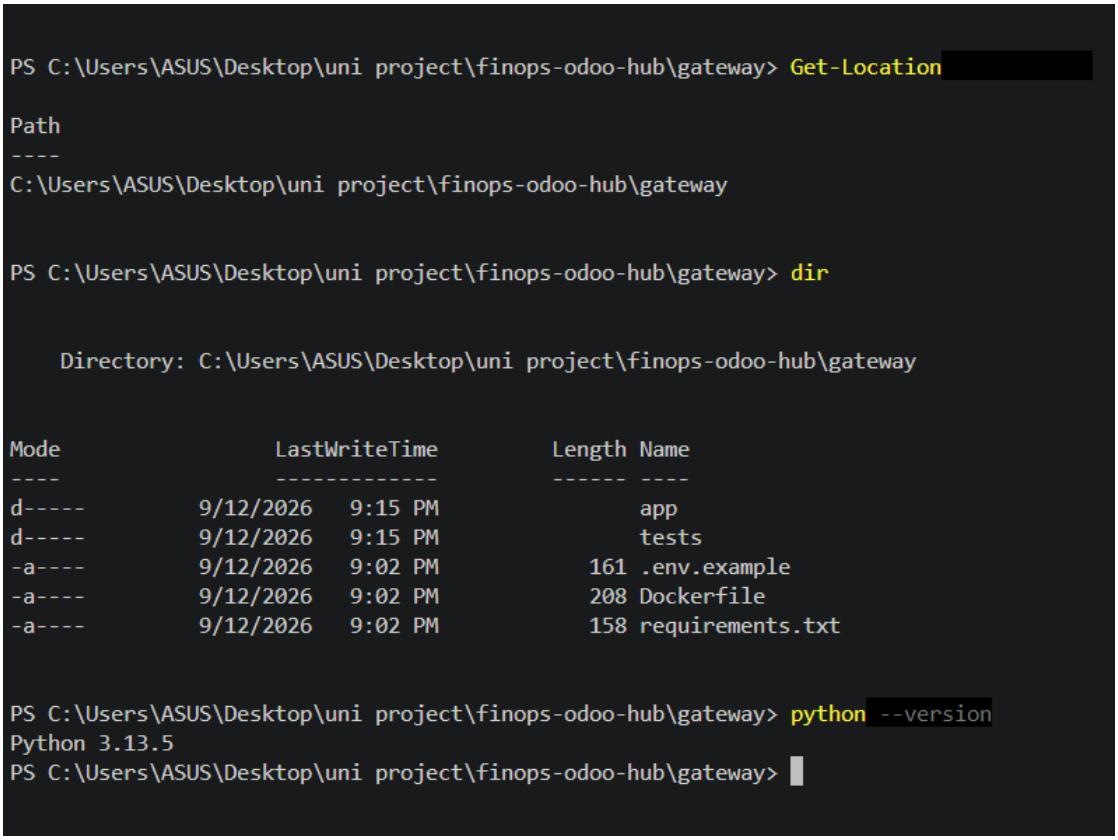
فصل 10

ضمیمه تصویری - تمام شواهد اجرایی ثبت‌شده

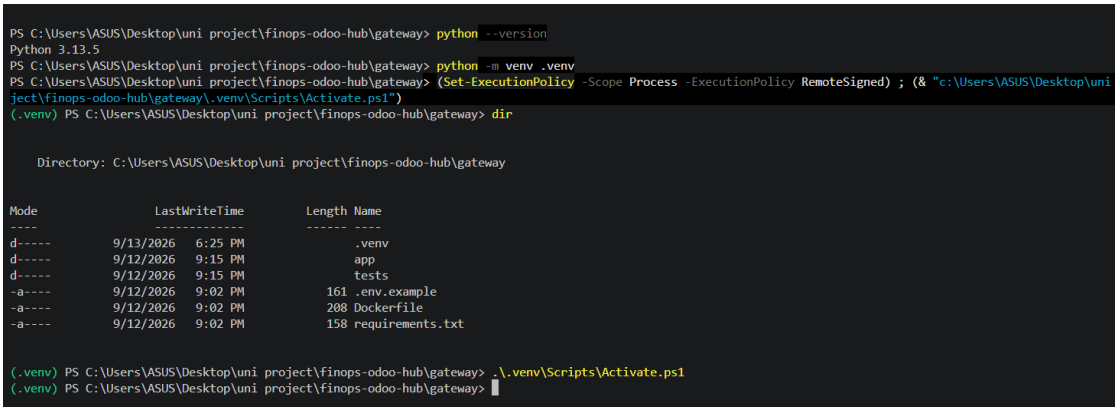
برای جلوگیری از حذف هیچ‌یک از شواهد، تصاویر باقی‌مانده‌ای که در فصل‌های قبل به‌صورت مستقل استفاده نشده‌اند نیز در این ضمیمه درج شده‌اند. برخی تصاویر ادامه همان خطای مرحله قبل هستند و برای ثبت کامل روند نگه‌داری شده‌اند.



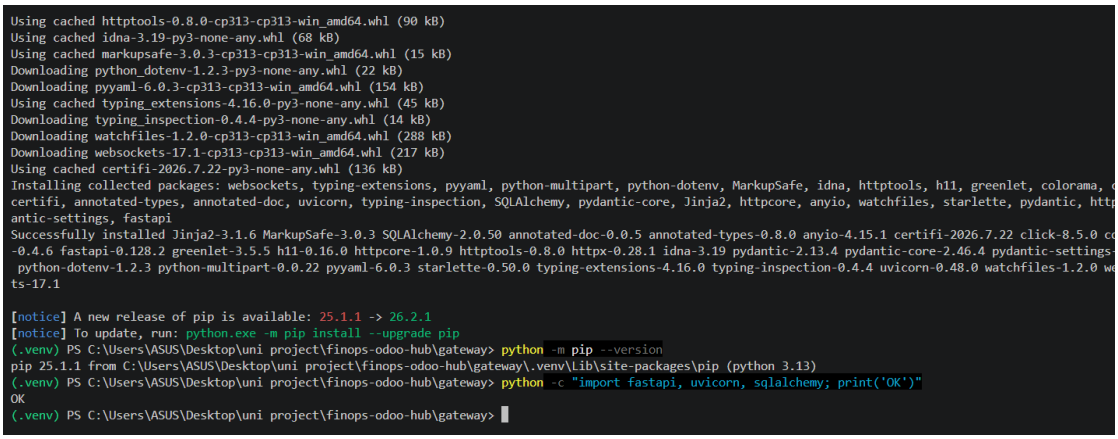
شکل 46 - گالری کامل - 13-09-2026 182229



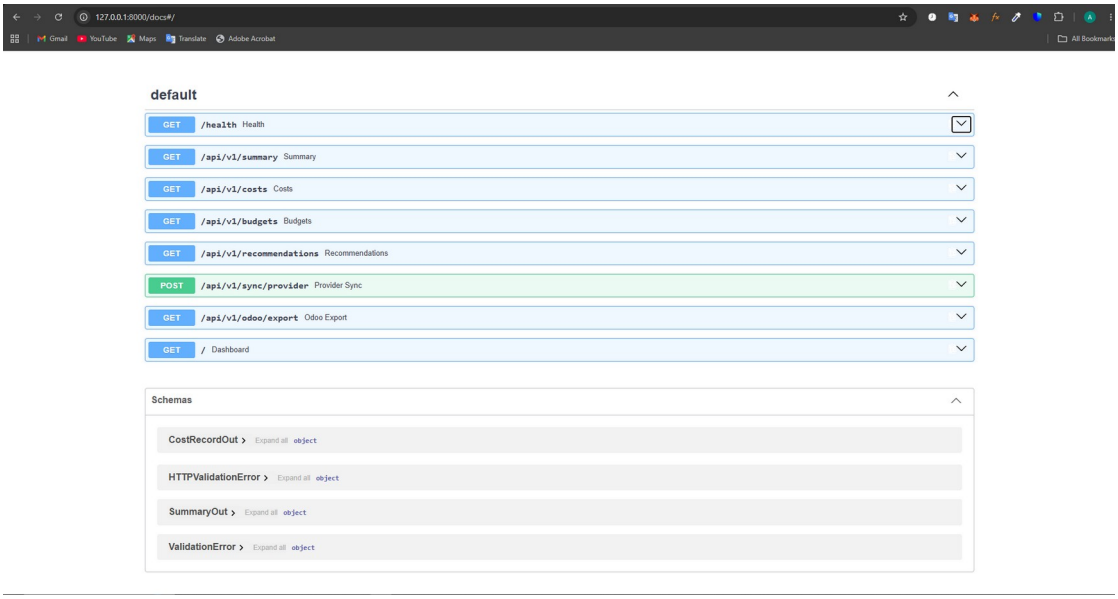
شکل 47 - گالری کامل - 13-09-2026 182438



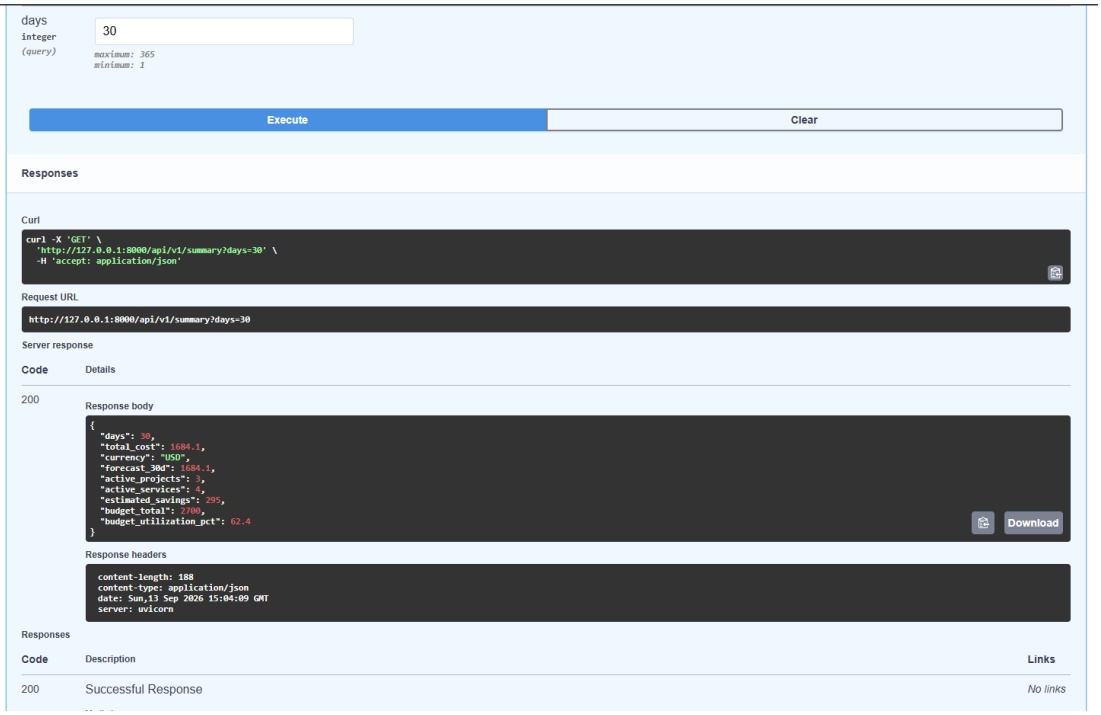
شکل 48 - گالری کامل - 182637 13-09-2026



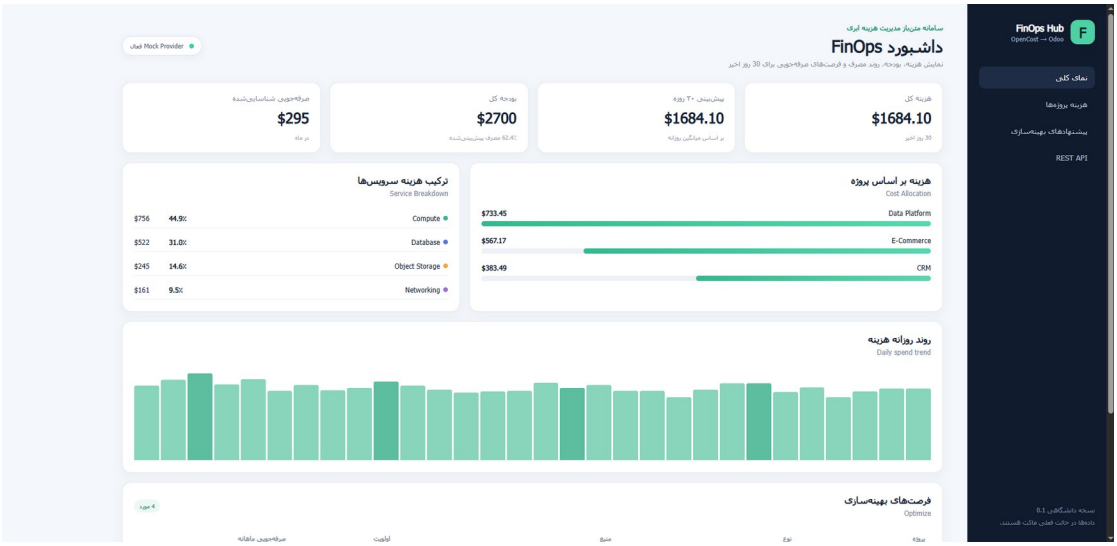
شکل 49 - گالری کامل - 183035 13-09-2026



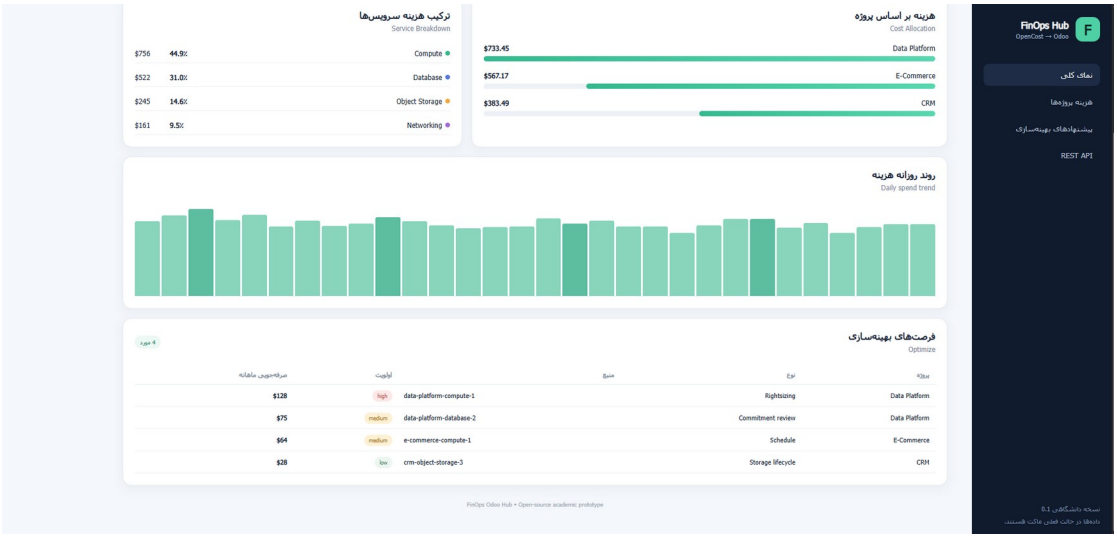
شکل 50 - گالری کامل - 183207 13-09-2026



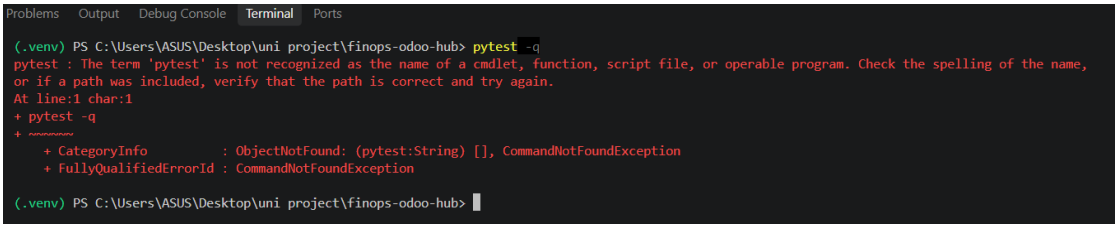
شکل 51 - گالری کامل - 183428 13-09-2026



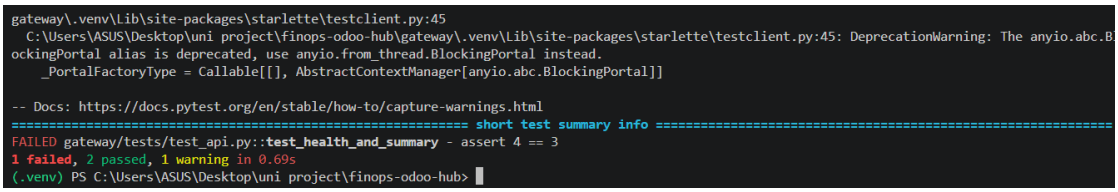
شکل 52 - گالری کامل - 13-09-2026 183725



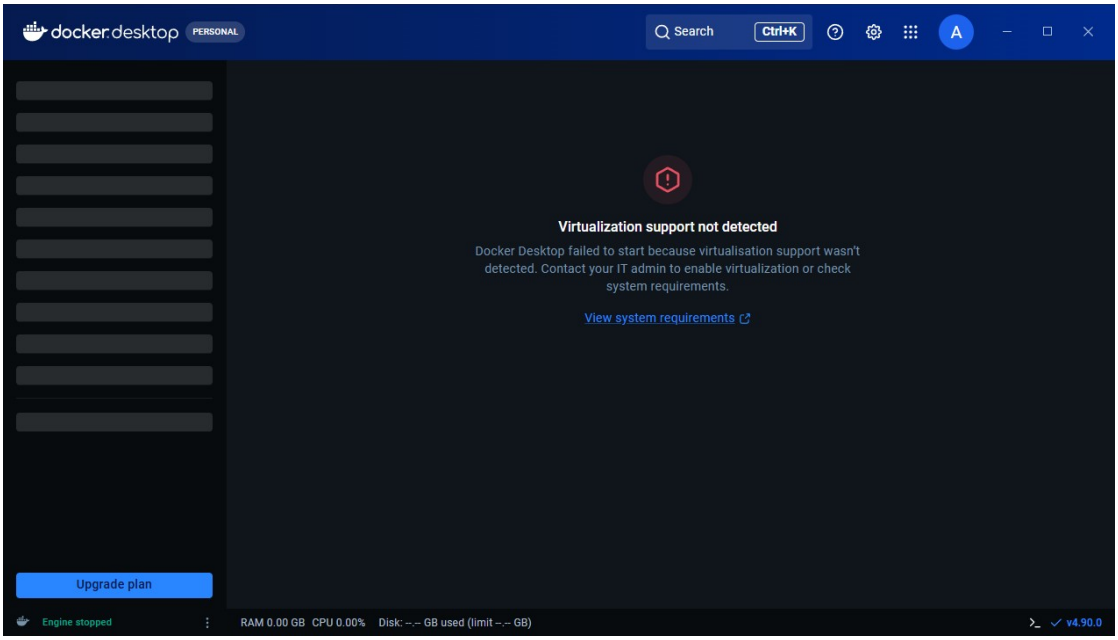
شکل 53 - گالری کامل - 13-09-2026 183735



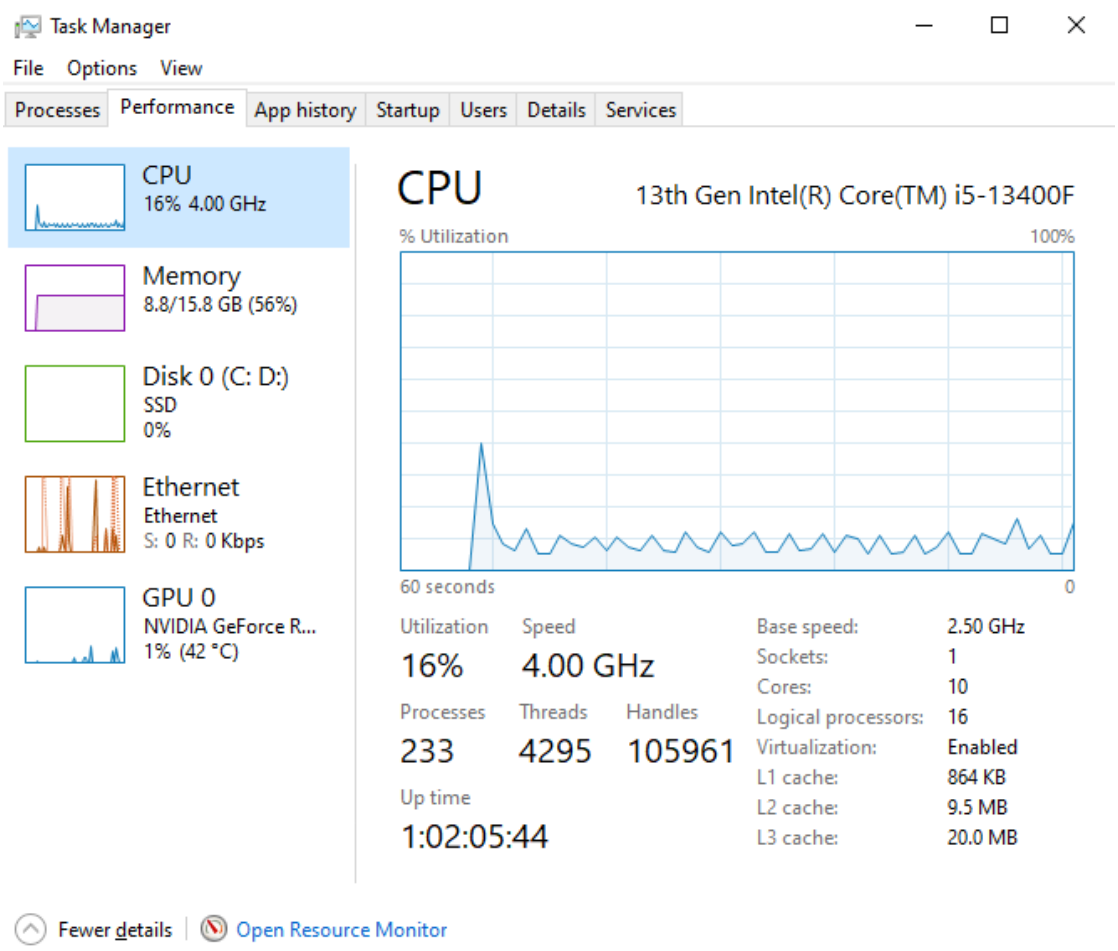
شکل 54 - گالری کامل - 202608 13-09-2026



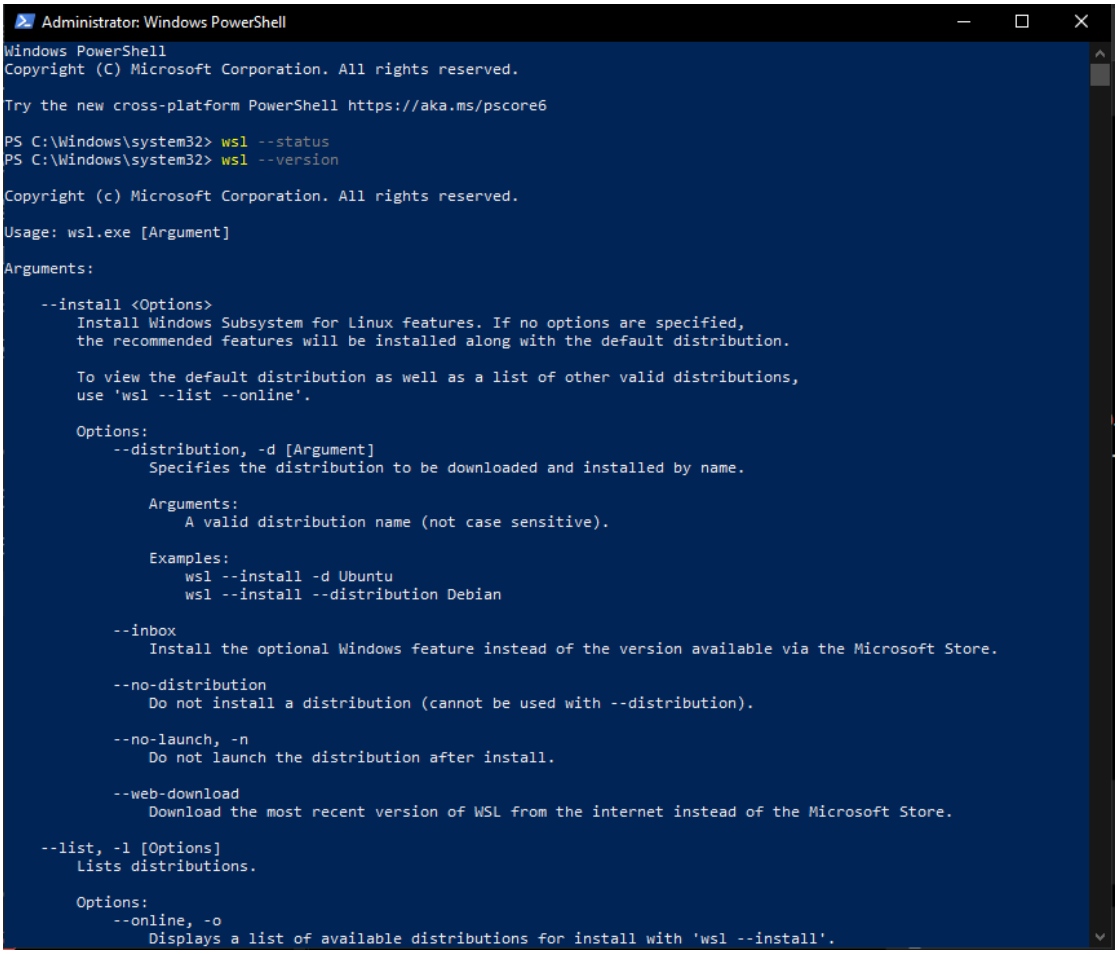
شکل 55 - گالری کامل - 202858 13-09-2026



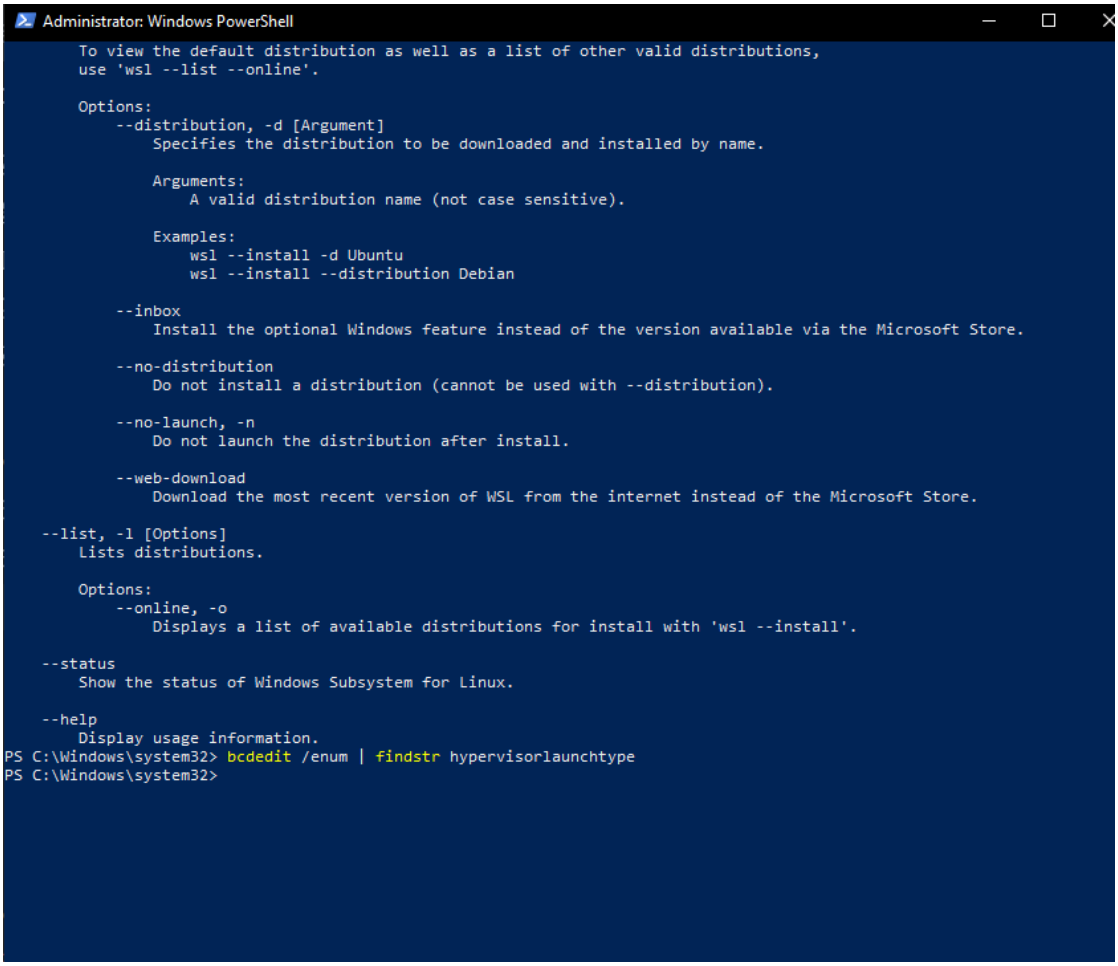
شکل 56 - گالری کامل - 204056 13-09-2026



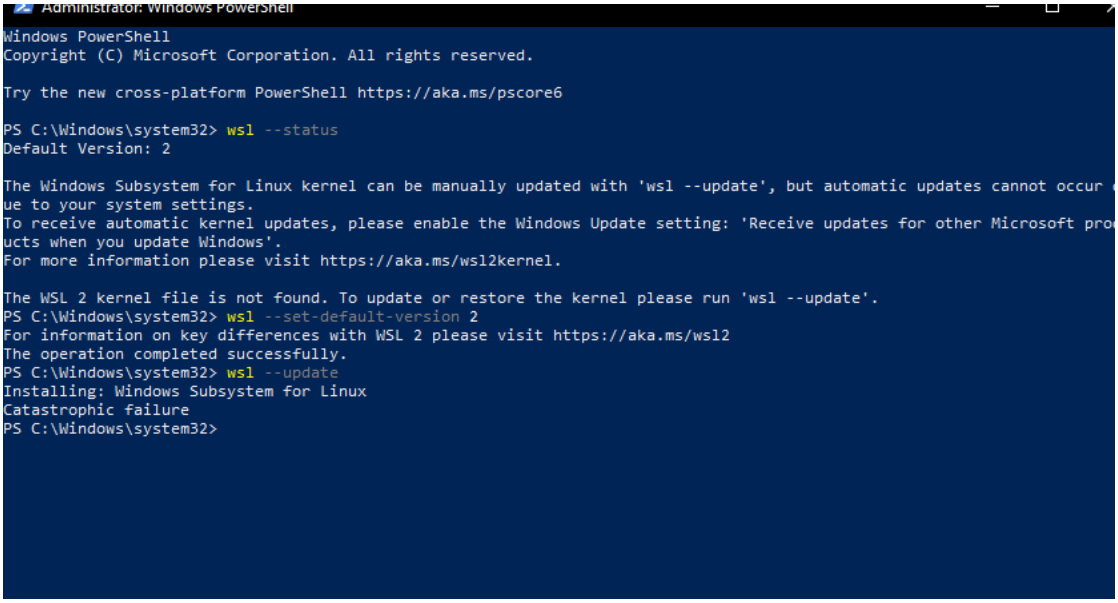
شکل 57 - گالری کامل - 204243 13-09-2026



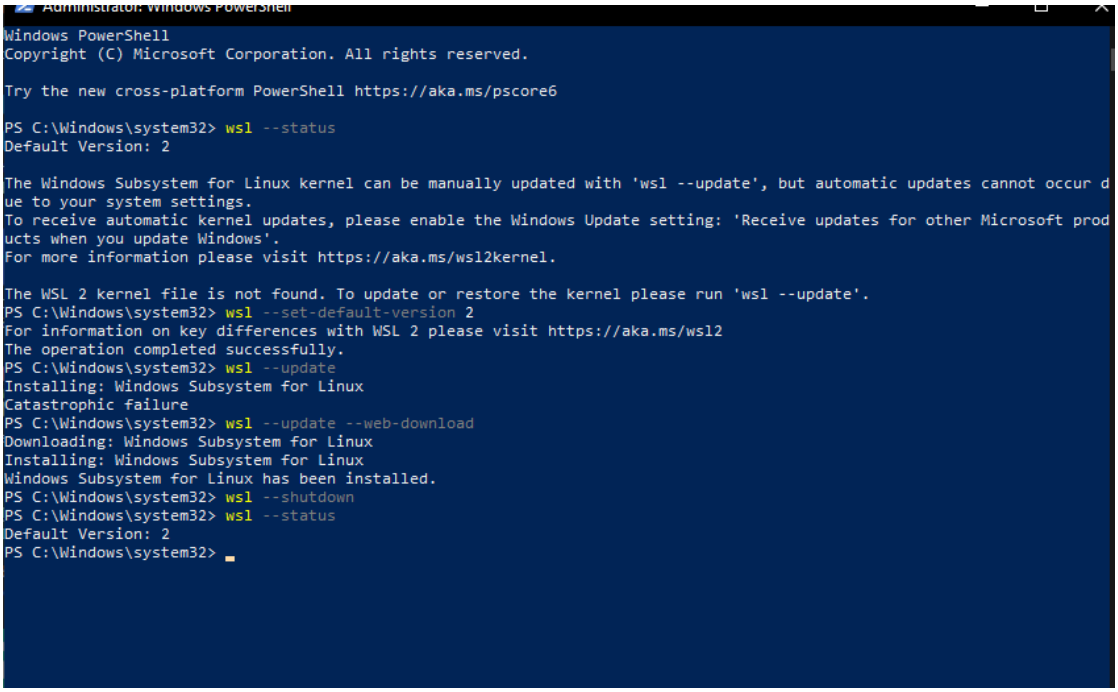
شکل 58 - گالری کامل - 13-09-2026 204628



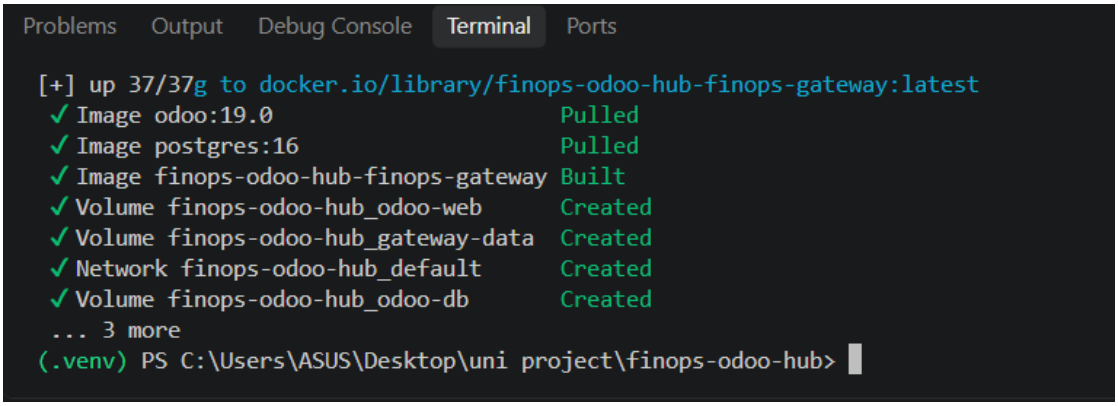
شکل 59 - گالری کامل - 204639 13-09-2026



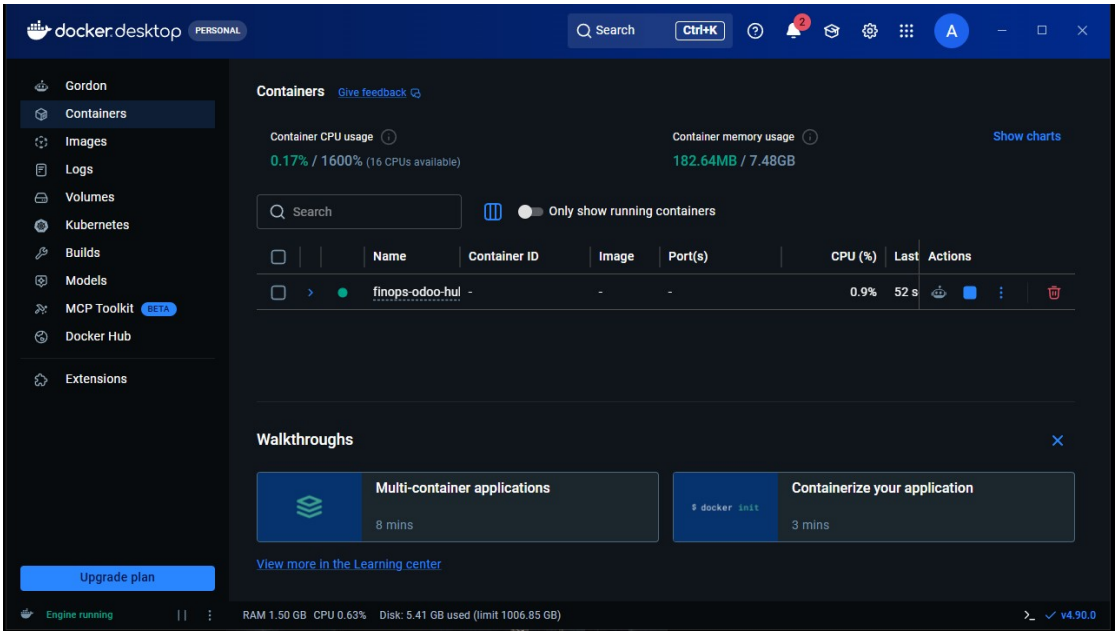
شکل 60 - گالری کامل - 13-09-2026 205332



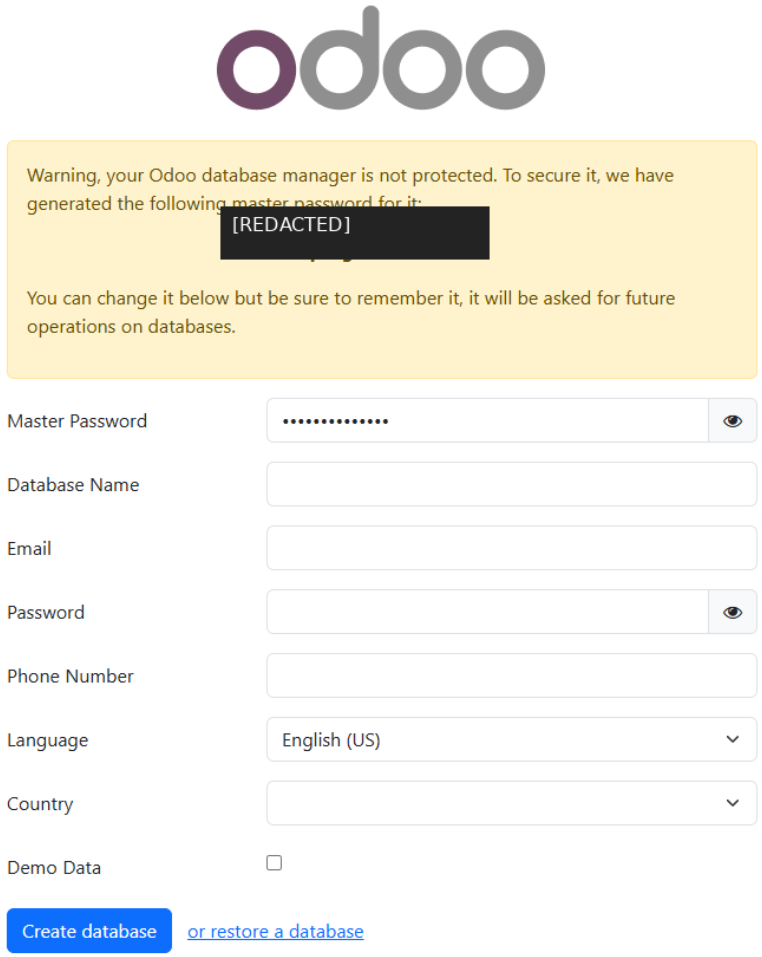
شکل 61 - گالری کامل - 13-09-2026 215909



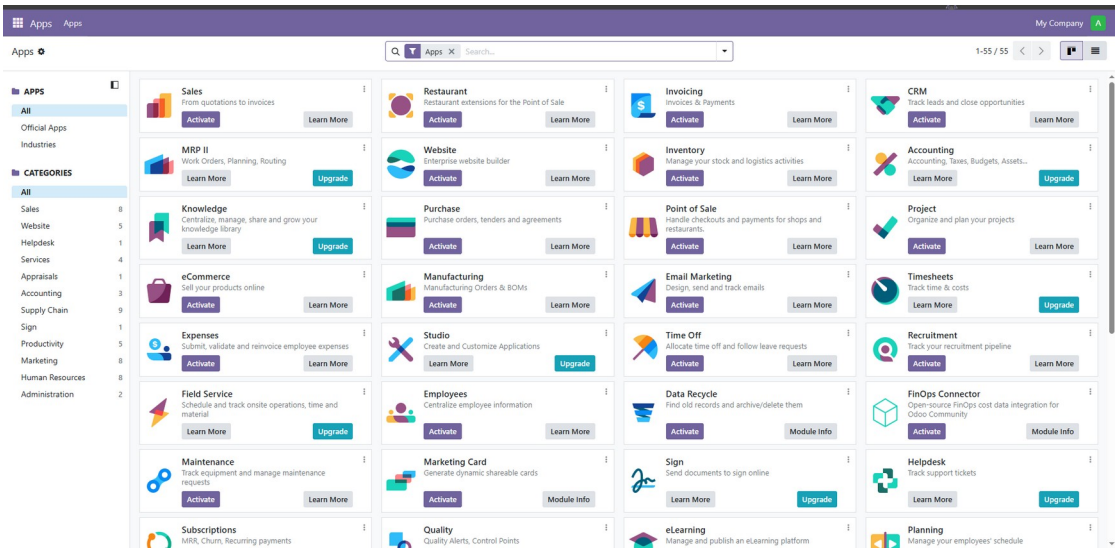
شکل 62 - گالری کامل - 233350 13-09-2026



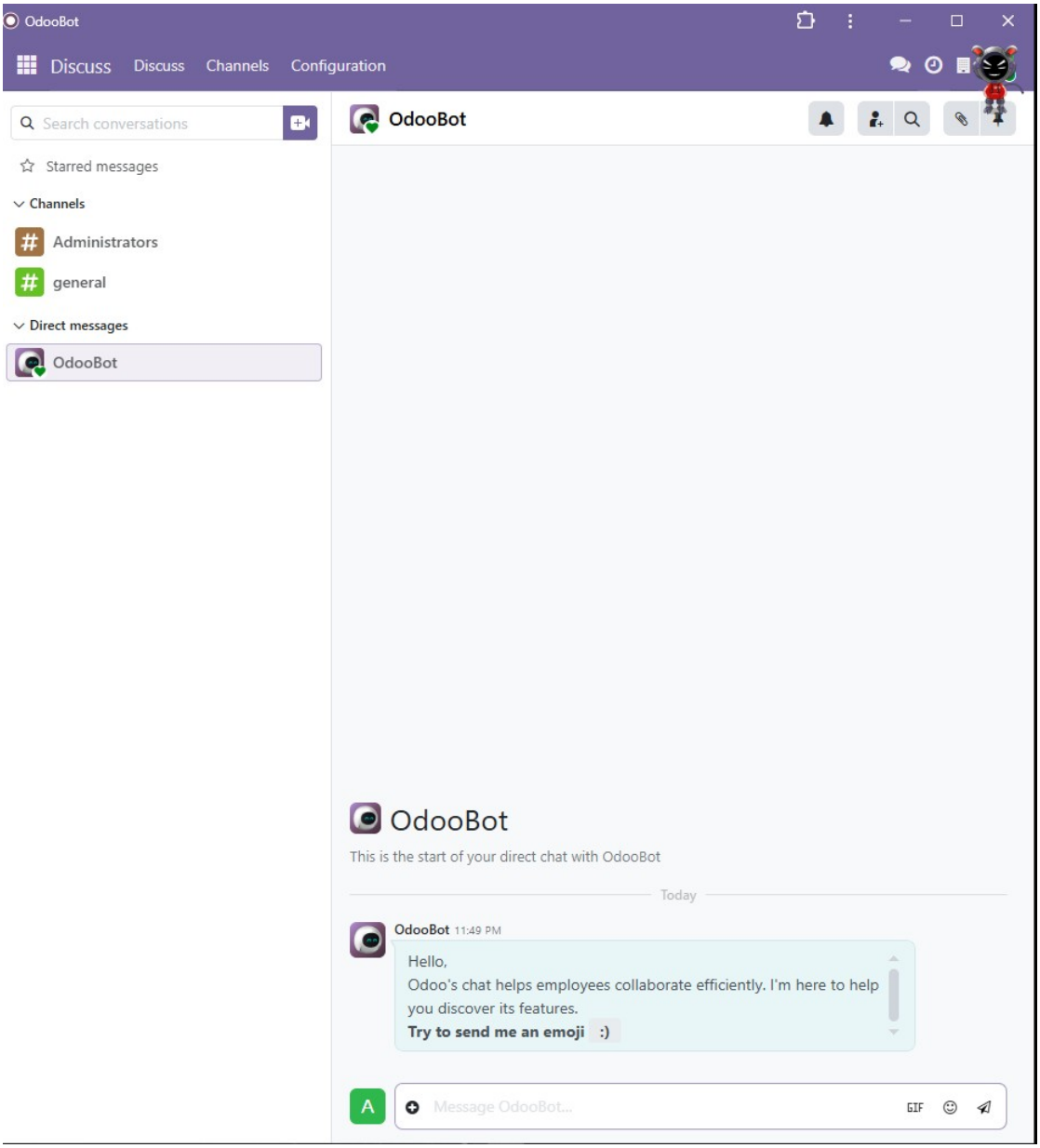
شکل 63 - گالری کامل - 233409 13-09-2026



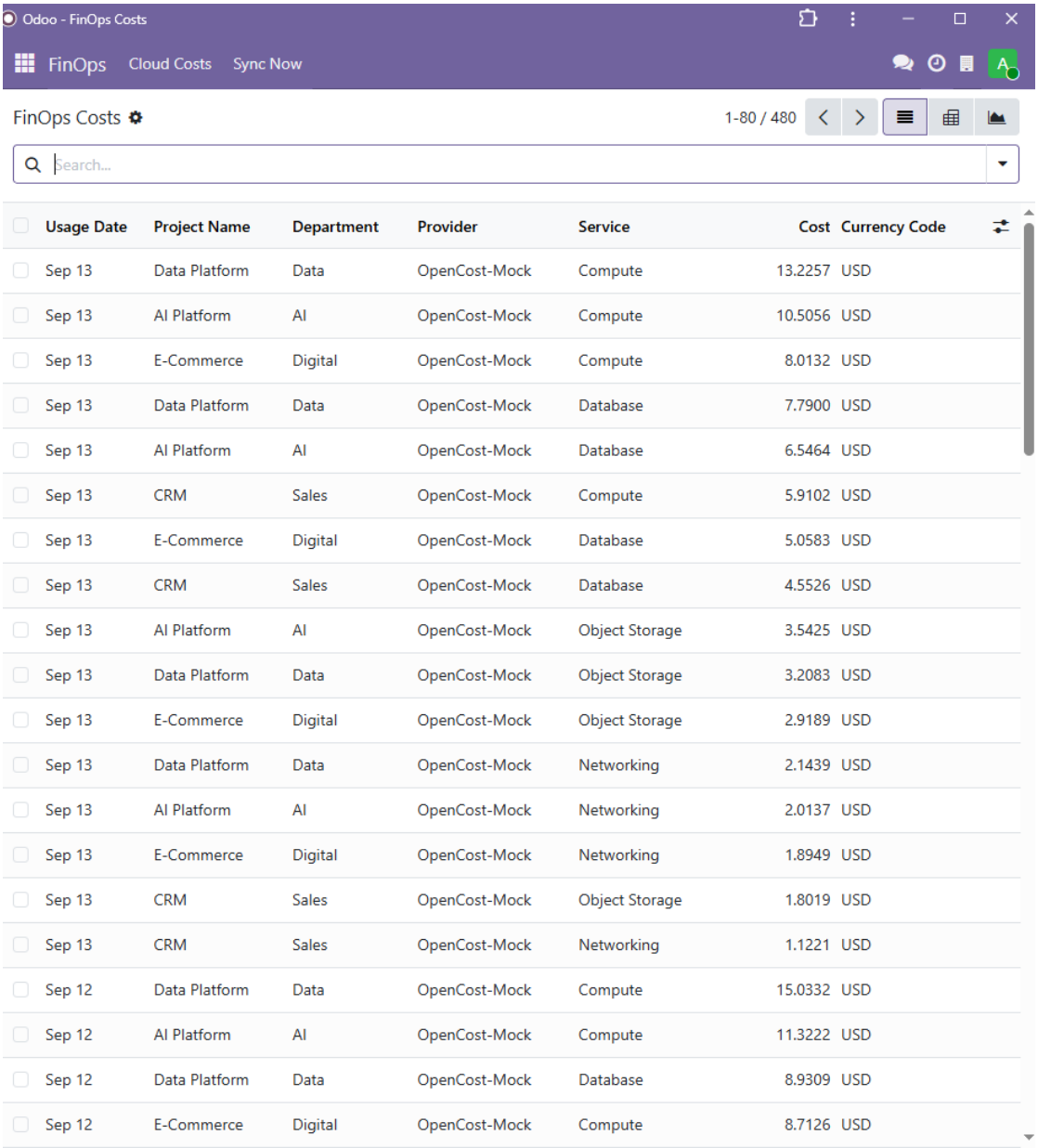
شکل 64 - گالری کامل - صفحه ساخت دیتابیس **Odoo** (رمز حذف شده)



شکل 65 - گالری کامل - 234321 13-09-2026

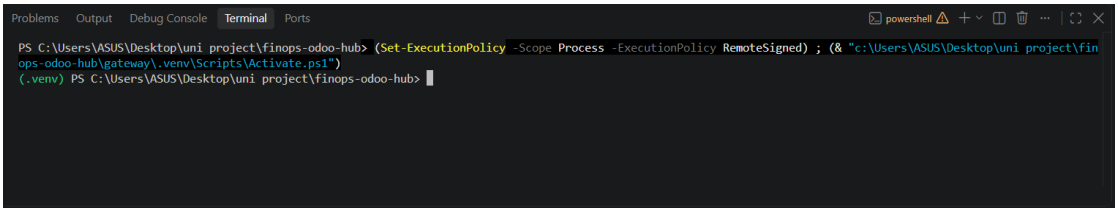


شکل 66 - گالری کامل - 13-09-2026 235031

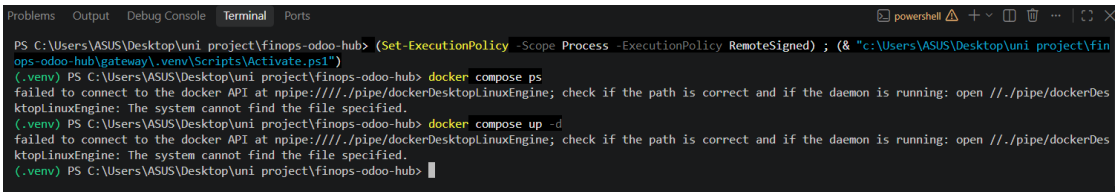


Usage Date	Project Name	Department	Provider	Service	Cost	Currency Code
Sep 13	Data Platform	Data	OpenCost-Mock	Compute	13.2257	USD
Sep 13	AI Platform	AI	OpenCost-Mock	Compute	10.5056	USD
Sep 13	E-Commerce	Digital	OpenCost-Mock	Compute	8.0132	USD
Sep 13	Data Platform	Data	OpenCost-Mock	Database	7.7900	USD
Sep 13	AI Platform	AI	OpenCost-Mock	Database	6.5464	USD
Sep 13	CRM	Sales	OpenCost-Mock	Compute	5.9102	USD
Sep 13	E-Commerce	Digital	OpenCost-Mock	Database	5.0583	USD
Sep 13	CRM	Sales	OpenCost-Mock	Database	4.5526	USD
Sep 13	AI Platform	AI	OpenCost-Mock	Object Storage	3.5425	USD
Sep 13	Data Platform	Data	OpenCost-Mock	Object Storage	3.2083	USD
Sep 13	E-Commerce	Digital	OpenCost-Mock	Object Storage	2.9189	USD
Sep 13	Data Platform	Data	OpenCost-Mock	Networking	2.1439	USD
Sep 13	AI Platform	AI	OpenCost-Mock	Networking	2.0137	USD
Sep 13	E-Commerce	Digital	OpenCost-Mock	Networking	1.8949	USD
Sep 13	CRM	Sales	OpenCost-Mock	Object Storage	1.8019	USD
Sep 13	CRM	Sales	OpenCost-Mock	Networking	1.1221	USD
Sep 12	Data Platform	Data	OpenCost-Mock	Compute	15.0332	USD
Sep 12	AI Platform	AI	OpenCost-Mock	Compute	11.3222	USD
Sep 12	Data Platform	Data	OpenCost-Mock	Database	8.9309	USD
Sep 12	E-Commerce	Digital	OpenCost-Mock	Compute	8.7126	USD

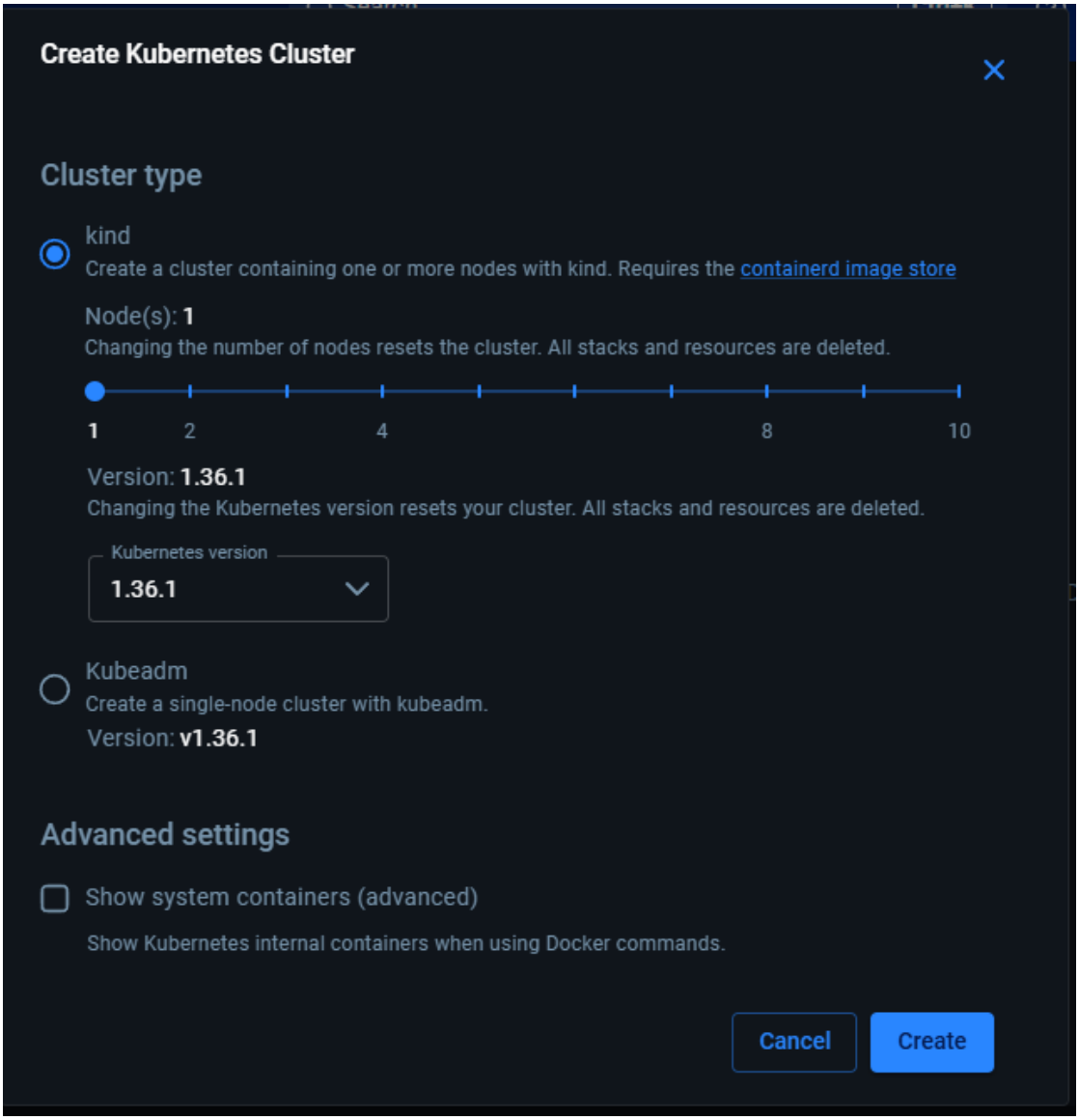
شکل 67 - گالری کامل - 13-09-2026 235229



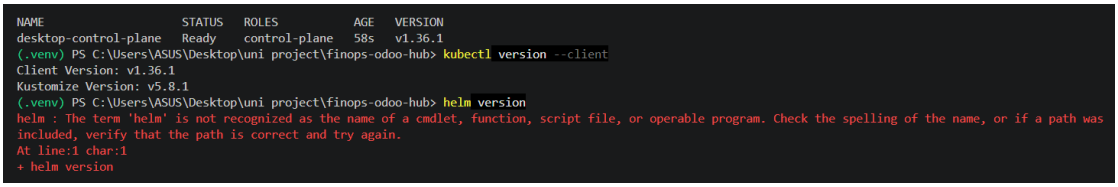
شکل 68 - گالری کامل - 14-09-2026 164746



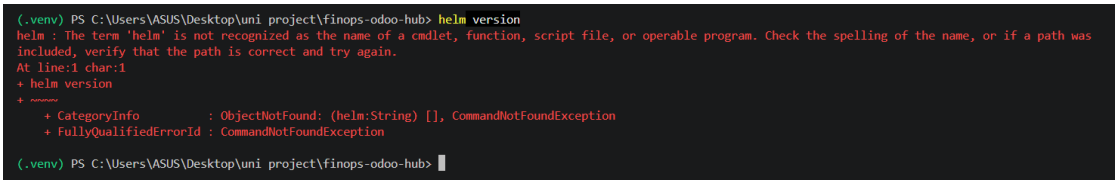
شکل 69 - گالری کامل - 14-09-2026 165442



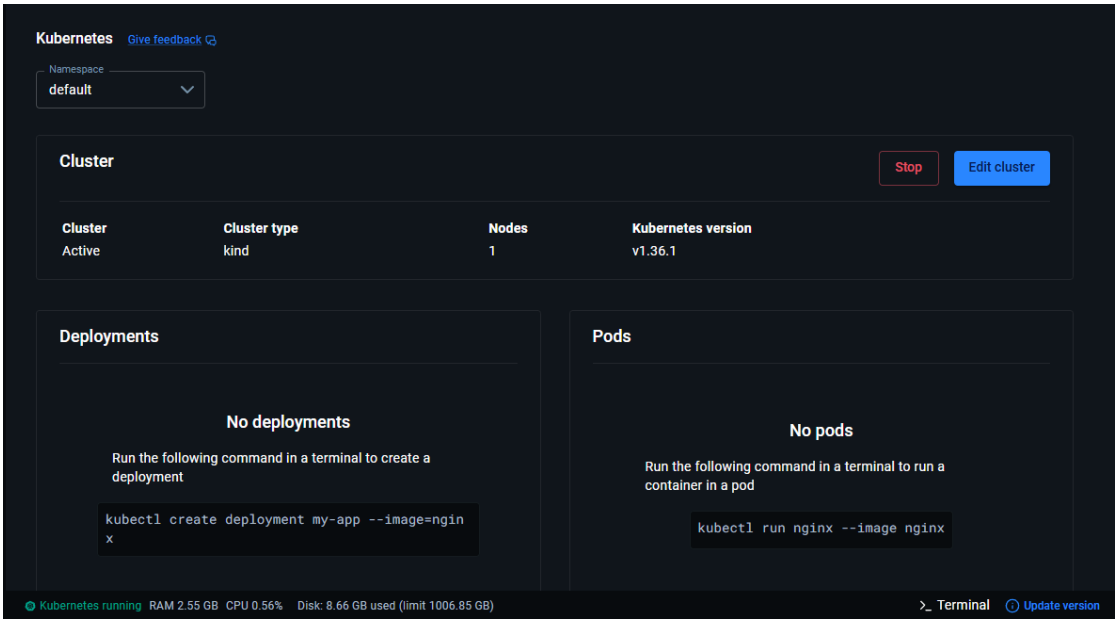
شکل 70 - گالری کامل - 171754 14-09-2026



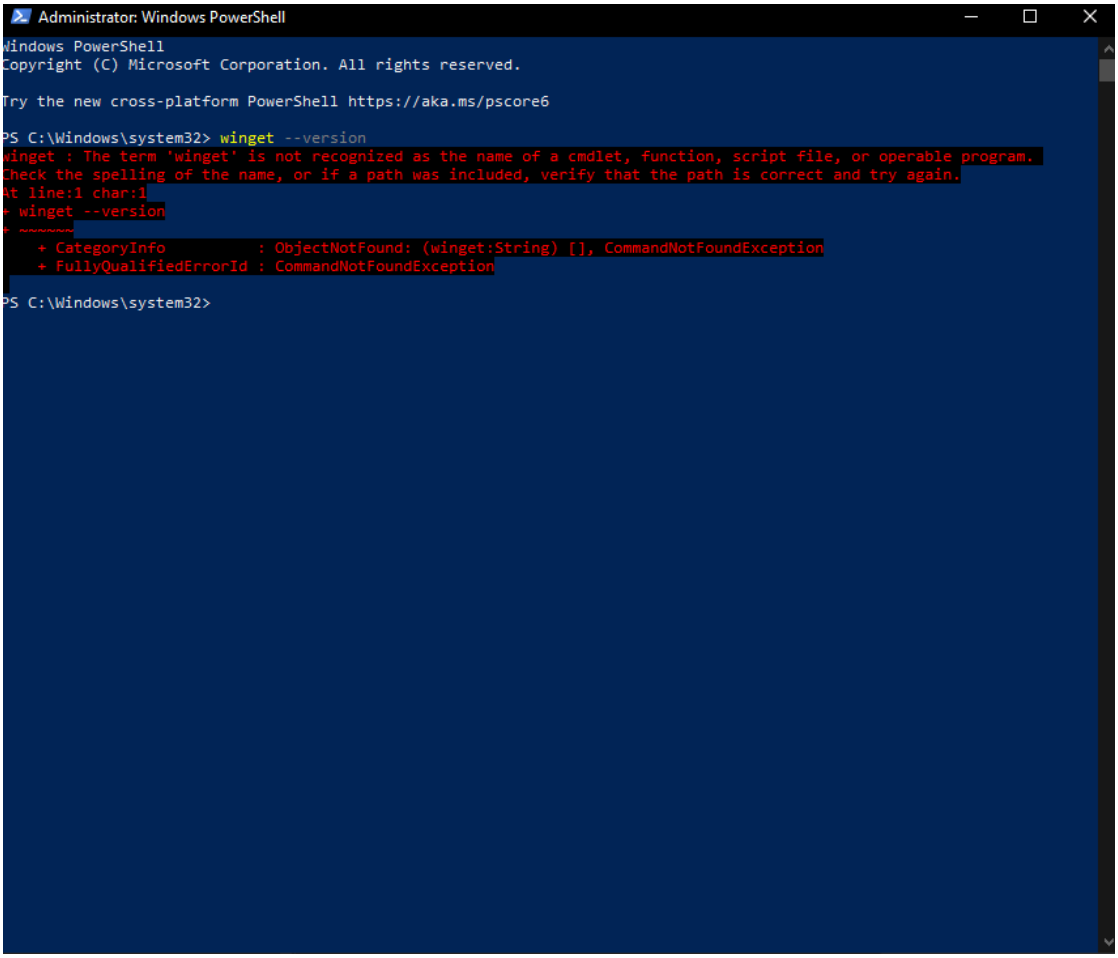
شکل 71 - گالری کامل - 173904 14-09-2026



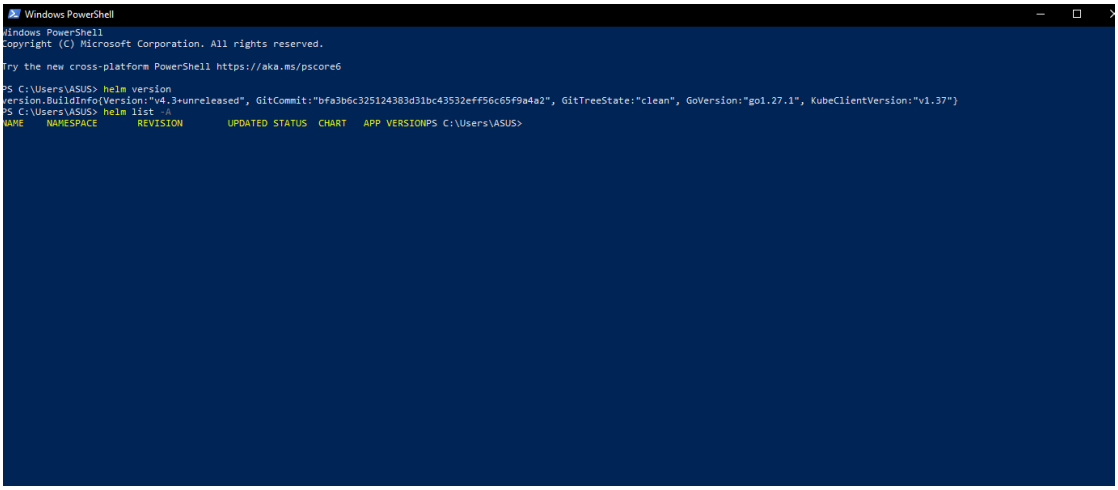
شکل 72 - گالری کامل - 173914 14-09-2026



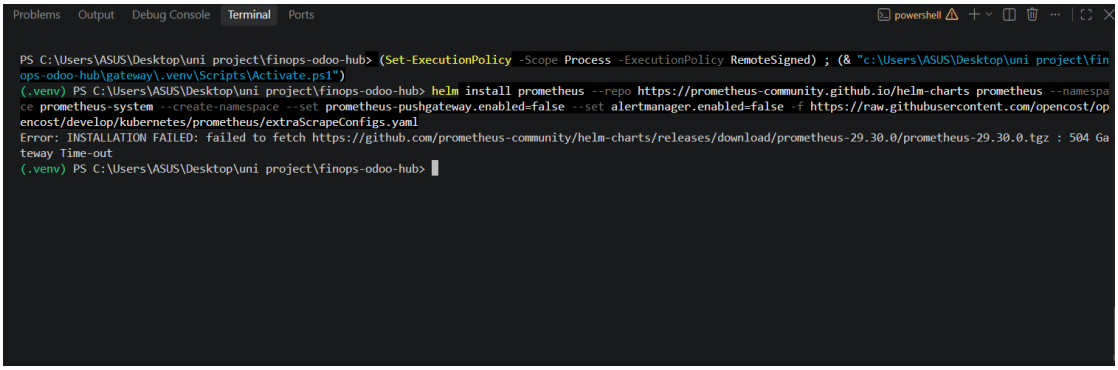
شکل 73 - گالری کامل - 173931 14-09-2026



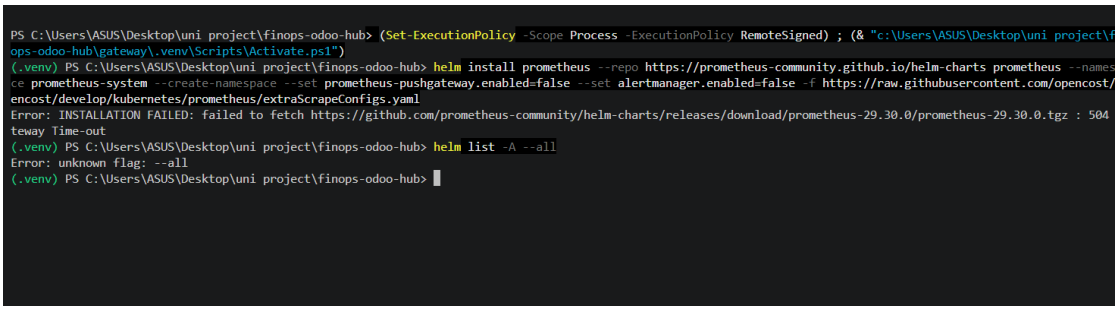
شکل 74 - گالری کامل - 14-09-2026 174721



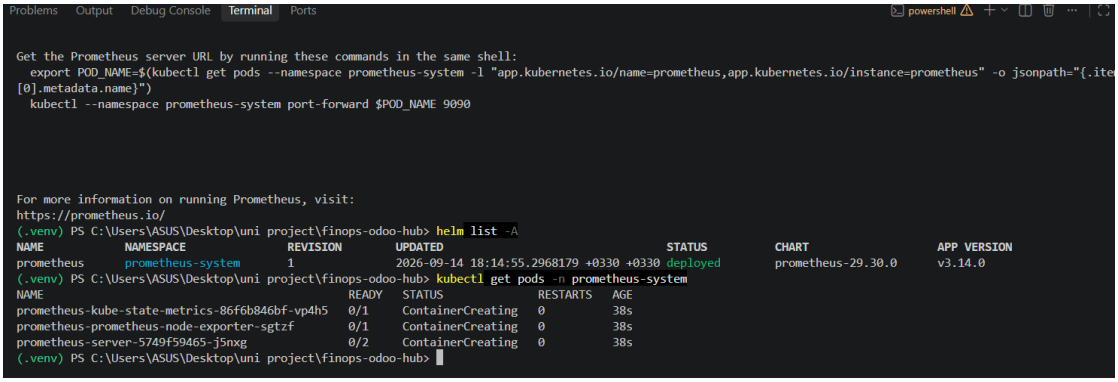
شکل 75 - گالری کامل - 14-09-2026 180154



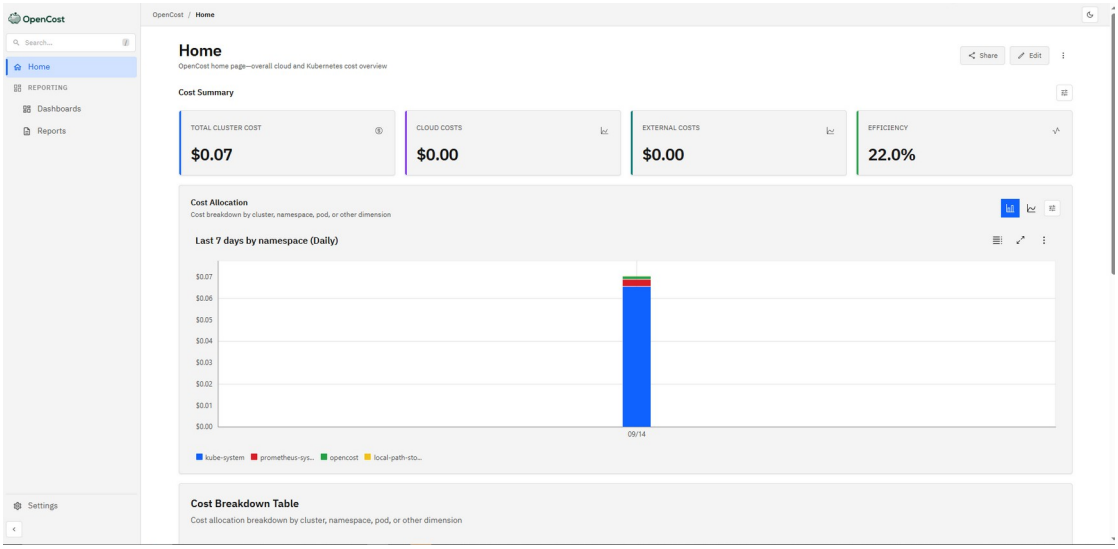
شکل 76 - گالری کامل - 14-09-2026 181106



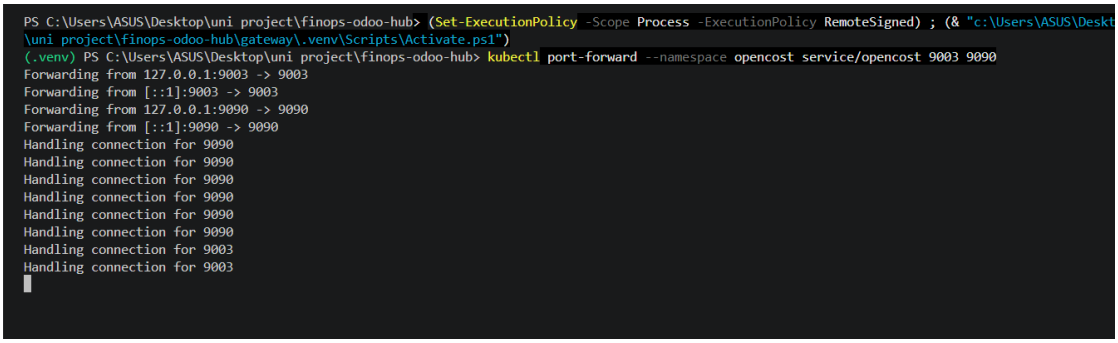
شکل 77 - گالری کامل - 14-09-2026 181241



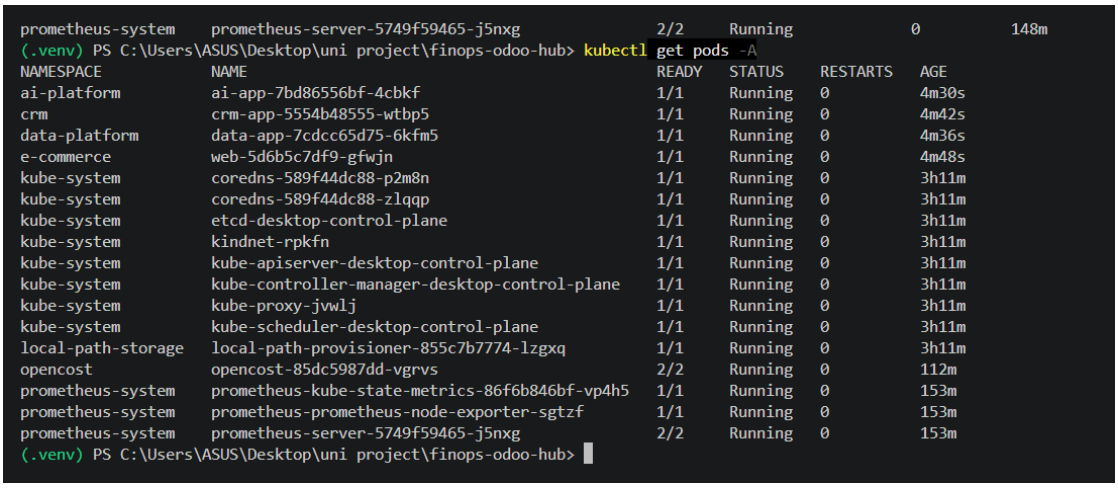
شکل 78 - گالری کامل - 14-09-2026 181608



شکل 79 - گالری کامل - 14-09-2026 202655



شکل 80 - گالری کامل - 14-09-2026 202749



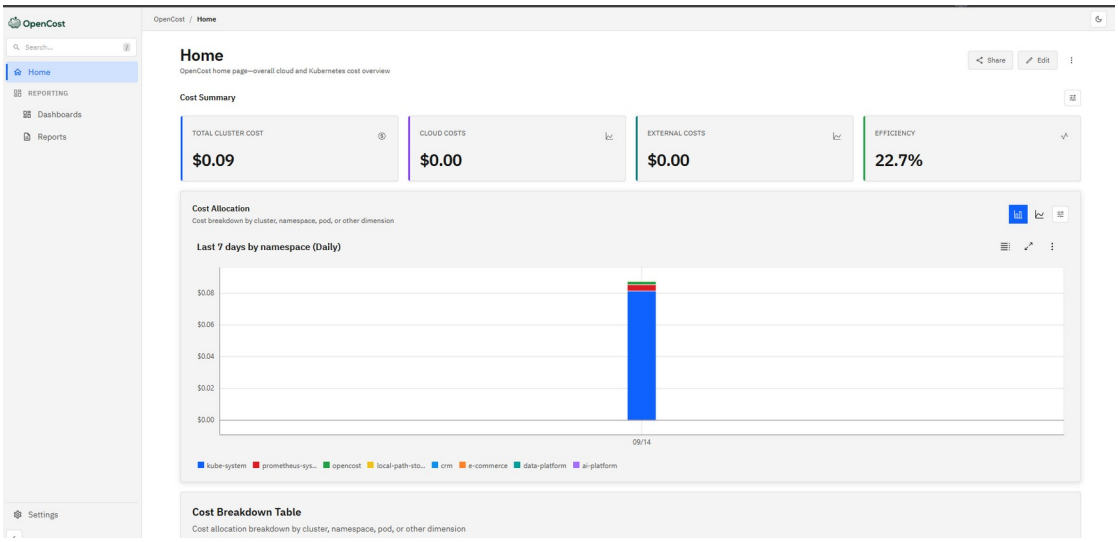
شکل 81 - گالری کامل - 14-09-2026 204825

```
200
(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub> $r.data[0].PSObject.Properties |
>> Where-Object { $_.Name -in @("e-commerce","data-platform","crm","ai-platform") } |
>> ForEach-Object {
>>     [PSCustomObject]@{
>>         Namespace = $_.Name
>>         TotalCost = [math]::Round($_.Value.totalCost, 6)
>>         CPUCost   = [math]::Round($_.Value.cpuCost, 6)
>>         RAMCost   = [math]::Round($_.Value.ramCost, 6)
>>     }
>> } | Format-Table

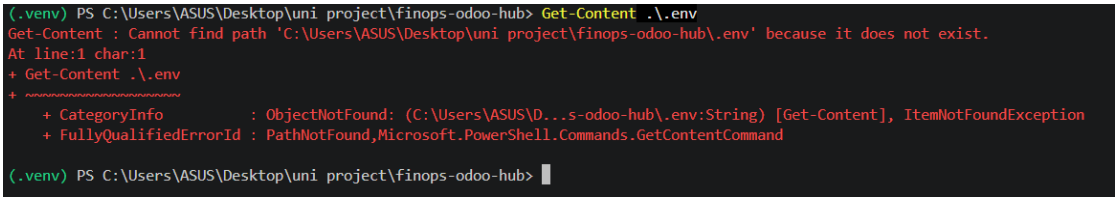
Namespace     TotalCost CPUCost RAMCost
---------     --------- ------- -------
ai-platform           0       0       0
crm                   0       0       0
data-platform         0       0       0
e-commerce      0.00001       0       0


(.venv) PS C:\Users\ASUS\Desktop\uni project\finops-odoo-hub>
```

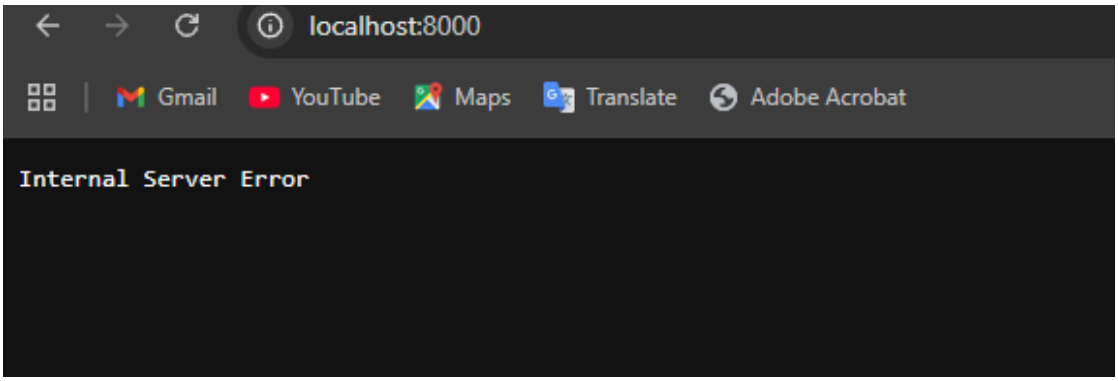
شکل 82 - گالری کامل - 205541 14-09-2026



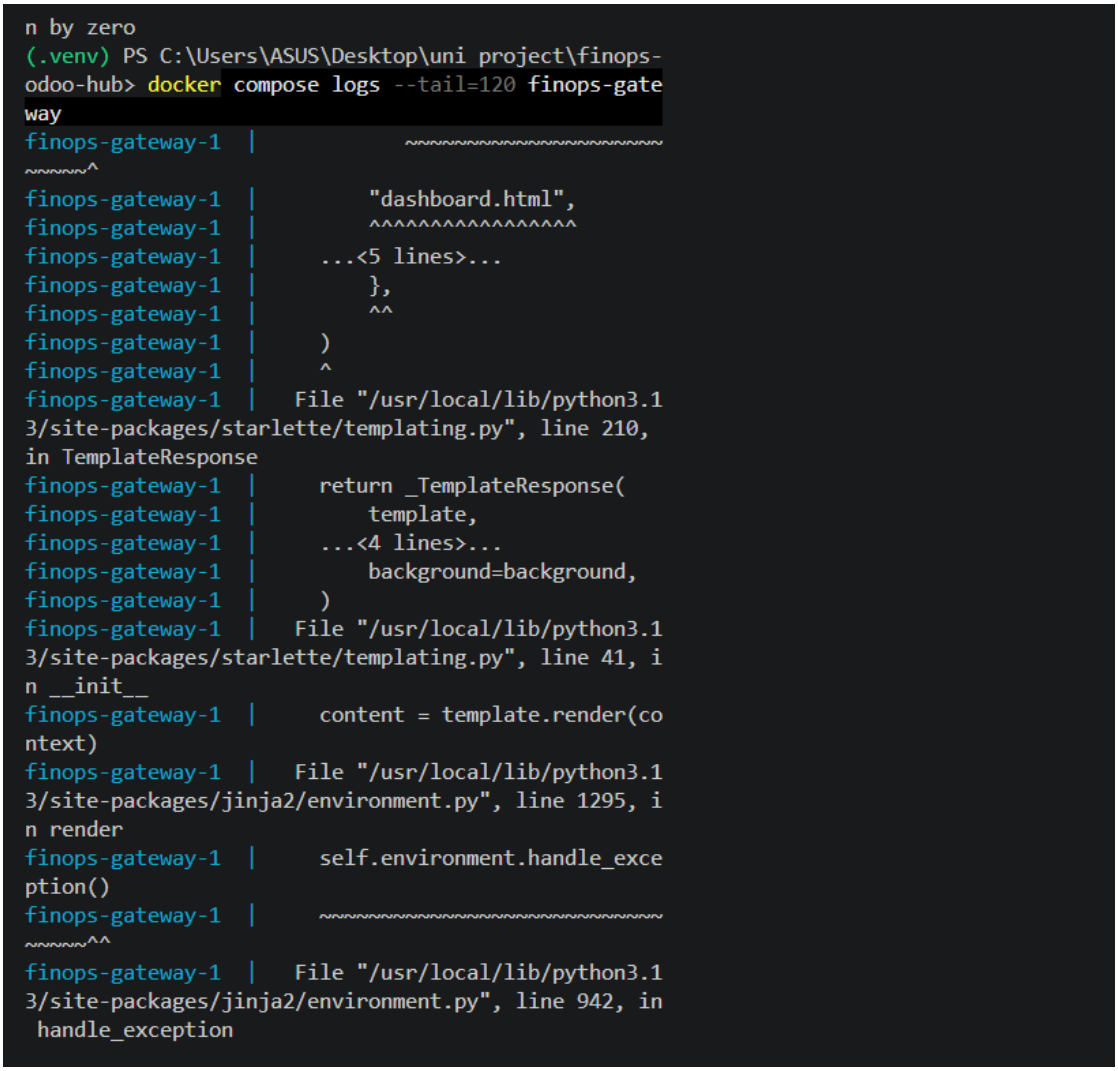
شکل 83 - گالری کامل - 205552 14-09-2026



شکل 84 - گالری کامل - 14-09-2026 212251



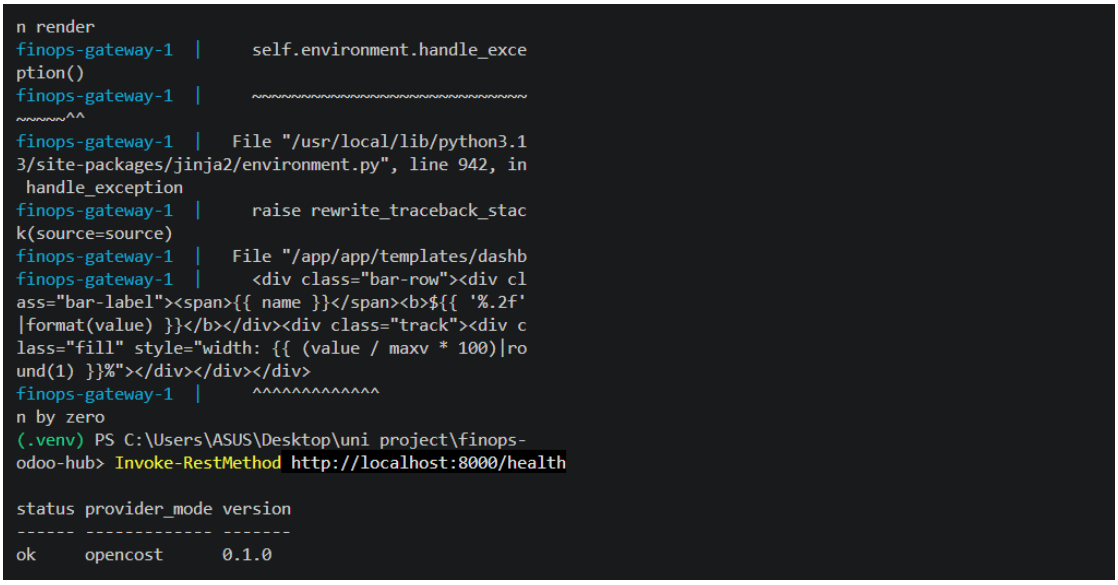
شکل 85 - گالری کامل - 14-09-2026 215601



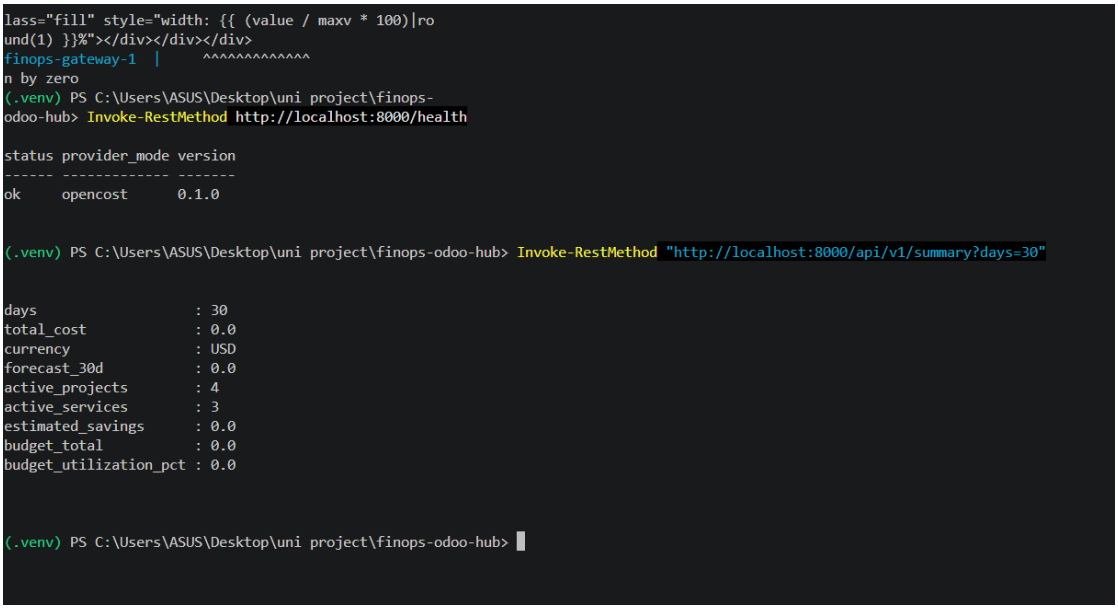
شکل 86 - گالری کامل - 14-09-2026 221346

```
finops-gateway-1  |     content = template.render(co
ntext)
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/jinja2/environment.py", line 1295, i
n render
finops-gateway-1  |     self.environment.handle_exce
ption()
finops-gateway-1  |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~^^
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/jinja2/environment.py", line 942, in
 handle_exception
finops-gateway-1  |     raise rewrite_traceback_stac
k(source=source)
finops-gateway-1  |   File "/app/app/templates/dashb
oard.html", line 34, in top-level template code
finops-gateway-1  |     <div class="bar-row"><div cl
ass="bar-label"><span>{{ name }}</span><b>${{ '%.2f'
|format(value) }}</b></div><div class="track"><div c
lass="fill" style="width: {{ (value / maxv * 100)|ro
und(1) }}%"></div></div></div>
finops-gateway-1  |     ^^^^^^^^^^^^
finops-gateway-1  | ZeroDivisionError: float divisio
n by zero
finops-gateway-1  | INFO:     172.18.0.1:35984 - "GE
T / HTTP/1.1" 500 Internal Server Error
finops-gateway-1  | ERROR:    Exception in ASGI appl
ication
finops-gateway-1  | Traceback (most recent call last
):
finops-gateway-1  |   File "/usr/local/lib/python3.1
3/site-packages/uvicorn/protocols/http/httptools_imp
l.py", line 421, in run_asgi
finops-gateway-1  |     result = await app(  # type:
 ignore[func-returns-value]
finops-gateway-1  |              ^^^^^^^^^^^^^^^^^^^
```

شکل 87 - گالری کامل - 221408 14-09-2026



شکل 88 - گالری کامل - 221439 14-09-2026



شکل 89 - گالری کامل - 221448 14-09-2026

Cheat Sheet و Runbook، پیوست دستورات

11-1. اجرای کامل از خاموش بودن سیستم تا Demo

```
# 1. Docker Desktop را باز کن و صبر کن Engine running شود.

# 2. در ریشه پروژه
Set-Location "C:/Users/ASUS/Desktop/uni project/finops-odoo-hub"
docker compose up -d

# 3. Kubernetes
kubectl get nodes
kubectl get pods -A

# 4. OpenCost Port Forward - این ترمینال باز بماند
kubectl port-forward --address=0.0.0.0 -n opencost service/opencost 9003:9003 9090:9090

# 5. در ترمینال دیگر
Invoke-RestMethod http://localhost:8000/health
Invoke-RestMethod -Method Post http://localhost:8000/api/v1/sync/provider

# 6. URLها
# Gateway: http://localhost:8000
# Swagger: http://localhost:8000/docs
# Odoo:    http://localhost:8069
# OpenCost:http://localhost:9090
# API:     http://localhost:9003
```

11-2. تست‌های قبل از ارائه

```
docker compose ps
kubectl get nodes
kubectl get pods -A
helm list -A

Invoke-RestMethod http://localhost:8000/health
Invoke-RestMethod "http://localhost:9003/allocation?window=1h&aggregate=namespace&resolution=1m"
Invoke-RestMethod -Method Post http://localhost:8000/api/v1/sync/provider

cd gateway
python -m pytest -q
cd ..
```

11-3. ترتیب خاموش‌کردن

```
# Port-forward را با Ctrl+C متوقف کن.

docker compose stop
```

```
# در صورت نیاز Kubernetes Cluster را از Docker Desktop Stop کن.
# برای حفظ تنظیمات، namespace های Reset را حذف نکن مگر عمداً قصد داشته باشی.
```

فصل 12

منابع رسمی و مبنای فنی

بخش مفهومی این راهنما بر روند واقعی اجرای پروژه و سورس پروژه استوار است. برای قابلیت‌های OpenCost و Odoo، منابع رسمی زیر مبنای طراحی مسیر تکمیل قرار گرفته‌اند:

/**OpenCost - Overview:** https://opencost.io/docs [1]

/**OpenCost - Prometheus prerequisite:** https://opencost.io/docs/installation/prometheus [2]

/**OpenCost - Helm installation:** https://opencost.io/docs/installation/helm [3]

/**OpenCost - API:** https://opencost.io/docs/integrations/api [4]

/**OpenCost - API Examples:** https://opencost.io/docs/integrations/api-examples [5]

/**OpenCost - Configuration / Custom Pricing:** https://opencost.io/docs/configuration [6]

/**OpenCost - Specification:** https://opencost.io/docs/specification [7]

Odoo 19 - Backend tutorials / building modules: [8]

https://www.odoo.com/documentation/19.0/developer/tutorials/backend.html

Odoo 19 - Actions and Scheduled Actions: [9]

https://www.odoo.com/documentation/19.0/developer/reference/backend/actions.html

Odoo 19 - View records: [10]

https://www.odoo.com/documentation/19.0/developer/reference/user_interface/view_records.html

جمع‌بندی نهایی

پروژه تا این نقطه از یک Mock Dashboard ساده عبور کرده و به یک زنجیره واقعی Kubernetes/Prometheus/OpenCost در کنار Odoo Community رسیده است. با اجرای فصل 7، خروجی نهایی باید یک Demo End-to-End باشد که در آن Workload واقعی Kubernetes هزینه تولید می‌کند، OpenCost آن را تخصیص می‌دهد، Gateway داده را نرمال می‌کند و Odoo آن را به Projectهای کسب‌وکاری متصل و تحلیل می‌کند. این خروجی هم برای ارائه دانشگاهی قابل دفاع است و هم معماری آن قابلیت توسعه به محیط واقعی سازمانی را دارد.

> قاعده ارائه
>
> هر جا داده از kind محلی یا Custom Pricing می‌آید، آن را «Cost Allocation محاسبه‌شده» بنام؛ فقط زمانی از عبارت «صورتحساب واقعی Cloud» استفاده کن که Cloud Billing Provider واقعاً متصل شده باشد.